

Go — compiled path

Compiled from /Users/darkonikolic/Documents/learning/paths/Go . Canonical sources stay in place.

Phase — 01-go-fundamentals-and-cli-thinking

Directory: 01-go-fundamentals-and-cli-thinking/

Source: 01-go-fundamentals-and-cli-thinking/00-quick-reference.md

Quick Reference — Go Fundamentals

Struct

```
type User struct {
    ID    int
    Name  string
    Age   int // zero value: 0
}
u := User{}           // all zero values
u := User{Name: "A"} // partial init
```

Value vs Pointer receiver

```
func (u User) Name() string { ... } // value — copy, safe for reads
func (u *User) SetName(s string) { u.Name = s } // pointer — mutates
```

Interface (implicit)

```
type Stringer interface { String() string }
// any type with String() method satisfies it automatically
```

Error handling

```
func divide(a, b int) (int, error) {
    if b == 0 {
        return 0, fmt.Errorf("divide by zero")
    }
    return a / b, nil
}
result, err := divide(10, 0)
if err != nil {
    log.Fatal(err)
}
```

Composition (not inheritance)

```
type Admin struct {
    User           // embedded — promotes fields/methods
    Permissions []string
}
```

Panic vs error

```
// panic: unrecoverable bug (nil deref, index OOB)
// error: expected failure path — always return and check
```

Source: `01-go-fundamentals-and-cli-thinking/01-scope-outcomes-cadence-and-go-lab.md`

Unit 1 — Module Scope: go-lab/ CLI

What You Will Build

A single CLI codebase called `go-lab/` that grows across every unit. No HTTP, no frameworks — just Go structs, methods, interfaces, and packages. By the end of this module you will have a working CLI that manages Users, Products, and Orders, and a second small binary `task-cli/` that creates, lists, completes, and deletes tasks.

The point is not the CLI itself. The point is that you stop reasoning “Go as PHP classes with a different syntax” and start reasoning in Go’s actual model: **structs hold data, methods attach behaviour to types, interfaces describe capabilities, errors are values, context carries cancellation.**

Mental Model Shift

Coming from class-based languages, the instinct is to ask “what class does this belong to?” Go’s question is different: “what type holds this data, and what behaviour does it need?” There is no inheritance, no constructor keyword, no `implements`. A type satisfies an interface simply by having the right methods. A zero value is valid until you decide it isn’t.

This shift takes deliberate practice, not just reading. Build the code. Hit the edge cases. When something breaks, understand why.

Module Goals (Checklist)

- [] Declare structs with exported and unexported fields; understand zero values
- [] Write value receivers (read) and pointer receivers (mutate); know when to use each
- [] Use `&` and `*` correctly; avoid nil pointer panics by guarding inputs
- [] Define and implement interfaces without an `implements` keyword
- [] Return and wrap errors as values; never use panic for normal error flow
- [] Organize code into packages with `cmd/` and `internal/` layout
- [] Initialize a module with `go mod init`; add and tidy dependencies
- [] Pass `context.Context` to functions that do slow or cancellable work
- [] Build capabilities through struct embedding, not inheritance
- [] Write table-driven tests with `go test ./...`

Spine Project: go-lab/

Carry these types forward across all units — each unit adds behaviour to them:

- `User` — ID, name, email
- `Product` — ID, name, price, stock
- `Order` — ID, user ID, product ID, quantity
- `Cart` — items, totals

Capstone (Unit 11): `task-cli/` binary with `create`, `list`, `delete`, `complete` subcommands. Uses structs, interfaces, explicit packages, `go.mod`, and `context` on any simulated slow work.

Unit 2 — Struct Thinking and Zero Values

Concept

Structs are Go's primary way to group related data. Every field has a zero value — `int` is 0, `string` is `""`, `bool` is `false`, pointer is `nil` — and Go initializes them automatically. This matters in production because it eliminates the “uninitialized variable” class of bugs common in other languages. Your job is to decide whether the zero value is valid for your type. If `Price == 0` means “free product” in your domain, fine. If it means “someone forgot to set a price”, enforce that with a constructor.

Code

```
package main

import "fmt"

type Product struct {
    ID      int
    Name    string
    Price   float64
    Stock   int
}

// IsAvailable returns true if the product can be purchased.
func (p Product) IsAvailable() bool {
    return p.Stock > 0 && p.Price > 0
}

// NewProduct is a constructor used because Price=0 and Stock<0 are invalid.
func NewProduct(id int, name string, price float64, stock int) (Product, error) {
    if price <= 0 {
        return Product{}, fmt.Errorf("NewProduct: price must be positive, got %g", price)
    }
    if stock < 0 {
        return Product{}, fmt.Errorf("NewProduct: stock cannot be negative, got %d", stock)
    }
    return Product{ID: id, Name: name, Price: price, Stock: stock}, nil
}

func main() {
    var p Product // zero value: ID=0 Name="" Price=0 Stock=0
    fmt.Printf("zero value: %v\n", p)
    fmt.Printf("zero IsAvailable: %v\n", p.IsAvailable()) // false — price is 0

    partial := Product{Name: "Widget"} // partial init — ID, Price, Stock are zero
    fmt.Printf("partial: %v\n", partial)

    good, err := NewProduct(1, "Gadget", 9.99, 10)
    if err != nil {
        fmt.Println("error:", err)
    } else {
        fmt.Printf("valid product: %v, available: %v\n", good, good.IsAvailable())
    }

    _, err = NewProduct(2, "Ghost", 0, 5) // price=0 rejected
    fmt.Println("invalid price:", err)

    _, err = NewProduct(3, "Broken", 5.0, -1) // negative stock rejected
    fmt.Println("invalid stock:", err)
}
```

Exercise

Build: A `Product` struct with `ID int`, `Name string`, `Price float64`, `Stock int`. Add an `IsAvailable() bool` method (pointer receiver is fine) and a `NewProduct()` constructor that validates inputs.

Input: Call `NewProduct` with valid args, with `price=0`, and with `stock=-1`.

Output: Valid product prints with `IsAvailable: true`. Invalid inputs return descriptive errors, not panics.

Acceptance: Write a `_test.go` file with at least three test cases (valid, zero price, negative stock). Run `go test ./...` — all pass.

Interview

- What is the zero value of a struct field, and why does Go guarantee it?
 - When should you write a constructor function (`NewX`) versus letting callers use struct literals directly?
 - What is the difference between `Product{}` and `var p Product` ?
-

Source: `01-go-fundamentals-and-cli-thinking/03-methods-value-and-pointer-receivers.md`

Unit 3 — Methods: Value and Pointer Receivers

Concept

A method with a value receiver operates on a copy of the struct — the original is unchanged. A method with a pointer receiver operates on the original. The rule is simple: if the method modifies state, use a pointer receiver. If it only reads, a value receiver is fine — but be consistent within a type. Mixing receiver types on the same struct causes subtle bugs and prevents clean interface satisfaction.


```

package main

import "fmt"

type CartItem struct {
    ProductID int
    Name      string
    Price      float64
    Quantity   int
}

type Cart struct {
    Items []CartItem
}

// AddItem uses a pointer receiver – it modifies the Cart's Items slice.
func (c *Cart) AddItem(item CartItem) {
    c.Items = append(c.Items, item)
}

// RemoveItem uses a pointer receiver – it modifies the Cart's Items slice.
func (c *Cart) RemoveItem(productID int) {
    filtered := c.Items[:0]
    for _, item := range c.Items {
        if item.ProductID != productID {
            filtered = append(filtered, item)
        }
    }
    c.Items = filtered
}

// Total uses a value receiver – it only reads data, never modifies.
func (c Cart) Total() float64 {
    var total float64
    for _, item := range c.Items {
        total += item.Price * float64(item.Quantity)
    }
    return total
}

// ItemCount uses a value receiver – read-only, consistent with Total.
func (c Cart) ItemCount() int {
    return len(c.Items)
}

func demonstrateValueReceiverBug() {
    // BUG: calling AddItem on a value copy does nothing to the original.
    cart := Cart{}
    valueCopy := cart // copies the struct
    valueCopy.Items = append(valueCopy.Items, CartItem{Name: "Ghost"})
    fmt.Println("original cart after value copy mutation:", cart.ItemCount()) // still 0
}

func main() {
    cart := &Cart{}

    cart.AddItem(CartItem{ProductID: 1, Name: "Widget", Price: 9.99, Quantity: 2})
    cart.AddItem(CartItem{ProductID: 2, Name: "Gadget", Price: 24.99, Quantity: 1})

    fmt.Printf("items: %d, total: $%.2f\n", cart.ItemCount(), cart.Total())

    cart.RemoveItem(1)
    fmt.Printf("after remove – items: %d, total: $%.2f\n", cart.ItemCount(), cart.Total())

    demonstrateValueReceiverBug()
}

```

Exercise

Build: A `Cart` struct with `Items []CartItem`. Implement `AddItem(item CartItem)` (pointer receiver), `RemoveItem(productID int)` (pointer receiver), and `Total() float64` (value receiver).

Input: Add three items with different prices and quantities. Remove one. Compute total.

Output: Correct item count and dollar total after each operation.

Acceptance: Add a test that creates a `Cart` by value (not pointer), calls `AddItem` on it, and confirms the original cart is unaffected — demonstrating why the pointer receiver matters. Run `go test ./...`.

Interview

- Why does calling a pointer receiver method on a value type sometimes fail to compile?
- A colleague wrote `Total()` with a pointer receiver. Is that wrong? What are the tradeoffs?
- What happens when you call a pointer receiver method on a nil pointer? When does it panic and when doesn't it?

Source: `01-go-fundamentals-and-cli-thinking/04-pointer-thinking-and-sharing-semantics.md`

Unit 4 — Pointer Thinking and Sharing Semantics

Concept

`&x` takes the address of `x` — it gives you a pointer. `*p` dereferences a pointer — it gives you the value it points to. Passing a pointer to a function means both the caller and the function see the same memory: any change the function makes is visible to the caller. Passing a value means the function gets a copy — changes stay local. Nil pointer dereference is the classic Go panic — always guard pointer inputs before using them.

Code

```
package main

import (
    "errors"
    "fmt"
)

type Product struct {
    ID    int
    Name  string
    Price float64
}

// UpdatePrice mutates the product in-place. The caller's product is changed.
func UpdatePrice(p *Product, newPrice float64) error {
    if p == nil {
        return errors.New("UpdatePrice: nil product")
    }
    if newPrice <= 0 {
        return fmt.Errorf("UpdatePrice: price must be positive, got %g", newPrice)
    }
    p.Price = newPrice
    return nil
}

// ApplyDiscount takes a value – it works on a copy and returns the discounted price.
// The original product is not changed.
func ApplyDiscount(p Product, pct float64) float64 {
    p.Price = p.Price * (1 - pct/100) // mutates local copy only
    return p.Price
}

func main() {
    // Pattern 1: pointer passed – mutation is visible to caller.
    product := Product{ID: 1, Name: "Widget", Price: 10.00}
    if err := UpdatePrice(&product, 12.50); err != nil {
        fmt.Println("error:", err)
    }
    fmt.Printf("after UpdatePrice: %.2f\n", product.Price) // 12.50 – original changed

    // Pattern 2: value passed – caller's product is unchanged.
    discounted := ApplyDiscount(product, 10)
    fmt.Printf("discounted price: %.2f\n", discounted) // 11.25
    fmt.Printf("original unchanged: %.2f\n", product.Price) // still 12.50

    // Pattern 3: nil guard fires.
    var nilProduct *Product
    if err := UpdatePrice(nilProduct, 5.00); err != nil {
        fmt.Println("nil guard caught:", err)
    }

    // Pattern 4: range-variable address trap – common bug in loops.
    products := []Product{{1, "A", 1.0}, {2, "B", 2.0}, {3, "C", 3.0}}
    ptrs := make([]*Product, len(products))
    for i := range products {
        ptrs[i] = &products[i] // correct: address of slice element
        // WRONG: ptrs[i] = &p (in a range p loop) – all ptrs would point to the same loop variable
    }
    ptrs[0].Name = "Alpha"
    fmt.Println("slice element updated:", products[0].Name) // Alpha
}
```


Exercise

Build: Write `UpdatePrice(p *Product, newPrice float64) error` that returns an error if `p` is nil or `newPrice <= 0`, otherwise sets `p.Price`. Also write `PriceWithTax(p Product, taxPct float64) float64` that returns the taxed price without modifying the original.

Input: One valid product, call `UpdatePrice` on it, then call `PriceWithTax` on the updated product.

Output: `UpdatePrice` modifies the original. `PriceWithTax` returns a number but the product's price is unchanged afterward.

Acceptance: Write tests for: nil pointer returns error, invalid price returns error, valid update modifies in place, `PriceWithTax` does not change `p.Price`. Run `go test ./...`.

Interview

- What is the difference between passing `Product` and `*Product` to a function?
- Why does `for _, p := range products { ptrs = append(ptrs, &p) }` produce unexpected results?
- When would you return `*Product` from a function versus returning `Product`?

Source: `01-go-fundamentals-and-cli-thinking/05-interfaces-implicit-contracts.md`

Unit 5 — Interfaces: Implicit Contracts

Concept

A type satisfies an interface simply by having the right methods — no `implements` keyword, no registration, no inheritance. If your type has a `Price() float64` method, it satisfies `Pricer`. This makes interfaces in Go decoupled by default: the interface and the concrete type can live in completely separate packages and never reference each other. The rule of thumb: accept interfaces, return concrete types. Keep interfaces small — one or two methods is better than ten.


```

package main

import "fmt"

// Pricer is satisfied by any type that has a Price() float64 method.
type Pricer interface {
    Price() float64
}

// Product is a physical item with stock.
type Product struct {
    ID      int
    Name    string
    BasePrice float64
}

func (p Product) Price() float64 {
    return p.BasePrice
}

// DigitalProduct has no stock – it's a download.
type DigitalProduct struct {
    ID      int
    Name    string
    LicensePrice float64
}

func (d DigitalProduct) Price() float64 {
    return d.LicensePrice
}

// TotalCost accepts a slice of Pricer – works for any type that satisfies the interface.
// It does not know or care whether items are physical or digital.
func TotalCost(items []Pricer) float64 {
    var total float64
    for _, item := range items {
        total += item.Price()
    }
    return total
}

// Discount returns a new Pricer that applies a percentage discount.
// Uses an anonymous struct that satisfies Pricer without a named type.
func Discount(p Pricer, pct float64) Pricer {
    return discountedItem{original: p, pct: pct}
}

type discountedItem struct {
    original Pricer
    pct      float64
}

func (d discountedItem) Price() float64 {
    return d.original.Price() * (1 - d.pct/100)
}

func main() {
    physical := Product{ID: 1, Name: "Keyboard", BasePrice: 79.99}
    digital := DigitalProduct{ID: 2, Name: "License", LicensePrice: 29.99}

    // Both satisfy Pricer – can be used anywhere the interface is accepted.
    items := []Pricer{physical, digital}
    fmt.Printf("total: %.2f\n", TotalCost(items)) // 109.98

    // Wrap with discount – still a Pricer, TotalCost still works.
    discounted := []Pricer{Discount(physical, 10), digital}

```

```

fmt.Printf("after 10%% discount on keyboard: %.2f\n", TotalCost(discounted)) // 101.99

// Interface variable – runtime type is checked with type assertion.
var p Pricer = physical
if conc, ok := p.(Product); ok {
    fmt.Printf("concrete type: %s\n", conc.Name)
}
}

```

Exercise

Build: Define type `Pricer` interface `{ Price() float64 }`. Create `Product` (physical, has `BasePrice`) and `DigitalProduct` (has `LicensePrice`). Both satisfy `Pricer`. Write `TotalCost(items []Pricer) float64`. Write a third type `BundledProduct` with two inner `Pricer` items — its `Price()` returns the sum of both at a 15% discount.

Input: A slice containing one `Product`, one `DigitalProduct`, and one `BundledProduct`.

Output: `TotalCost` returns the correct sum.

Acceptance: `TotalCost` must not use type switches or type assertions — it only calls `.Price()`. Run `go test ./...` with a test that checks the math.

Interview

- How does Go's implicit interface satisfaction differ from Java's `implements`? What does it enable?
- Why is a large interface (10+ methods) a design smell in Go?
- What does “accept interfaces, return concrete types” mean, and why is it the dominant pattern?

Source: `01-go-fundamentals-and-cli-thinking/06-errors-and-responsibility.md`

Unit 6 — Error handling thinking: responsibilities, wrapping, panic discipline

Learning outcome

Treat Go errors as ordinary **first-class returned values**:

```
value, err := Do()
```

You practise:

- `errors.New`
- `fmt.Errorf` with verbs including `%w` when layering failure causes expect introspection downstream.

Understand **defer / recovery** folklore only after recognising `panic` is rarely your happy-path instrument.

Go **does not** elevate exceptions controlling normal outcomes; pretending it does produces unreliable services quietly.

Core ideas

- Every failure path chooses **classification**: caller-retry-able? programmer bug? sanitation for user-visible message?
- **Wrap** sparingly yet meaningfully—“open config: ...” beats anonymous `return err`.
- `panic` signals **stop the world-ish** invariant violation or impossible states after programming mistakes—not missing files in CLI unless you consciously scope recovery at `main`.

CLI practice sketch

Exercise file operations (read absent file, unreadable permission). Thread errors upward enriching context at boundaries.

Contrast:

- returning wrapped errors,

- logging then swallowing (**forbidden habitual pattern** articulation),

Lab

Document three scenarios:

1. **network-ish** ephemeral errors (simulate),
2. **validation** misuse,
3. **programmer** misuse.

Argue succinctly panic appropriateness (likely none—but justify edge cases responsibly).

Interview prompts

Topic	Probe
panic vs error operational culture	articulate
sentinel errors introduction (errors.Is)	bridging future modules
when not to wrap (avoid chaining noise pollution)	

Source: 01-go-fundamentals-and-cli-thinking/07-package-layout-cmd-internal-and-pkg.md

Unit 7 — Package layout (cmd , internal , guarded pkg)

Learning outcome

Internalise cmd/ binaries, cohesive domain packages (user/ , order/ , config/ , etc.—names illustrative), sparing third-party-esque pkg/ , and restrictive internal/ enforcement—so import graphs stay honest.

Ownership question always: “Who may depend on whom, and why?”

Canonical mental map

```
repo/
├ cmd/...           ← thin mains wiring dependencies
├ internal/...      ← not importable by external modules
└ <domain pkgs> ← reusable-but-not-published libraries if internal not appropriate
```

Practice refactoring go-lab/

Migrate from monolith file sprawl progressively:

- Separate domain types & behaviours cleanly.
- Isolate configuration parsing so flags/env handling doesn't haunt business logic accidentally.

Forbidden dumping ground temptation: miscellaneous helpers package—articulate refactor plan when you drift there.

Lab

Draw import DAG (ASCII okay). Locate **potential cycles**. If hypothetical cycle emerges, narrate dissolution strategy splitting interfaces / moving interfaces consumer-side thoughtfully.

Interview prompts

- difference between internal enforcement vs social convention folders,
 - when breaking packages prematurely hurts velocity vs premature monolith viscosity,
 - cohesion vs duplication boundaries.
-

Source: 01-go-fundamentals-and-cli-thinking/08-modules-and-dependency-discipline.md

Unit 8 — Modules: reproducible graphs and dependency honesty (`go.mod`)

Learning outcome

You operate `go mod init + go mod tidy` fluently—not mystically—including understanding **semver intent**, pinning vs floating trade-offs for learning reproducibility.

Add exactly **one** legitimate third-party library solving a mundane problem (**CLI flags**, **YAML/TOML config**, hashing/UUID—you choose) documenting:

- rationale,
- behavioural surface you rely on.

Concepts to internalise

- **Module path uniqueness** anchors import strings.
- **Sum file integrity** reasoning (supply chain conscience).
- **Pseudo-versions / retracted versions**: awareness—they impact upgrade paths.

Avoid dependency shopping sprees—they expand attack surface mentally and operationally even if scanners quiet today.

Lab written answer

Produce short risk note:

- hypothetical upgrade breakage scenario,
- how you'd bisect regressions responsibly (`go.mod` pinning strategies at learning scale).

Interview prompts

- minimal dependency philosophy pragmatic cases,
- `replace` directives usage cautions (`go.mod` escapes),
- private module proxy awareness (conceptual readiness).

Source: 01-go-fundamentals-and-cli-thinking/09-context-cancellation-timeout.md

Unit 9 — Context: timeouts, cancellations, disciplined propagation—not globals

Learning outcome

Treat `ctx context.Context` as **deadlines + cancellations + bounded values (sparingly)** threading cooperatively—not a mystical bag chucking incidental configuration everywhere.

Mandatory internalisation sketches:

```
ctx, cancel := context.WithCancel(parent)
defer cancel()
```

```
ctx, cancel := context.WithTimeout(parent, 2*time.Second)
defer cancel()
```

Misuse anti-patterns to articulate

Stuffing unrelated config arbitrarily into `context.Value` without typed accessor discipline.

Propagating `context.TODO()` permanently—document only transitional scenarios.

Ignoring `ctx.Done` in loops performing simulated network or IO.

CLI practice blueprint

Implement a **download-like** faker (sleep or read large file concurrently) enforcing **overall budget** shorter than faker duration—observe clean abort vs leaky goroutine tendencies (note leak—fix responsibly even if minimally).

Highlight **why** `context` **is not global singleton state**.

Interview prompts

- parent → child propagation mental model diagrams,
- when context cancellation should preempt vs escalate metric logging,
- relationship with upcoming HTTP inbound deadlines (Area 05), DB calls (Area 08), gRPC (Area 06).

Source: `01-go-fundamentals-and-cli-thinking/10-composition-for-capabilities.md`

Unit 10 — Composition: inheritance intuition reframed deliberately

Learning outcome

Rebuild **capabilities** assembling structs embedding collaborators / explicit fields instead of phantom inheritance hierarchies.
Construct:

```
OrderService + Logger collaborator + injected Config abstraction
```

Clarify how PHP mental models tempt **deep derivation** stacks that obscure dependency directions.

Embedding nuance checklist

Exported promoted methods readability vs surprise—discipline naming so promoted API surface remains intentional—not accidental shotgun.

Practice

Wire `OrderService` constructor accepting collaborators via interfaces/concrete collaborators where justified.

Show **alternate composition** rewriting a hypothetical naive inheritance doodle from PHP-think into idiomatic decomposition.

Lab

Articulate succinctly inheritance advantages you surrendered consciously **and what Go compositional ergonomics reimbursed you** regarding testability/import clarity.

Interview prompts

- embedding vs plain field trade-offs,
- when shallow embedding becomes API leak,
- mocking/service substitution via interfaces interplay preview (Area 09),
- synergy with structs/receivers (Units 2–4).

Unit 11 — Capstone: task-cli integration proving Area 01 themes

Build target

Produce task-cli/ (standalone main acceptable) supporting:

Command semantics	Behaviour
create	Stores new task identifiable later.
list	Shows tasks with status fields you designed.
delete	Removes logically or tombstones consciously—explain choice briefly.
complete	Marks progression with consistent invariants (completed_at ?).

Underlying storage can be naive JSON/files while learning—explain trade-offs anticipating Area 08 seriousness later.

Apply deliberately:

Theme	Artefact reviewers expect
struct	expressive domain types
interface	seams (repos, clocks, printers) narrow where helpful
pointer/value choices	articulated in short design note
context	at least one bounded operation cancelling/timeout cleanly
package split	cohesion story over single-file sprawl unless justified temporary
go.mod	tidy dependency rationale
error handling	no silent ignores; layering clarity

Deliverable narratives

Produce documentation emphasising WHY not giant code dumps exclusively:

1. Ownership chart (bullet list).
2. Panic-vs-error rationale statements.
3. Pointer/receiver decisions summarised succinctly referencing earlier units.

Oral rehearsal checklist (must be smooth unscripted-ish):

- implicit interfaces,
- pointer semantics anecdotes,
- receiver reasoning,
- context placements,
- module hygiene,
- panic vs error,
- composition story.

Phase — 02-go-tooling-engineering-workflow

Directory: 02-go-tooling-engineering-workflow/

Source: 02-go-tooling-engineering-workflow/00-quick-reference.md

Quick Reference — Go Tooling

Essential commands

```
go build ./...      # build all packages
go run main.go      # run directly
go test ./...       # all tests
go test -v -run TestFoo # specific test, verbose
go test -race ./...  # race detector (ALWAYS in CI)
go vet ./...         # static analysis
gofmt -w .           # format in place
goimports -w .       # format + fix imports
```

Build tags

```
//go:build integration
// +build integration  (Go <1.17 compat)
```

```
go test -tags=integration ./...
```

go.mod essentials

```
go mod tidy      # remove unused deps
go mod download  # fetch all deps
go get pkg@v1.2.3 # add/update dep
```

Makefile pattern

```
.PHONY: build test lint
build: ; go build -o bin/app ./cmd/app
test:  ; go test -race ./...
lint:  ; golangci-lint run
```

Delve (debugger)

```
dlv debug ./cmd/app
dlv test ./pkg/...
```

```
(dlv) break main.go:25
(dlv) continue
(dlv) print varname
```

Unit 1 — Formatting, Linting, and Task Runners

Concept

`gofmt` is non-negotiable — all Go code in the world is formatted the same way, and CI rejects unformatted diffs. `golangci-lint` runs dozens of analysers in one pass and catches real bugs: missing error checks, shadowed variables, incorrect mutex usage. A `Makefile` gives every team member and CI the same commands. Without it, “works on my machine” problems multiply fast.

Code

```
# Makefile — standard targets every Go service should have
```

```
.PHONY: build test lint fmt tidy run
```

```
build:
```

```
    go build -o bin/go-lab ./cmd/go-lab
```

```
test:
```

```
    go test -race -count=1 ./...
```

```
lint:
```

```
    golangci-lint run ./...
```

```
fmt:
```

```
    gofmt -w .
```

```
    goimports -w .
```

```
tidy:
```

```
    go mod tidy
```

```
run:
```

```
    go run ./cmd/go-lab
```

```
# Run all checks before committing
```

```
check: fmt tidy lint test
```

```
# .golangci.yml — start with high-signal linters, add more as you go
```

```
linters:
```

```
  enable:
```

- errcheck # catches ignored errors (the #1 Go bug source)
- govet # suspicious constructs: misaligned printf, shadowed vars
- staticcheck # advanced correctness checks and deprecation warnings
- gosimple # simplification suggestions
- unused # dead code detection

```
linters-settings:
```

```
  errcheck:
```

```
    check-type-assertions: true # flag unchecked `x.(T)` assertions
```

```
issues:
```

```
  max-issues-per-linter: 0
```

```
  max-same-issues: 0
```

Exercise

Build: Add a `Makefile` and `.golangci.yml` to your `go-lab/` project with the targets above.

Input: Run `make lint` on your existing code. Then intentionally introduce two lint violations: ignore an error return from `NewProduct`, and add an unused variable.


```
// Build tag: only compiled when -tags integration is passed
//go:build integration

package main

import "fmt"

func init() {
    fmt.Println("integration mode: using real external services")
}

# Cross-compile a static Linux binary from macOS
CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -o bin/go-lab-linux ./cmd/go-lab

# Verify it's statically linked
file bin/go-lab-linux # should say: ELF 64-bit LSB executable, statically linked

# Run integration tests only
go test -tags integration ./...
```

Exercise

Build: Add the `sumFirstN` function with the off-by-one bug to your `go-lab/` project. Use `dlv debug` to find the exact line where `i` goes out of bounds.

Input: Set a breakpoint in `sumFirstN`. Step through the loop watching `i` and `total`.

Output: Identify the line where the bug occurs. Fix it. Confirm `sumFirstN(nums, 3)` returns `60` (10+20+30).

Acceptance: Fix passes `go test ./...`. Add a `make cross-build` target that produces a Linux binary using `CGO_ENABLED=0`.

Interview

- When would you use `Delve` over `fmt.Printf` for debugging?
 - What does `CGO_ENABLED=0` do, and why does it matter for Docker deployments?
 - How do build tags differ from `//go:generate` directives?
-

Source: `02-go-tooling-engineering-workflow/03-linux-and-network-process-triage-for-backend-debugging.md`

Unit 3 — Linux and Network Process Triage for Backend Debugging

Concept

Backend debugging is not just reading code — it is observing a running system. When a service is wedged, slow, or refusing connections, you need to answer: is the process running, what port is it on, how many connections does it have, and is it blocking on I/O or burning CPU? These tools give you ground truth about a running system in seconds, before you touch the codebase.

Code

```
# --- Find the process ---
pgrep -la go-lab          # list PIDs and command lines matching "go-lab"
ps aux | grep go-lab      # broader view with CPU and memory

# --- Check what port it is listening on ---
ss -tlnp | grep 8080      # show TCP listening sockets, grep for port 8080
lsof -i :8080             # show all processes using port 8080

# --- Inspect open connections for a specific PID ---
lsof -p <pid> | grep TCP   # list open TCP connections for the process
ss -tnp | grep <pid>       # alternative: netstat-like view filtered by pid

# --- Check connection state counts (useful when things look wedged) ---
ss -s                     # summary: LISTEN, ESTAB, CLOSE_WAIT counts
ss -tn state close-wait   # list all CLOSE_WAIT connections (server not reading responses)

# --- Read /proc for low-level info when debuggers unavailable ---
cat /proc/<pid>/limits     # file descriptor limits (ulimit)
ls -la /proc/<pid>/fd | wc -l # count open file descriptors
cat /proc/<pid>/status     # memory usage, thread count, state

# --- Send signals ---
kill -SIGTERM <pid>       # graceful shutdown – app should handle this
kill -SIGINT <pid>        # same as Ctrl+C
kill -SIGKILL <pid>       # force kill – no cleanup, last resort

# --- Trace syscalls briefly (use on a replica, not production) ---
strace -p <pid> -e trace=network # show only network syscalls
strace -p <pid> -c              # summary: count syscalls, find the expensive ones
```

```
// In your Go server: handle SIGTERM for graceful shutdown
package main

import (
    "context"
    "log"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"
)

func main() {
    srv := &http.Server{Addr: ":8080"}

    go func() {
        if err := srv.ListenAndServe(); err != http.ErrServerClosed {
            log.Fatal(err)
        }
    }()
    log.Println("listening on :8080")

    // Wait for SIGTERM or SIGINT
    quit := make(chan os.Signal, 1)
    signal.Notify(quit, syscall.SIGTERM, syscall.SIGINT)
    <-quit

    log.Println("shutting down...")
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()
    if err := srv.Shutdown(ctx); err != nil {
        log.Fatal("shutdown error:", err)
    }
    log.Println("server stopped")
}
```

Exercise

Build: Start your `go-lab` server (the HTTP version from module 05, or a minimal `net/http` server on `:8080`).

Input: With the server running in one terminal, open another and run the triage commands above.

Output: Find the PID, confirm it is listening on 8080 with `ss -tlnp`, check its open file descriptors with `lssof -p <pid>`, then send `SIGTERM` and verify the server logs “server stopped” and exits cleanly.

Acceptance: You can answer these without looking at code: what is the PID, what port, how many open FDs. Graceful shutdown completes within 5 seconds.

Interview

- A service is not accepting new connections but the process is running. What commands help you diagnose this?
 - What does a high count of `CLOSE_WAIT` connections indicate?
 - When would you reach for `strace` versus `go tool pprof` to diagnose a slow Go service?
-

Phase — 03-go-concurrency-and-queue-workers

Directory: 03-go-concurrency-and-queue-workers/

Source: 03-go-concurrency-and-queue-workers/00-quick-reference.md

Quick Reference — Concurrency Primitives

Goroutine launch

```
go func() { ... }()           // fire and forget (needs coordination)
var wg sync.WaitGroup
wg.Add(1)
go func() { defer wg.Done(); work() }()
wg.Wait()
```

Channel directions

```
ch := make(chan int)          // unbuffered — blocks until receiver ready
ch := make(chan int, 10)      // buffered — blocks only when full
func send(ch chan<- int) {}    // send-only
func recv(ch <-chan int) {}    // receive-only
```

Select

```
select {
case v := <-ch1: ...
case ch2 <- val: ...
case <-ctx.Done(): return ctx.Err()
default: ...                // non-blocking
}
```

Close and range

```
close(ch)                    // signal no more sends
for v := range ch { ... }    // reads until closed
```

Common mistakes

```
// WRONG: goroutine outlives main
go func() { time.Sleep(1*time.Second); fmt.Println("done") }()
// main exits before goroutine prints — add wg.Wait()
```

Source: 03-go-concurrency-and-queue-workers/01-scope-capstone-queue-worker.md

Unit 1 — Module Scope: queue-worker/

What You Will Build

A job queue with concurrent workers — the `queue-worker/` codebase. A producer generates jobs and sends them to a shared channel. A pool of N workers drains the channel concurrently. A `select` loop handles both incoming jobs and shutdown signals. The whole system shuts down cleanly when a context is cancelled.

No frameworks, no HTTP. Pure Go concurrency primitives from the standard library.

What You Will Know by the End

- **Goroutines:** launch with `go f()`, understand that launch and completion are separate — you must synchronize explicitly
- **Channels:** pass data ownership between goroutines; close to signal completion; use directional types in function signatures
- **Buffered channels:** sender does not block until the buffer is full; useful for absorbing bursts
- **select:** wait on multiple channels simultaneously; integrate context cancellation cleanly
- **WaitGroup:** wait for a known set of goroutines to finish
- **Race detector:** `go test -race` catches concurrent map writes and other data races before they hit production

Spine Project: queue-worker/

Carry this structure through units 2–11:

```
queue-worker/  
  main.go      — wires producer + workers + shutdown  
  job.go       — Job type  
  worker.go    — worker function  
  producer.go  — producer function  
  pool.go      — (added later) bounded worker pool
```

Capstone (Unit 11): producer generates 50 jobs, 5 workers process them concurrently, a 2-second context timeout cancels remaining work, and the program reports how many jobs completed versus were cancelled.

Mental Model

Go's concurrency motto is “share memory by communicating.” Instead of a shared variable that multiple goroutines write to (and a mutex to protect it), pass the data through a channel. The goroutine that owns the channel is the only one writing to it. This eliminates entire classes of race conditions.

A goroutine is not a thread — it is multiplexed onto OS threads by the Go runtime. You can have tens of thousands of goroutines. You cannot have tens of thousands of OS threads. The cost of spawning a goroutine is ~2KB of stack and a scheduler entry, not a kernel syscall.

Source: 03-go-concurrency-and-queue-workers/02-goroutines-and-nonblocking-launch.md

Unit 2 — Goroutines and Non-Blocking Launch

Concept

`go f()` schedules `f` to run concurrently and returns immediately — it does not wait for `f` to finish. This means if `main()` exits before your goroutines are done, they are killed mid-execution with no cleanup. `sync.WaitGroup` is the standard solution: call `wg.Add(1)` before launching, `wg.Done()` (via `defer`) inside the goroutine, and `wg.Wait()` in main. The print order of goroutines is non-deterministic — the scheduler decides, and that is intentional.

Code

```
package main

import (
    "fmt"
    "sync"
    "time"
)

// processEmail simulates sending one email.
// In production this would be an SMTP call or API request.
func processEmail(workerID, emailID int, wg *sync.WaitGroup) {
    defer wg.Done() // always called, even if the function panics

    time.Sleep(50 * time.Millisecond) // simulate network latency
    fmt.Printf("worker %d: sent email %d\n", workerID, emailID)
}

func main() {
    const workers = 5
    const emailsPerWorker = 3

    var wg sync.WaitGroup

    for w := 1; w <= workers; w++ {
        for e := 1; e <= emailsPerWorker; e++ {
            wg.Add(1) // increment BEFORE launching the goroutine
            go processEmail(w, e, &wg)
        }
    }

    // Blocks here until all 15 goroutines call wg.Done().
    wg.Wait()
    fmt.Println("all 15 emails sent")

    // Without wg.Wait(), main would exit after ~0ms and 0 emails would print.
    // That is the classic "goroutine orphan" bug.
}
```

Exercise

Build: An email dispatcher that launches 5 workers, each processing 3 emails. Each “send” takes 50ms (use `time.Sleep`). Print `worker W: sent email E` for each.

Input: No user input — hardcoded workers=5, emails=3.

Output: All 15 `worker X: sent email Y` lines printed before `all 15 emails sent`. Print order can vary — that is expected.

Acceptance: Move `wg.Wait()` to a comment and observe that many lines are missing when you run without it. Put it back. Add a test using `runtime.NumGoroutine()` — assert that goroutine count returns to baseline after `wg.Wait()`.

Interview

- Why does `go f()` return immediately instead of waiting for `f` to finish?
 - What is the difference between a goroutine leak and a goroutine that exits too early?
 - What happens to a goroutine if the function that launched it returns?
-

Unit 3 — Channels: Share by Communicating

Concept

A channel transfers ownership of a value from one goroutine to another — no shared memory, no locks needed for the transfer itself. An unbuffered channel forces a rendezvous: the sender blocks until a receiver is ready, and the receiver blocks until a sender sends. Use directional channel types in function signatures (`chan<- int` for send-only, `<-chan int` for receive-only) — they document intent and the compiler enforces them. Close a channel from the sender side when there are no more values; receivers use `range` to drain it cleanly.

Code

```
package main

import (
    "fmt"
    "sync"
)

// producer sends job IDs on a send-only channel, then closes it.
// Closing signals to consumers: no more jobs are coming.
func producer(jobs chan<- int, count int) {
    for i := 1; i <= count; i++ {
        jobs <- i // blocks until a consumer is ready (unbuffered channel)
    }
    close(jobs) // sender closes, never the receiver
}

// consumer reads from a receive-only channel until it is closed and drained.
func consumer(id int, jobs <-chan int, wg *sync.WaitGroup) {
    defer wg.Done()
    for job := range jobs { // range exits when channel is closed AND empty
        fmt.Printf("consumer %d: processed job %d\n", id, job)
    }
}

func main() {
    jobs := make(chan int) // unbuffered: every send blocks until a receive is ready
    var wg sync.WaitGroup

    // Start 2 consumers — they race to receive from the same channel.
    wg.Add(2)
    go consumer(1, jobs, &wg)
    go consumer(2, jobs, &wg)

    // Producer runs concurrently; it will block on each send until a consumer picks up.
    go producer(jobs, 6)

    wg.Wait()
    fmt.Println("pipeline complete — 6 jobs processed by 2 consumers")
}
```

Exercise

Build: A two-stage pipeline: `generate(n int) <-chan int` produces integers 1 through n, `square(in <-chan int) <-chan int` reads from the first channel and sends each value squared. Chain them and print the results in main.

Input: `n = 5`

Output:

```
1
4
9
16
25
```

Acceptance: Both `generate` and `square` must close their output channels when done. Main must drain the final channel with `range` — no `WaitGroup` needed. Run `go test -race ./...` — no race conditions.

Interview

- Why must only the sender close a channel? What happens if a receiver closes it?
 - What is the difference between `for v := range ch` and `for { v := <-ch }`?
 - How do directional channel types (`chan<-` , `<-chan`) improve code clarity and safety?
-

Source: `03-go-concurrency-and-queue-workers/04-buffered-channels-capacity-tradeoffs.md`

Unit 4 — Buffered Channels: Capacity and Tradeoffs

Concept

A buffered channel (`make(chan T, N)`) lets the sender proceed without a receiver until the buffer is full. Once full, the sender blocks — just like an unbuffered channel. This is useful for absorbing short bursts: a producer can send N items before it has to wait. But buffering hides backpressure: if you buffer too much, your system accepts more work than it can handle and you get memory growth instead of a clear “slow down” signal. Choose buffer size based on expected burst, not as a way to avoid thinking about flow control.

Code

```
package main

import (
    "fmt"
    "time"
)

func main() {
    // Unbuffered: every send blocks until someone receives.
    unbuffered := make(chan int)
    go func() {
        start := time.Now()
        for i := 1; i <= 3; i++ {
            unbuffered <- i
            fmt.Printf("unbuffered: sent %d after %v\n", i, time.Since(start).Round(time.Millisecond))
        }
        close(unbuffered)
    }()
    for v := range unbuffered {
        time.Sleep(20 * time.Millisecond) // slow consumer
        fmt.Printf("unbuffered: received %d\n", v)
    }

    fmt.Println()

    // Buffered: sender can put up to 3 items without blocking.
    buffered := make(chan int, 3)
    start := time.Now()
    buffered <- 1 // does not block – buffer has room
    buffered <- 2
    buffered <- 3
    fmt.Printf("buffered: all 3 sent in %v (no receiver yet)\n", time.Since(start).Round(time.Millisecond))

    // Fourth send would block because buffer is full.
    // buffered <- 4 // deadlock if uncommented here with no receiver

    close(buffered)
    for v := range buffered {
        fmt.Printf("buffered: received %d\n", v)
    }

    // Measure producer wait time when buffer fills up.
    bursty := make(chan int, 5)
    go func() {
        for i := 1; i <= 10; i++ {
            t := time.Now()
            bursty <- i
            waited := time.Since(t)
            if waited > time.Millisecond {
                fmt.Printf("producer blocked on item %d for %v (buffer full)\n", i, waited.Round(time.Millisecond))
            }
        }
        close(bursty)
    }()
    for v := range bursty {
        time.Sleep(15 * time.Millisecond) // consumer slower than producer
        _ = v
    }
}
```

Exercise

Build: A rate-limited batch processor. Producer sends 20 jobs into a buffered channel of size 5. Consumer processes each job in 10ms. Use `time.Now()` to measure how long the producer is blocked each time the buffer fills.

Input: 20 jobs, buffer size 5, consumer 10ms per job.

Output: Print which job number caused the producer to block and for how long.

Acceptance: The total runtime should be roughly `20 * 10ms / 1 consumer = 200ms` (not 0ms, which would indicate buffering masked all wait). Run `go test -race ./...`.

Interview

- A buffered channel of size 100 never blocks the producer. Is this a good thing?
 - What is the observable difference between a buffer-full block and a deadlock?
 - When would you use an unbuffered channel even when you know the sender is faster than the receiver?
-

Source: `03-go-concurrency-and-queue-workers/05-select-multicase-coordination.md`

Unit 5 — select: Multi-Case Coordination

Concept

`select` waits on multiple channels simultaneously and proceeds on whichever becomes ready first. If multiple cases are ready at the same time, Go picks one at random — never rely on ordering. The most important use of `select` is combining a work channel with `ctx.Done()`: when the context is cancelled, the goroutine exits cleanly instead of being stuck waiting for work that will never come. Never add a `default` case unless you explicitly want non-blocking polling — a `default` in a loop becomes a busy-spin that pegs the CPU.

Code

```
package main

import (
    "context"
    "fmt"
    "time"
)

// worker processes jobs until the channel is closed or the context is done.
func worker(ctx context.Context, id int, jobs <-chan int) {
    for {
        select {
        case job, ok := <-jobs:
            if !ok {
                fmt.Printf("worker %d: jobs channel closed, exiting\n", id)
                return
            }
            fmt.Printf("worker %d: processing job %d\n", id, job)
            time.Sleep(30 * time.Millisecond) // simulate work

        case <-ctx.Done():
            // Context timed out or was cancelled – stop accepting new work.
            fmt.Printf("worker %d: context done (%v), exiting\n", id, ctx.Err())
            return

            // No default case: without it, select blocks until one case is ready.
            // A default here would spin the CPU when both channels are empty.
        }
    }
}

func main() {
    // Give the whole operation 200ms.
    ctx, cancel := context.WithTimeout(context.Background(), 200*time.Millisecond)
    defer cancel()

    jobs := make(chan int, 10)

    // Send 8 jobs spaced 30ms apart – total 240ms, which exceeds the 200ms deadline.
    go func() {
        for i := 1; i <= 8; i++ {
            select {
            case jobs <- i:
                fmt.Printf("sent job %d\n", i)
                time.Sleep(30 * time.Millisecond)
            case <-ctx.Done():
                fmt.Println("producer cancelled, closing jobs channel")
                close(jobs)
                return
            }
        }
        close(jobs)
    }()

    worker(ctx, 1, jobs)
    fmt.Println("done")
}
```

Exercise

Build: A worker that processes jobs but stops after 200ms (use `context.WithTimeout`). Jobs take 30ms each. The producer sends one job every 25ms for as long as the context is alive.

Input: 200ms timeout, 30ms per job, jobs arriving every 25ms.

Output: Print each job processed. When the timeout fires, print how many jobs were processed.

Acceptance: The program must exit cleanly — no goroutine leak (verify with `runtime.NumGoroutine()` at the end). In-flight job must complete before the worker exits. Run `go test -race ./...`.

Interview

- What happens when multiple `select` cases are ready simultaneously?
 - Why is adding a `default` case to a `select` loop dangerous?
 - How does `select` on `ctx.Done()` differ from `select` on a dedicated `quit chan struct{}`?
-

Source: `03-go-concurrency-and-queue-workers/06-context-cancellation-hierarchies.md`

Unit 6 — Context cancellation hierarchies propagate termination intent

Patterns:

```
ctx, cancel := context.WithCancel(parent)
defer cancel()
```

Interpret **parent** $\Rightarrow$ **child** $\Rightarrow$ **grandchildren** subgraph cancellation semantics when parent fire cancellation—downstream observes `Done` closures.

Contrasts mistaken mental model stacking detached `context.Background()` trees ignoring propagated coordination—enterprise Go begins structuring cross-service boundaries conscientiously—even if exercise limited locally first.

Practice extensions

Enhance `queue-worker/` scaffolding: cancelling long faker jobs cooperatively—not merely killing goroutines brutally.

Mandatory reflection writing

Demonstrate cancelling propagates cleanly without lingering blocked senders mistakenly—coordinate channel closure thoughtfully.

Interview prompts

Always defer `cancel` discipline reasoning.

Context misuse stuffing unrelated config overshadowing readability—disciplined restraint.

Source: `03-go-concurrency-and-queue-workers/07-timeouts-payment-fakes.md`

Unit 7 — Timeout discipline: simulated slow dependencies respect budgets

Learning outcome

Bind external-ish latency envelopes using `context.WithTimeout` consciously contrasting unbounded sleeps spelling operational doom.

Practice scenario:

- faker “payment gateway” delaying **5s intentionally** while consumer policy dictates **≤ 2 s** overall budget.

Document resulting `context.DeadlineExceeded` style handling path plus logging categorisation distinctions (transient vs caller misuse vs dependency SLA breach).

Cross-link anticipating distributed HTTP timeouts later (**Area 12**) philosophically—even if code stays local here.

Lab

Enumerate timeout placement strategies distinguishing:

- coarse per overall job,
- nested inner budgets tightened iteratively responsibly.

Explain misplacement causing duplicate cancellation confusion.

Interview prompts

Propagating shortening child contexts vs independent sibling contexts—diagram narrative verbally.

Source: `03-go-concurrency-and-queue-workers/08-mutexes-and-shared-mutable-state.md`

Unit 8 — Mutex & shared mutation: pessimistic guarding when purity cracks

Bad habit anti-pattern caricature:

“Just share maps across goroutines because map writes feel easy.” → **crash / data race chaos**.

Responsible pattern:

Synchronize with `sync.Mutex` guarding invariants—not sprinkling blindly after-the-fact when `-race` yells louder than design.

Practice

Increment shared counter pressured by ~100 concurrent goroutines.

1. Demonstrate flaky wrong results without instrumentation (may appear “fine” sporadically—never trust nondeterministic green runs).
2. Re-run enforcing `-race` highlighting data races vividly.
3. Serialize increments via mutex concluding determinism reproducibly—also measure coarse latency inflation qualitatively (not microseconds wars yet).

Lab written reflection

Enumerate **** Mutex vs channels**** decision heuristics: linear ownership pipelines vs shared counter-style hotspot needing lock.

Discuss **ordering deadlocks**: acquiring multiple locks inconsistently lethal pattern—sketch hypothetical cross-lock scenario resolved by consistent acquisition partial ordering conventions.

Interview prompts

Contrast **optimistic concurrency** glimpses forward (defer Area `08` DB depth) layering mental hooks.

Source: 03-go-concurrency-and-queue-workers/09-worker-pool-enterprise-pattern.md

Unit 9 — Worker pool structuring: amortise goroutines deliberately

Corporate Go systems rarely spawn unbounded ephemeral goroutines for each remote event—economics degrade under fan-in storms.

Prefer **bounded worker goroutine set** draining shared job intake channel responsibly.

Practice assignment:

Design **email-ish sender** analogue:

```
1000 jobs total
≤ 5 simultaneous active workers draining queue
shutdown waits in-flight politely honoring context cancellation tightening
```

Learning outcomes articulated

Enumerate **graceful draining** choreography:

- cease accepting inbound,
- cooperative completion,
- join / wait layering.

Articulate **backpressure**: sluggish workers imply queue growth—eventually producers must choke, throttle, or shed load—explicit policy decision not accidental memory ballooning unnoticed.

Interview prompts

Sizing worker pools via metrics-informed loops vs arbitrary constants—risk articulation.

When pool too small induces SLA misses vs oversized spinning idle CPU waste.

Source: 03-go-concurrency-and-queue-workers/10-race-detector-drills.md

Unit 10 — Race detector: instrument, provoke, refactor—don't shrug nondeterminism

Operational command habits

Embed `go run -race` / `go test -race` into learning loops—even if slowdown painful—prefer catching unsynchronised accesses early morally cheaper than flaky prod heisenbugs exhausting teams.

Go race detector observes **happens-before** instrumentation approximations—still invaluable teaching discipline.

Practice cycle

Purposefully synthesize deterministic race regressions—you know root cause consciously—observe detector reports reading until clarity appears—repair via:

- narrower ownership pipelines,
- channel handoff rewriting,
- explicit mutex guarding chosen deliberately.

Forbidden outcome: patching sleep durations hoping races vanish probabilistically—that's folklore debugging punishment later.

Lab deliverable checklist

Brief write-up interpreting race report lines mapping to offending source lines—even if abbreviated paraphrasing.

Interview prompts

Instrumentation overhead magnitude expectation order-of-mouse-not-science-ballpark honesty.

Contrast property-based concurrency tests vs `-race` complementarity philosophies.

Source: `03-go-concurrency-and-queue-workers/11-capstone-queue-worker-integration.md`

Unit 11 — Capstone integration: disciplined `queue-worker/` journey

Compose features holistically—not isolated toys:

```
producer emits → bounded buffer/channel → worker pool draining
context cancellation/timeouts cooperating
shutdown discipline narrated plainly
race detector cleanliness targets (no knowingly ignored diagnostics)
```

Fault injection arcs (ethical “break then heal” rehearsal)

Produce **alternate scratch branches or tagged commits capturing intentional pathologies**:

pathology	pedagogical payoff
deadlock	lock/channel ordering cognition
goroutine leak	forgotten join / ignoring cancellation
data race	detector literacy + refactor

Heal each cleanly documenting **cause** → **structural fix rationale**—“because `sleep(10ms)` waved problem away” disqualifies academically.

Interview consolidation emphasises:

```
goroutines • channels (+buffering contrasts) • mutex • select contexts
timeouts • cancellations • `-race` • worker pooling • graceful shutdown choreography
```

Deliverables

runnable reference path README chunk emphasising WHY boundaries chosen—not merely command invocation screenshots.

Phase — 04-go-concurrency-advanced-labs

Directory: 04-go-concurrency-advanced-labs/

Source: 04-go-concurrency-advanced-labs/00-quick-reference.md

Quick Reference — Advanced Concurrency

Mutex

```
var mu sync.Mutex
mu.Lock()
defer mu.Unlock()
// critical section
```

sync.Map (concurrent-safe, no generics)

```
var m sync.Map
m.Store("key", value)
v, ok := m.Load("key")
m.LoadOrStore("key", defaultVal)
m.Range(func(k, v any) bool { ...; return true })
```

errgroup (golang.org/x/sync/errgroup)

```
g, ctx := errgroup.WithContext(ctx)
g.Go(func() error { return doWork(ctx) })
if err := g.Wait(); err != nil { ... }
```

context

```
ctx, cancel := context.WithCancel(context.Background())
defer cancel()
ctx, cancel := context.WithTimeout(ctx, 5*time.Second)
defer cancel()
// check: ctx.Done(), ctx.Err()
```

Race detector

```
go test -race ./...
go build -race ./cmd/app # run binary with race detection
```

Worker pool shape

```
jobs := make(chan Job, 100)
for i := 0; i < numWorkers; i++ {
    go worker(ctx, jobs)
}
```

Source: `04-go-concurrency-advanced-labs/01-races-syncmap-and-shared-maps.md`

Unit 1 — Data Races, `sync.Mutex`, and `sync.Map`

Concept

A regular Go map is not goroutine-safe. If two goroutines write to the same map concurrently — even to different keys — the runtime panics or silently corrupts data. The race detector (`go test -race`) catches this and reports the exact goroutine and line numbers. The fix is `sync.Mutex` (exclusive lock for all operations) or `sync.RWMutex` (shared lock for reads, exclusive for writes). Use `sync.Map` when keys are written once and read many times, or when different goroutines work on disjoint key sets — it is not a general replacement for a mutex-protected map.


```

package main

import (
    "fmt"
    "sync"
)

// UnsafeCache demonstrates the race – DO NOT use this in production.
// Run with: go run -race . to see the data race report.
type UnsafeCache struct {
    store map[string]string
}

// Cache is a goroutine-safe in-memory store protected by RWMutex.
type Cache struct {
    mu    sync.RWMutex
    store map[string]string
}

func NewCache() *Cache {
    return &Cache{store: make(map[string]string)}
}

// Set acquires an exclusive write lock – blocks all readers and writers.
func (c *Cache) Set(key, value string) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.store[key] = value
}

// Get acquires a shared read lock – multiple readers can proceed concurrently.
func (c *Cache) Get(key string) (string, bool) {
    c.mu.RLock()
    defer c.mu.RUnlock()
    v, ok := c.store[key]
    return v, ok
}

// Delete acquires an exclusive write lock.
func (c *Cache) Delete(key string) {
    c.mu.Lock()
    defer c.mu.Unlock()
    delete(c.store, key)
}

func main() {
    cache := NewCache()
    var wg sync.WaitGroup

    // 10 concurrent writers
    for i := 0; i < 10; i++ {
        wg.Add(1)
        go func(n int) {
            defer wg.Done()
            cache.Set(fmt.Sprintf("user:%d", n), fmt.Sprintf("Alice-%d", n))
        }(i)
    }

    // 10 concurrent readers – safe alongside other readers
    for i := 0; i < 10; i++ {
        wg.Add(1)
        go func(n int) {
            defer wg.Done()
            v, ok := cache.Get(fmt.Sprintf("user:%d", n))
            if ok {
                _ = v // use the value
            }
        }(i)
    }
    wg.Wait()
}

```

```

    }
    }(i)
}

wg.Wait()
fmt.Println("no races — all goroutines complete")
// Verify: go test -race ./...
}

```

Exercise

Build: A thread-safe `Cache` with `Set(key, value string)`, `Get(key string) (string, bool)`, and `Delete(key string)`. Use `sync.RWMutex`.

Input: 20 goroutines writing, 20 goroutines reading, running simultaneously.

Output: No race detector errors. All written values are readable by readers.

Acceptance: Write a test that uses `go test -race`. First make the test fail by removing the mutex (comment it out). Then restore the mutex — test must pass with `-race`. Also write a test that verifies `Get` on a missing key returns `("", false)`.

Interview

- What is a data race, and how is it different from a race condition?
- When would you choose `sync.RWMutex` over `sync.Mutex`?
- What are the tradeoffs of `sync.Map` vs a mutex-protected `map`?

Source: `04-go-concurrency-advanced-labs/02-worker-pool-fan-out-fan-in.md`

Unit 2 — Worker Pool: Fan-Out and Fan-In

Concept

Fan-out means distributing work across N workers via a shared input channel — each worker takes jobs as fast as it can. Fan-in means collecting results from N workers back into a single output channel. A bounded worker pool (fixed N goroutines) prevents goroutine explosion: without a bound, you could spawn 10,000 goroutines for 10,000 jobs and exhaust memory. The pattern: close the jobs channel when all work is submitted, use a `WaitGroup` to know when all workers are done, then close the results channel so the collector can use `range`.


```

package main

import (
    "fmt"
    "sync"
    "time"
)

type Job struct {
    ID    int
    Size int // simulated image size in KB
}

type Result struct {
    JobID      int
    OutputSize int
    Duration   time.Duration
}

// worker reads jobs, simulates resize, writes results.
func worker(id int, jobs <-chan Job, results chan<- Result, wg *sync.WaitGroup) {
    defer wg.Done()
    for job := range jobs { // exits when jobs is closed and drained
        start := time.Now()
        time.Sleep(10 * time.Millisecond) // simulate resize work
        results <- Result{
            JobID:      job.ID,
            OutputSize: job.Size / 2,
            Duration:   time.Since(start),
        }
    }
}

func main() {
    const numWorkers = 4
    const numJobs = 20

    jobs := make(chan Job, numJobs)
    results := make(chan Result, numJobs)

    // Launch fixed-size worker pool.
    var wg sync.WaitGroup
    for w := 1; w <= numWorkers; w++ {
        wg.Add(1)
        go worker(w, jobs, results, &wg)
    }

    // Feed all jobs.
    start := time.Now()
    for j := 1; j <= numJobs; j++ {
        jobs <- Job{ID: j, Size: 1024}
    }
    close(jobs) // signal workers: no more jobs

    // Close results once all workers are done – in a separate goroutine
    // to avoid deadlock (results is buffered but could fill if collector is slow).
    go func() {
        wg.Wait()
        close(results)
    }()

    // Collect all results (fan-in).
    var processed int
    for r := range results {
        processed++
        _ = r
    }
}

```

```
}

fmt.Printf("processed %d jobs with %d workers in %v\n",
    processed, numWorkers, time.Since(start).Round(time.Millisecond))
// ~4 workers * 10ms = 50ms for 20 jobs (vs 200ms with 1 worker)
}
```

Exercise

Build: An image resize simulator: 20 “images” with random sizes (100–2000 KB). Run with 4 workers first, then 1 worker. Each “resize” takes 10ms.

Input: 20 jobs, configurable worker count.

Output: `processed 20 jobs with 4 workers in ~50ms`. With 1 worker: `~200ms`.

Acceptance: The results channel must be closed properly — the collector must use `range` with no goroutine leaks. Run with `go test -race`. Write a benchmark (`BenchmarkPool`) that measures 4-worker vs 1-worker throughput.

Interview

- Why close the jobs channel from the sender rather than sending a “poison pill” sentinel value?
 - What happens if you forget to close the results channel? How does the collector get stuck?
 - A pool of 100 workers on a machine with 4 cores — is this wasteful for CPU-bound work? For I/O-bound work?
-

Source: `04-go-concurrency-advanced-labs/03-context-cancel-timeout-deadline-mastery.md`

Unit 3 — Context: Cancel, Timeout, and Deadline

Concept

`context` is how Go propagates cancellation, deadlines, and request-scoped values across goroutines and function calls. Always pass `ctx` as the first argument to functions that do I/O, sleep, or call other services. Always check `ctx.Done()` in loops and before blocking operations. The critical mistake is spawning a fresh `context.Background()` inside a function — this severs the cancellation chain and the child operation runs even after the caller has given up. Use `errors.Is(err, context.DeadlineExceeded)` to check for timeout — never string matching.


```

package main

import (
    "context"
    "errors"
    "fmt"
    "time"
)

// fetchURL simulates an HTTP call that takes ~50ms.
// It respects context cancellation.
func fetchURL(ctx context.Context, url string) (string, error) {
    select {
    case <-time.After(50 * time.Millisecond): // simulated latency
        return "body:" + url, nil
    case <-ctx.Done():
        return "", fmt.Errorf("fetchURL %s: %w", url, ctx.Err())
    }
}

// dbQuery simulates a database call that takes ~30ms.
func dbQuery(ctx context.Context, query string) (string, error) {
    select {
    case <-time.After(30 * time.Millisecond):
        return "row:" + query, nil
    case <-ctx.Done():
        return "", fmt.Errorf("dbQuery: %w", ctx.Err())
    }
}

// processRequest simulates an HTTP handler with a 100ms total budget.
// It calls dbQuery then a downstream service.
func processRequest(requestID string) {
    // Outer timeout: the whole request must complete in 100ms.
    ctx, cancel := context.WithTimeout(context.Background(), 100*time.Millisecond)
    defer cancel() // always defer cancel to release resources

    // Step 1: database call (30ms – fits in budget).
    row, err := dbQuery(ctx, "SELECT user WHERE id=1")
    if err != nil {
        fmt.Printf("[%s] db error: %v\n", requestID, err)
        return
    }
    fmt.Printf("[%s] db result: %s\n", requestID, row)

    // Step 2: downstream service with a tighter per-call deadline.
    // Give the downstream service only 40ms of the remaining budget.
    callCtx, callCancel := context.WithTimeout(ctx, 40*time.Millisecond)
    defer callCancel()

    result, err := fetchURL(callCtx, "https://api.example.com/enrich")
    if err != nil {
        if errors.Is(err, context.DeadlineExceeded) {
            fmt.Printf("[%s] downstream timed out\n", requestID)
        } else if errors.Is(err, context.Canceled) {
            fmt.Printf("[%s] request was cancelled\n", requestID)
        } else {
            fmt.Printf("[%s] downstream error: %v\n", requestID, err)
        }
        return
    }
    fmt.Printf("[%s] enriched: %s\n", requestID, result)
}

func main() {
    // req-1: db takes 30ms, downstream takes 50ms but only has 40ms – times out.

```

```
processRequest("req-1")

fmt.Println()

// Demonstrate WithCancel: manual cancellation from another goroutine.
ctx, cancel := context.WithCancel(context.Background())
go func() {
    time.Sleep(20 * time.Millisecond)
    cancel() // cancel from outside after 20ms
}()

_, err := fetchURL(ctx, "https://slow.example.com")
fmt.Println("manual cancel:", err)
}
```

Exercise

Build: Fetch 10 “URLs” concurrently (simulate each with `time.Sleep` of a random 20–80ms duration). Set a 300ms total context timeout. Launch all 10 as goroutines; collect results into a slice.

Input: 10 URLs, 300ms total deadline.

Output: Print which URLs succeeded and which were cancelled. The number of successes depends on timing — that is expected.

Acceptance: All 10 goroutines must exit cleanly when context expires — no goroutine leak. Use `errors.Is(err, context.DeadlineExceeded)` to categorize results. Run `go test -race ./...`.

Interview

- Why is creating a new `context.Background()` inside a function a correctness bug?
 - What is the difference between `context.WithTimeout` and `context.WithDeadline`?
 - How do you check if a context error is a deadline versus a cancellation?
-

Source: `04-go-concurrency-advanced-labs/04-leaks-deadlocks-unblocking-drills.md`

Unit 4 — Goroutine Leaks, Deadlocks, and Unblocking Drills

Concept

A goroutine leak is a goroutine that never exits. It silently consumes memory and scheduler capacity — the runtime does not tell you about it. Deadlock is different: two goroutines each waiting for the other. Go’s runtime detects full deadlocks immediately and panics with `all goroutines are asleep – deadlock!`. Partial deadlocks (some goroutines stuck, others running) are silent. Use `runtime.NumGoroutine()` in tests to catch leaks, and `go test -race` to catch data races that can hide underlying leak conditions.


```

package main

import (
    "context"
    "fmt"
    "runtime"
    "time"
)

// LEAK 1: goroutine sends to unbuffered channel with no receiver.
// The goroutine blocks forever because nobody reads from ch.
func leakyNoReceiver() {
    ch := make(chan int) // unbuffered
    go func() {
        ch <- 42 // blocks forever – nobody reads
        fmt.Println("this never prints")
    }()
    // ch goes out of scope but the goroutine is still blocked.
}

// FIX 1: use a buffered channel, or ensure a receiver exists, or use select with done.
func fixedNoReceiver() {
    ch := make(chan int, 1) // buffered: send does not block
    go func() {
        ch <- 42
    }()
    <-ch // drain it
}

// LEAK 2: goroutine ranges over a channel that is never closed.
// range blocks waiting for more values indefinitely.
func leakyNeverClosed() {
    ch := make(chan int, 5)
    go func() {
        for v := range ch { // blocks forever – channel never closed
            _ = v
        }
    }()
    ch <- 1
    ch <- 2
    // forgot to close(ch)
}

// FIX 2: always close the channel when the sender is done.
func fixedNeverClosed() {
    ch := make(chan int, 5)
    go func() {
        for v := range ch {
            _ = v
        }
    }()
    ch <- 1
    ch <- 2
    close(ch) // goroutine exits cleanly
    time.Sleep(time.Millisecond) // give it a moment to exit
}

// LEAK 3: goroutine ignores ctx.Done() and runs forever.
func leakyIgnoresContext(ctx context.Context) {
    go func() {
        for {
            time.Sleep(10 * time.Millisecond) // does work
            // never checks ctx.Done() – runs after context is cancelled
        }
    }()
}

```

```
// FIX 3: always check ctx.Done() in long-running goroutines.
func fixedWithContext(ctx context.Context) {
    go func() {
        for {
            select {
            case <-ctx.Done():
                fmt.Println("goroutine exiting via context")
                return
            case <-time.After(10 * time.Millisecond):
                // do work
            }
        }
    }()
}

func main() {
    baseline := runtime.NumGoroutine()
    fmt.Println("baseline goroutines:", baseline)

    leakyNoReceiver()
    time.Sleep(10 * time.Millisecond)
    fmt.Println("after leak 1:", runtime.NumGoroutine(), "(should be baseline+1)")

    fixedNoReceiver()
    time.Sleep(10 * time.Millisecond)
    fmt.Println("after fix 1:", runtime.NumGoroutine(), "(should be back to baseline)")

    ctx, cancel := context.WithTimeout(context.Background(), 50*time.Millisecond)
    defer cancel()
    fixedWithContext(ctx)
    time.Sleep(100 * time.Millisecond) // wait for context to expire and goroutine to exit
    fmt.Println("after context fix:", runtime.NumGoroutine(), "(should be back to baseline)")
}
```

Exercise

Build: Three isolated functions, each containing one of the bugs above. Write a test for each that: 1. Records `runtime.NumGoroutine()` before calling the buggy function. 2. Calls the function. 3. Waits 100ms. 4. Asserts goroutine count has not increased (leak) or that the program did not deadlock.

Input: The three buggy functions in this file.

Output: Tests fail for buggy versions, pass after applying fixes.

Acceptance: Run `go test -race -v ./...`. All three “fixed” versions show goroutine count returning to baseline within 100ms. Goroutine counts for leaky versions are clearly higher.

Interview

- How do you detect a goroutine leak in production without adding code to the binary?
 - What is the difference between a full deadlock (runtime panics) and a partial deadlock?
 - A goroutine is blocked on `<-ch` and the channel will never be written to. Is this a deadlock or a leak?
-

Phase — 05-http-apis-service-ownership-and-chi

Directory: 05-http-apis-service-ownership-and-chi/

Source: 05-http-apis-service-ownership-and-chi/00-quick-reference.md

Quick Reference — HTTP APIs with chi

Handler signature

```
func(w http.ResponseWriter, r *http.Request)
```

chi router setup

```
r := chi.NewRouter()
r.Use(middleware.Logger)
r.Use(middleware.Recoverer)
r.Get("/users/{id}", getUserHandler)
r.Post("/users", createUserHandler)
r.Route("/api/v1", func(r chi.Router) {
    r.Get("/health", healthHandler)
})
http.ListenAndServe(":8080", r)
```

URL param

```
id := chi.URLParam(r, "id")
```

JSON response helper

```
func respond(w http.ResponseWriter, status int, v any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    json.NewEncoder(w).Encode(v)
}
```

Decode request body

```
var req CreateUserRequest
if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
    http.Error(w, "bad request", http.StatusBadRequest); return
}
```

Middleware shape

```
func AuthMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // check auth
        next.ServeHTTP(w, r)
    })
}
```

validator tags

```
type CreateUserRequest struct {
    Email string `json:"email" validate:"required,email"`
    Age    int    `json:"age"    validate:"min=0,max=150"`
}
```

Source: 05-http-apis-service-ownership-and-chi/01-scope-go-api-capstone.md

Unit 1 — Module Scope: go-api/ REST Service

What You Will Build

A REST API for a simple e-commerce backend — the `go-api/` codebase. Users, Products, and Orders as resources. By the end of this module you will have:

- A `chi` router with route groups, URL parameters, and middleware
- JSON request decoding and response encoding with proper status codes
- DTO types that separate HTTP shapes from domain models
- Input validation with `go-playground/validator` and structured error responses
- A service layer that contains business logic, separate from the HTTP handlers

No ORM, no database yet (that comes in module 08 with `sqlx`). In-memory storage is fine for now.

What You Will Know by the End

- How `net/http` handler lifecycle works: headers, status code, body — order matters
- How `chi`'s middleware chain works and why logging/recovery come first
- What a DTO is and why you do not pass raw JSON structs into your service layer
- How to return structured validation errors instead of bare 400 strings
- What “handler must not contain business logic” means in practice

Project Structure

```
go-api/
cmd/api/main.go      - wire everything together, start server
internal/
  domain/
    product.go        - Product struct (domain model)
    user.go           - User struct
    order.go          - Order struct
  service/
    product_service.go - ProductService interface + in-memory impl
  handler/
    product_handler.go - HTTP handlers for /products
    user_handler.go
  dto/
    product_dto.go     - CreateProductRequest, ProductResponse
go.mod
Makefile
```

The Layering Rule

Each layer has one job:

- **Handler:** decode request, validate, call service, encode response. No SQL, no business rules.
- **Service:** business logic and validation rules. No HTTP types (`http.Request`, `http.ResponseWriter`).
- **Domain:** pure data types. No HTTP, no SQL tags (yet).

This separation means you can test your service without an HTTP server, and you can swap your storage layer without touching handlers.

Source: `05-http-apis-service-ownership-and-chi/02-http-handler-lifecycle.md`

Unit 2 — HTTP Handler Lifecycle

Concept

A handler receives a `ResponseWriter` and a `*Request`. The `ResponseWriter` writes the response in this order: set headers, call `WriteHeader(status)`, write the body. If you write the body before calling `WriteHeader`, Go implicitly sends a 200 — after that, calling `WriteHeader` does nothing and logs a warning. Read the request body once (it is a stream). Always set `Content-Type`. Always return after writing an error response — otherwise the handler continues and writes a second response on top of the first.


```

package main

import (
    "encoding/json"
    "log"
    "net/http"
)

type CreateUserRequest struct {
    Name  string `json:"name"`
    Email string `json:"email"`
}

type UserResponse struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Email string `json:"email"`
}

type ErrorResponse struct {
    Error string `json:"error"`
}

// writeJSON sets Content-Type, status code, then encodes the body.
// Order matters: headers must be set before WriteHeader, WriteHeader before body.
func writeJSON(w http.ResponseWriter, status int, v any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    if err := json.NewEncoder(w).Encode(v); err != nil {
        log.Printf("writeJSON: encode error: %v", err)
    }
}

func createUserHandler(w http.ResponseWriter, r *http.Request) {
    // Decode body – reads the stream once. Subsequent calls return empty.
    var req CreateUserRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        writeJSON(w, http.StatusBadRequest, ErrorResponse{Error: "invalid JSON"})
        return // MUST return – otherwise execution continues and writes a second response
    }

    if req.Email == "" {
        writeJSON(w, http.StatusBadRequest, ErrorResponse{Error: "email is required"})
        return
    }

    if req.Name == "" {
        writeJSON(w, http.StatusBadRequest, ErrorResponse{Error: "name is required"})
        return
    }

    // Stub: in production, call the service layer here.
    user := UserResponse{ID: 1, Name: req.Name, Email: req.Email}
    writeJSON(w, http.StatusCreated, user)
}

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("POST /users", createUserHandler)
    mux.HandleFunc("GET /health", func(w http.ResponseWriter, r *http.Request) {
        writeJSON(w, http.StatusOK, map[string]string{"status": "ok"})
    })

    log.Println("listening on :8080")
    if err := http.ListenAndServe(":8080", mux); err != nil {
        log.Fatal(err)
    }
}

```

```
}  
}
```

Exercise

Build: A `POST /users` handler that decodes `{"name": "...", "email": "..."}` , validates that both fields are non-empty, and returns: - `400 {"error": "email is required"}` if email is empty - `400 {"error": "invalid JSON"}` if the body is malformed - `201 {"id": 1, "name": "...", "email": "..."}` on success

Input: Test with `curl -X POST localhost:8080/users -d '{"name":"Alice","email":"a@b.com"}'` and with missing fields.

Output: Correct status codes and JSON bodies.

Acceptance: Write an `httptest`-based test covering all three cases. Use `httptest.NewRecorder()` and `httptest.NewRequest()` — no real server needed. Run `go test ./...`.

Interview

- Why must you call `w.Header().Set(...)` before `w.WriteHeader(...)`?
- What happens if a handler writes a body without ever calling `WriteHeader`?
- Why is `return` after writing an error response critical, not just a style choice?

Source: `05-http-apis-service-ownership-and-chi/03-chi-router-shaping-apis.md`

Unit 3 — chi Router: Shaping APIs

Concept

`chi` is a lightweight router built on `net/http` — handlers are standard `http.HandlerFunc`, no magic types. It adds URL parameters (`{id}`), route groups with shared middleware, and a clean middleware chain. Middleware runs in the order you call `r.Use()` — put `Logger` and `Recoverer` first so every request is logged and panics become 500s instead of crashing the server. The router is thin: it extracts URL params and delegates to handlers. Business logic does not belong here.


```

package main

import (
    "encoding/json"
    "log"
    "net/http"

    "github.com/go-chi/chi/v5"
    "github.com/go-chi/chi/v5/middleware"
)

type Product struct {
    ID    string `json:"id"`
    Name  string `json:"name"`
    Price float64 `json:"price"`
}

func writeJSON(w http.ResponseWriter, status int, v any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    json.NewEncoder(w).Encode(v)
}

// stub auth middleware – checks for a header, passes through for now
func authMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // In production: verify JWT, check API key, etc.
        // For now, just pass through.
        next.ServeHTTP(w, r)
    })
}

func main() {
    r := chi.NewRouter()

    // Global middleware – runs for every request, in this order.
    r.Use(middleware.Logger)    // logs method, path, status, duration
    r.Use(middleware.Recoverer) // converts panics into 500 responses

    // Health check – no auth required
    r.Get("/health", func(w http.ResponseWriter, r *http.Request) {
        writeJSON(w, http.StatusOK, map[string]string{"status": "ok"})
    })

    // Product routes – grouped so authMiddleware applies to all of them
    r.Route("/products", func(r chi.Router) {
        r.Use(authMiddleware)

        r.Get("/", func(w http.ResponseWriter, r *http.Request) {
            // Stub: return hardcoded list
            products := []Product{
                {ID: "1", Name: "Widget", Price: 9.99},
                {ID: "2", Name: "Gadget", Price: 24.99},
            }
            writeJSON(w, http.StatusOK, products)
        })

        r.Post("/", func(w http.ResponseWriter, r *http.Request) {
            var p Product
            if err := json.NewDecoder(r.Body).Decode(&p); err != nil {
                writeJSON(w, http.StatusBadRequest, map[string]string{"error": "invalid JSON"})
                return
            }
            p.ID = "new-id" // stub: real code calls service
            writeJSON(w, http.StatusCreated, p)
        })
    })
}

```



```

    r.Get("/{id}", func(w http.ResponseWriter, r *http.Request) {
        id := chi.URLParam(r, "id") // extract {id} from path
        // Stub: real code calls service.GetProduct(id)
        writeJSON(w, http.StatusOK, Product{ID: id, Name: "stub-" + id, Price: 0})
    })

    r.Delete("/{id}", func(w http.ResponseWriter, r *http.Request) {
        id := chi.URLParam(r, "id")
        log.Printf("delete product %s", id)
        w.WriteHeader(http.StatusNoContent) // 204: success, no body
    })
})

log.Println("listening on :8080")
http.ListenAndServe(":8080", r)
}

```

Exercise

Build: The full route structure for the e-commerce API: `GET /products`, `GET /products/{id}`, `POST /products`, `DELETE /products/{id}`. Use a chi route group with the stub `authMiddleware`. Add a `GET /health` outside the group.

Input: Run the server. Use `curl` to hit each endpoint.

Output: Correct status codes. `GET /products` returns a list. `GET /products/42` returns `{"id": "42", ...}`. `DELETE /products/42` returns 204.

Acceptance: Write chi handler tests using `httptest` — test that the router dispatches to the correct handler for each route. Test that `GET /products/123` extracts `id=123` correctly. Run `go test ./...`.

Interview

- What is the difference between global middleware (`r.Use`) and route-group middleware?
- How does chi extract `{id}` from the URL path? What does it return if the param is missing?
- Why should route handlers be thin — no business logic, no SQL?

Source: `05-http-apis-service-ownership-and-chi/04-dtos-handler-service-boundaries.md`

Unit 4 — DTOs and Handler–Service Boundaries

Concept

A DTO (Data Transfer Object) is the shape of an HTTP request or response — it is not your domain model. The handler decodes a DTO, calls the service with domain types, and converts the result back to a response DTO. This separation means your service does not know about HTTP, your domain model does not have JSON tags for fields that should not be exposed, and your handler does not contain any business logic. The test for correct separation: can you test the service without starting an HTTP server? Yes? Good.

Code

```
// dto/product_dto.go
package dto

// CreateProductRequest is the shape of the POST /products body.
type CreateProductRequest struct {
    Name  string `json:"name"`
    Price float64 `json:"price"`
    Stock int    `json:"stock"`
}

// ProductResponse is what the API returns — may omit internal fields.
type ProductResponse struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Price float64 `json:"price"`
    Stock int    `json:"stock"`
}
```

```
// domain/product.go
package domain

// Product is the domain model — no HTTP, no JSON tags (added later for DB).
type Product struct {
    ID    int
    Name  string
    Price float64
    Stock int
}
```

```
// service/product_service.go
package service

import (
    "errors"
    "fmt"
    "sync"

    "github.com/example/go-api/internal/domain"
)

var ErrNotFound = errors.New("product not found")

type ProductService interface {
    Create(name string, price float64, stock int) (domain.Product, error)
    GetByID(id int) (domain.Product, error)
    List() []domain.Product
}

type inMemoryProductService struct {
    mu      sync.RWMutex
    products map[int]domain.Product
    nextID   int
}

func NewProductService() ProductService {
    return &inMemoryProductService{
        products: make(map[int]domain.Product),
        nextID:   1,
    }
}

func (s *inMemoryProductService) Create(name string, price float64, stock int) (domain.Product, error) {
    if price <= 0 {
        return domain.Product{}, fmt.Errorf("price must be positive")
    }
    s.mu.Lock()
    defer s.mu.Unlock()
    p := domain.Product{ID: s.nextID, Name: name, Price: price, Stock: stock}
    s.products[s.nextID] = p
    s.nextID++
    return p, nil
}

func (s *inMemoryProductService) GetByID(id int) (domain.Product, error) {
    s.mu.RLock()
    defer s.mu.RUnlock()
    p, ok := s.products[id]
    if !ok {
        return domain.Product{}, fmt.Errorf("product %d: %w", id, ErrNotFound)
    }
    return p, nil
}

func (s *inMemoryProductService) List() []domain.Product {
    s.mu.RLock()
    defer s.mu.RUnlock()
    result := make([]domain.Product, 0, len(s.products))
    for _, p := range s.products {
        result = append(result, p)
    }
    return result
}

```

```
// handler/product_handler.go
package handler

import (
    "encoding/json"
    "errors"
    "net/http"

    "github.com/example/go-api/internal/dto"
    "github.com/example/go-api/internal/service"
)

type ProductHandler struct {
    svc service.ProductService
}

func NewProductHandler(svc service.ProductService) *ProductHandler {
    return &ProductHandler{svc: svc}
}

func (h *ProductHandler) Create(w http.ResponseWriter, r *http.Request) {
    var req dto.CreateProductRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        writeJSON(w, http.StatusBadRequest, map[string]string{"error": "invalid JSON"})
        return
    }

    p, err := h.svc.Create(req.Name, req.Price, req.Stock)
    if err != nil {
        writeJSON(w, http.StatusUnprocessableEntity, map[string]string{"error": err.Error()})
        return
    }

    writeJSON(w, http.StatusCreated, dto.ProductResponse{
        ID: p.ID, Name: p.Name, Price: p.Price, Stock: p.Stock,
    })
}

func (h *ProductHandler) GetByID(w http.ResponseWriter, r *http.Request) {
    // chi.URLParam extraction omitted for brevity — see unit 3
}

func writeJSON(w http.ResponseWriter, status int, v any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    json.NewEncoder(w).Encode(v)
}

func mapServiceError(err error) int {
    if errors.Is(err, service.ErrNotFound) {
        return http.StatusNotFound
    }
    return http.StatusInternalServerError
}

```

Exercise

Build: Wire the full create-product flow: `POST /products` decodes `CreateProductRequest`, calls `ProductService.Create`, returns `ProductResponse`. Handler must not contain any business logic (no price validation in the handler — that lives in the service).

Input: `curl -X POST /products -d '{"name":"Widget","price":9.99,"stock":100}'`

Output: `201 {"id":1,"name":"Widget","price":9.99,"stock":100}`

Acceptance: Write a unit test for `ProductService.Create` with no HTTP involved. Write a separate handler test using `httptest` that mocks the service. Run `go test ./...`

Interview

- What does “the handler must not contain business logic” mean concretely? Give an example of business logic that belongs in the service.
 - Why have a separate `ProductResponse` DTO instead of returning the domain `Product` directly?
 - How would you add an `internal_cost` field to `Product` that is stored but never returned by the API?
-

Source: `05-http-apis-service-ownership-and-chi/05-validation-with-playground-validator.md`

Unit 5 — Validation with go-playground/validator

Concept

Validate at the boundary — the HTTP handler — before the request reaches the service. Use struct tags for field-level rules: `required, min, max, gt, email`. Return structured error responses that name the failing field and explain the rule, not just a bare “400 Bad Request”. This lets API clients show useful error messages to users. Never return Go’s internal validation error strings directly — they expose implementation details and vary by library version.


```

package main

import (
    "encoding/json"
    "net/http"

    "github.com/go-playground/validator/v10"
)

// CreateProductRequest with validation tags.
type CreateProductRequest struct {
    Name string `json:"name" validate:"required,min=2,max=100"`
    Price float64 `json:"price" validate:"required,gt=0"`
    Stock int `json:"stock" validate:"min=0"`
}

// FieldError is one validation failure for one field.
type FieldError struct {
    Field string `json:"field"`
    Message string `json:"message"`
}

// ValidationErrorResponse is the structured 400 body.
type ValidationErrorResponse struct {
    Errors []FieldError `json:"errors"`
}

var validate = validator.New()

func init() {
    // Use the json tag name in error messages, not the Go field name.
    validate.RegisterTagNameFunc(func(fld interface{} Tag(string) string) string {
        return fld.Tag("json")
    })
}

// parseValidationErrors converts validator errors to our API shape.
func parseValidationErrors(err error) []FieldError {
    var errs []FieldError
    for _, ve := range err.(validator.ValidationErrors) {
        errs = append(errs, FieldError{
            Field: ve.Field(),
            Message: humanMessage(ve),
        })
    }
    return errs
}

func humanMessage(ve validator.FieldError) string {
    switch ve.Tag() {
    case "required":
        return "this field is required"
    case "min":
        return "must be at least " + ve.Param() + " characters"
    case "max":
        return "must be at most " + ve.Param() + " characters"
    case "gt":
        return "must be greater than " + ve.Param()
    default:
        return "invalid value"
    }
}

func writeJSON(w http.ResponseWriter, status int, v any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
}

```

```

    json.NewEncoder(w).Encode(v)
}

func createProductHandler(w http.ResponseWriter, r *http.Request) {
    var req CreateProductRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        writeJSON(w, http.StatusBadRequest, map[string]string{"error": "invalid JSON"})
        return
    }

    if err := validate.Struct(req); err != nil {
        writeJSON(w, http.StatusBadRequest, ValidationErrorResponse{
            Errors: parseValidationErrors(err),
        })
        return
    }

    // Validation passed — proceed to service layer.
    writeJSON(w, http.StatusCreated, map[string]any{
        "name": req.Name,
        "price": req.Price,
        "stock": req.Stock,
    })
}

func main() {
    http.HandleFunc("/products", createProductHandler)
    http.ListenAndServe(":8080", nil)
}

```

Exercise

Build: Add validation to `CreateProductRequest`: `name` required, min length 2; `price` required, greater than 0; `stock` min 0 (zero stock is allowed — product exists but is out of stock).

Input: Three bad requests: missing name, `price=-1`, `stock=-5`. One valid request.

Output:

```

{"errors": [{"field": "name", "message": "this field is required"}]}
{"errors": [{"field": "price", "message": "must be greater than 0"}]}
{"errors": [{"field": "stock", "message": "invalid value"}]}
201 Created (valid request)

```

Acceptance: Write `httptest` tests for each failure case. Assert the response body contains the correct `field` name (not the Go struct field name — use the `json` tag name). Run `go test ./...`.

Interview

- Why validate in the handler rather than in the service?
- What is the risk of returning validator library error strings directly to the client?
- How would you validate a field rule that depends on another field's value (e.g., `discount < price`)?

Source: `05-http-apis-service-ownership-and-chi/06-middleware-cross-cutting-stripes.md`

Unit 6 — Middleware cross-cutting fibres: tracing hooks, audit, auth placeholders

Learning outcome catalogue

Treat middleware onion ordering intentionally:

Suggested outer→inner illustrative ordering (adapt consciously):

```
panic/recovery sentinel (thin)
logging / correlation id enrichment
authentication / authorization scaffolding (possibly stub verifying header presence minimally)
metrics future hook comment anchors
routing entry
```

Practice capabilities:

```
request id attribution (propagate contextual logging downstream)
rudimentary structured logging scaffolding bridging next unit amplification
opaque future auth verifying static bearer token minimally if time permits—not security theatre pretending completeness—label clearly educational stubbing
```

Lab essay

Articulate distinctions **middleware vs handler** responsibilities preventing domain logic creep upward accidentally.

Enumerate danger compressing heavyweight DB transactions inside middleware—justify rarely exceptional cases if any—even hypothetical negatives primarily.

Source: 05-http-apis-service-ownership-and-chi/07-structured-logging-production-shape.md

Unit 7 — Structured logging readiness (`slog` or `zap`)

Transition motivation

Eliminate habitual `fmt.Println` tracing masquerading observability—it collapses grep-ability correlating outages.

Prefer keyed fields aligning:

```
logger.Info("creating order",
    "request_id", requestIDFromCtx,
    "user_id", userID)
```

Practice linking **middleware-injected identifiers** bridging each log emission minimum baseline contract.

Fabricate deliberate **internal 500** scenario verifying discoverability slicing logs later conceptually—even absent full ELK stack integration now.

Interview angles:

- cardinality cautions injecting high fan-out dynamic labels exploding metric/log storage costs unintentionally glimpsed verbally,
- PII scrubbing instincts even in learning exercises ethically.

Source: 05-http-apis-service-ownership-and-chi/08-error-wrapping-and-http-semantics.md

Unit 8 — Error wrapping + purposeful HTTP semantics (no silent bubbling)

Practice pattern emphasis:

```
fmt.Errorf("create order: %w", err)
```

Then boundary translation mapping internal errors selectively:

```
domain/repository distinctions → sanitized external API errors
versus diagnostic logging enriching internal-only detail fields
```

Anti-pattern spotlight

Returning raw sentinel strings upward unclassified—observer teams cannot categorize retry vs permanent failure cleanly.
Produce mini internal taxonomy—even if handwritten constants initially—staging future `errors.Is` adoption consciously.
Interview expectation: reconcile **explicit errors** bridging earlier fundamentals Area01 with transport mapping discipline now.

Source: `05-http-apis-service-ownership-and-chi/09-http-timeouts-and-context-propagation.md`

Unit 9 — Per-request timeouts & propagated context narrowing

Practice embedding synthetic slow payment collaborator sleeping beyond acceptable SLA enforcing earlier abort via tightened derived context timelines.

Demonstrate cooperating cancellation signals reaching nested repository fakers honouring `ctx.Done()`—even if repos still ephemeral memory until Area08 deepens Postgres realism.

Enumerate strategies when legacy dependency wrappers lack native `context` signatures—adaptation façade sketch patterns minimally.

Interview drill: summarise failure classification when client disconnects prematurely mid-handler—observe possible wasted work nuances articulating politely.

Source: `05-http-apis-service-ownership-and-chi/10-service-repository-shape.md`

Unit 10 — Service + repository seam rehearsal (thin handler religion)

Formalise layering narrative:

```
Handler (HTTP glue)
  → Application service orchestrating workflows + invariants bridging multiple repos eventually
  → Repository abstractions encapsulating persistence soon (temporary in-memory permissible learning stepping stone if documented transitional honesty)
```

Refactor drifting fat handlers surgically—even if simplistic domain rules currently.

Interview alignment previewing DDD-heavy Area 11 but pragmatic now: avoid cargo-cult aggregator services empty pass-through blindly—justify added layer cognitive tax via upcoming multi-repository coordination expectations honestly.

Source: `05-http-apis-service-ownership-and-chi/11-capstone-go-api-integration.md`

Unit 11 — `go-api/` capstone integration consolidation

Mandatory route surface practising integrated themes:

```
GET /orders
POST /orders
GET /products
GET /health
```

Stack obligations recap:

```
chi routing
explicit validation middleware / handler interplay
structured logging keyed by request/trace correlation fields
timeouts contextually aligned
explicit error envelopes / mapping strategy succinctly documented
middleware ordering rationale appendix short note
```

Deliverables (documentation-weighted—not code dump fetishisation)

Articulate succinctly:

```
ownership layering chain  
anticipated failure taxonomy example table (even if partial)  
latency budget sketch commentary at high-level  
upgrade evolution hook (serialization/gRPC bridging forward Area06)
```

Interview consolidation checklist previously scattered—must verbally chain story uninterrupted simulating onboarding walk-through.

Phase — 06-grpc-protobuf-and-service-contracts

Directory: 06-grpc-protobuf-and-service-contracts/

Source: 06-grpc-protobuf-and-service-contracts/00-quick-reference.md

Quick Reference — gRPC & Protobuf

proto3 basics

```
syntax = "proto3";
package user.v1;
option go_package = "github.com/org/repo/gen/user/v1;userv1";

message User {
    int64 id = 1;
    string email = 2;
}

service UserService {
    rpc GetUser(GetUserRequest) returns (User);
    rpc ListUsers(ListUsersRequest) returns (stream User);
}
```

Generate

```
protoc --go_out=. --go-grpc_out=. user.proto
```

Server setup

```
s := grpc.NewServer(
    grpc.UnaryInterceptor(loggingInterceptor),
)
userpb.RegisterUserServiceServer(s, &userServer{})
s.Serve(lis)
```

Unary interceptor shape

```
func loggingInterceptor(ctx context.Context, req any,
    info *grpc.UnaryServerInfo, handler grpc.UnaryHandler) (any, error) {
    start := time.Now()
    resp, err := handler(ctx, req)
    log.Printf("%s took %s", info.FullMethod, time.Since(start))
    return resp, err
}
```

Context deadline (client side)

```
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
resp, err := client.GetUser(ctx, &userpb.GetUserRequest{Id: 1})
```

Schema evolution rules

```
// SAFE: add new field (gets zero value on old clients)
// SAFE: rename field (wire uses field numbers, not names)
// UNSAFE: change field type or field number
```

Source: 06-grpc-protobuf-and-service-contracts/01-scope-protobuf-foundations.md

Unit 1 — Module Scope: gRPC Product Catalog Service

What You Will Build

A gRPC product catalog service — the `grpc-catalog/` codebase. A proto file defines the contract. Generated Go code provides the server interface and client stubs. You implement the server, write a client, add interceptors, and practice safe schema evolution.

By the end of this module you will have:

- A proto file with `ProductService` defining unary and server-streaming RPCs
- A Go gRPC server implementing the generated interface
- A Go client that calls the server using the generated stubs
- A logging interceptor that records method, duration, and error code
- Understanding of which proto changes are safe and which break the wire format

Why gRPC Instead of REST

gRPC uses HTTP/2 and protobuf binary encoding — messages are smaller and faster to parse than JSON. The contract is in the `.proto` file, not in documentation that gets out of sync. Code is generated from the schema — client and server are always aligned. Streaming is a first-class feature, not a bolt-on. The tradeoff: harder to debug (binary wire format), requires tooling (`protoc`, `buf`), and browser clients need a proxy (`grpc-web`).

Project Structure

```
grpc-catalog/
  proto/
    product/v1/product.proto    - service and message definitions
  gen/
    product/v1/                - generated Go code (do not edit)
  cmd/
    server/main.go             - gRPC server
    client/main.go             - command-line client
  internal/
    service/product_service.go - business logic, implements generated interface
  buf.yaml                    - buf tool configuration
  buf.gen.yaml                - code generation config
  Makefile                    - proto:generate target
```

The Wire Contract Rule

Proto field numbers are the wire contract, not field names. When protobuf encodes a message, each field is tagged with its number. If you reuse field number 3 for a different type after removing the original field 3, old clients will silently decode the wrong data. Field numbers are permanent. Safe changes: add new fields with new numbers, rename fields (names are not on the wire). Unsafe: change field type, reuse numbers, change from `repeated` to singular.

Unit 2 — Unary RPC: Server and Client

Concept

A unary RPC is the simplest pattern: one request, one response, like a function call over the network. The proto defines the contract — message types and the service. `protoc` generates Go interfaces: the server implements them, the client calls methods on a stub that handles the network. Error handling uses `status.Error(codes.NotFound, "message")` — not standard Go errors. Always embed `UnimplementedXxxServer` in your server struct so new RPC methods added to the proto do not break compilation.

Code

```
// proto/product/v1/product.proto
syntax = "proto3";
package product.v1;
option go_package = "github.com/example/grpc-catalog/gen/product/v1;productv1";

message Product {
    int64 id = 1;
    string name = 2;
    double price = 3;
    int32 stock = 4;
}

message GetProductRequest { int64 id = 1; }
message CreateProductRequest {
    string name = 1;
    double price = 2;
    int32 stock = 3;
}

service ProductService {
    rpc GetProduct (GetProductRequest) returns (Product);
    rpc CreateProduct (CreateProductRequest) returns (Product);
}
```

```

// cmd/server/main.go
package main

import (
    "context"
    "errors"
    "log"
    "net"
    "sync"

    "google.golang.org/grpc"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"

    pb "github.com/example/grpc-catalog/gen/product/v1"
)

type productServer struct {
    pb.UnimplementedProductServiceServer // forward-compatible: new methods won't break build
    mu      sync.RWMutex
    products map[int64]*pb.Product
    nextID   int64
}

func newProductServer() *productServer {
    return &productServer{
        products: make(map[int64]*pb.Product),
        nextID:   1,
    }
}

func (s *productServer) GetProduct(ctx context.Context, req *pb.GetProductRequest) (*pb.Product, error) {
    s.mu.RLock()
    defer s.mu.RUnlock()

    p, ok := s.products[req.Id]
    if !ok {
        // Use gRPC status codes – not fmt.Errorf
        return nil, status.Errorf(codes.NotFound, "product %d not found", req.Id)
    }
    return p, nil
}

func (s *productServer) CreateProduct(ctx context.Context, req *pb.CreateProductRequest) (*pb.Product, error) {
    if req.Price <= 0 {
        return nil, status.Error(codes.InvalidArgument, "price must be positive")
    }

    s.mu.Lock()
    defer s.mu.Unlock()

    p := &pb.Product{
        Id:      s.nextID,
        Name:    req.Name,
        Price:   req.Price,
        Stock:   req.Stock,
    }
    s.products[s.nextID] = p
    s.nextID++
    return p, nil
}

func main() {
    lis, err := net.Listen("tcp", ":50051")

```

```

    if err != nil {
        log.Fatalf("listen: %v", err)
    }
    srv := grpc.NewServer()
    pb.RegisterProductServiceServer(srv, newProductServer())
    log.Println("gRPC server listening on :50051")
    if err := srv.Serve(lis); err != nil {
        log.Fatalf("serve: %v", err)
    }
}

```

```

// cmd/client/main.go
package main

import (
    "context"
    "fmt"
    "log"
    "time"

    "google.golang.org/grpc"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/credentials/insecure"
    "google.golang.org/grpc/status"

    pb "github.com/example/grpc-catalog/gen/product/v1"
)

func main() {
    conn, err := grpc.NewClient("localhost:50051",
        grpc.WithTransportCredentials(insecure.NewCredentials()))
    if err != nil {
        log.Fatalf("connect: %v", err)
    }
    defer conn.Close()

    client := pb.NewProductServiceClient(conn)
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    // Create a product
    created, err := client.CreateProduct(ctx, &pb.CreateProductRequest{
        Name: "Widget",
        Price: 9.99,
        Stock: 100,
    })
    if err != nil {
        log.Fatalf("create: %v", err)
    }
    fmt.Printf("created: id=%d name=%s\n", created.Id, created.Name)

    // Get it back
    got, err := client.GetProduct(ctx, &pb.GetProductRequest{Id: created.Id})
    if err != nil {
        log.Fatalf("get: %v", err)
    }
    fmt.Printf("got: id=%d price=%.2f\n", got.Id, got.Price)

    // Get non-existent - check status code
    _, err = client.GetProduct(ctx, &pb.GetProductRequest{Id: 999})
    if s, ok := status.FromError(err); ok && s.Code() == codes.NotFound {
        fmt.Printf("expected not found: %v\n", s.Message())
    }
}

```


Exercise

Build: Implement `GetProduct` (returns `NotFound` if ID does not exist) and `CreateProduct` (returns `InvalidArgument` if price ≤ 0). Write an in-process test using `bufconn` (no real network needed).

Input: Create a product, then get it by ID. Try to get ID 999. Try to create with price -1.

Output: Correct responses and gRPC status codes for each case.

Acceptance: Test with `bufconn.Listen` — no real TCP port. Assert `codes.NotFound` for missing product, `codes.InvalidArgument` for bad price. Run `go test ./...`

Interview

- Why use `status.Errorf(codes.NotFound, ...)` instead of returning a standard Go error?
 - What does embedding `UnimplementedProductServiceServer` protect against?
 - How does a gRPC client check whether an error is a `NotFound` versus a network error?
-

Source: `06-grpc-protobuf-and-service-contracts/03-streaming-patterns.md`

Unit 3 — Streaming Patterns

Concept

Server streaming sends one request and receives a stream of responses — the client calls `Recv()` in a loop until it gets `io.EOF`. Use this for large result sets where returning everything in one response would be too large or too slow to assemble. Client streaming is the reverse: multiple requests, one response — useful for batch ingestion. Bidirectional streaming runs both simultaneously. Always check `stream.Context().Err()` before each send — if the client disconnects, stop sending instead of writing to a dead stream.

Code

```
// Add to product.proto
message ListProductsRequest {
    double max_price = 1; // 0 means no filter
}

service ProductService {
    rpc GetProduct    (GetProductRequest)    returns (Product);
    rpc CreateProduct (CreateProductRequest) returns (Product);
    rpc ListProducts  (ListProductsRequest)  returns (stream Product); // server streaming
}
```

```
// Server: implement ListProducts on productService
func (s *productServer) ListProducts(req *pb.ListProductsRequest, stream pb.ProductService_ListProduct
sServer) error {
    s.mu.RLock()
    defer s.mu.RUnlock()

    for _, p := range s.products {
        // Apply filter
        if req.MaxPrice > 0 && p.Price > req.MaxPrice {
            continue
        }

        // Check if client cancelled before sending each item
        if err := stream.Context().Err(); err != nil {
            return status.FromContextError(err).Err()
        }

        if err := stream.Send(p); err != nil {
            // Client disconnected or network error
            return err
        }
    }
    // Returning nil sends EOF to the client – stream is complete
    return nil
}
```

```
// Client: receive a stream of products
func listProducts(client pb.ProductServiceClient, maxPrice float64) ([]*pb.Product, error) {
    ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
    defer cancel()

    stream, err := client.ListProducts(ctx, &pb.ListProductsRequest{MaxPrice: maxPrice})
    if err != nil {
        return nil, fmt.Errorf("ListProducts: %w", err)
    }

    var products []*pb.Product
    for {
        p, err := stream.Recv()
        if err == io.EOF {
            break // server sent all products
        }
        if err != nil {
            return nil, fmt.Errorf("Recv: %w", err)
        }
        products = append(products, p)
    }
    return products, nil
}

func main() {
    // ... (connection setup same as unit 2)

    // Create some products first, then list filtered
    products, err := listProducts(client, 15.00) // only products under $15
    if err != nil {
        log.Fatal(err)
    }
    fmt.Printf("found %d products under $15\n", len(products))
    for _, p := range products {
        fmt.Printf("  %s: $%.2f\n", p.Name, p.Price)
    }
}
```

Exercise

Build: Add `ListProducts(filter) stream<Product>` to the proto and implement it on the server. The filter has a single field `max_price float64` — return all products if 0, otherwise only those below the price. Client collects the stream into a slice and prints the count.

Input: Create 5 products with prices 5, 10, 15, 20, 25. Call `ListProducts(max_price=16)`.

Output: 3 products returned (5, 10, 15).

Acceptance: Test with `bufconn`. Assert count=3 for `max_price=16`. Assert count=5 for `max_price=0`. Verify the server respects client cancellation (cancel context after receiving 2 items — server must not panic). Run `go test ./...`.

Interview

- When would you use server streaming instead of pagination with `GET /products?page=N`?
 - What does `stream.Recv()` return to signal the stream is complete?
 - What happens on the server if you call `stream.Send()` after the client has cancelled the context?
-

Source: `06-grpc-protobuf-and-service-contracts/04-interceptors-metadata-deadlines.md`

Unit 4 — Interceptors, Metadata, and Deadlines

Concept

Interceptors are middleware for gRPC — they run before and after every RPC call. A unary interceptor wraps a single call; a streaming interceptor wraps the stream lifecycle. Use them for logging, authentication, metrics, and panic recovery. Metadata is gRPC's equivalent of HTTP headers — key-value pairs sent alongside the request. Always propagate deadlines from the incoming context to any downstream calls — never create a fresh `context.Background()` inside a handler that already has a deadline.

Code

```
// Unary logging interceptor – logs method, duration, and error code for every call
func loggingInterceptor(
    ctx context.Context,
    req any,
    info *grpc.UnaryServerInfo,
    handler grpc.UnaryHandler,
) (any, error) {
    start := time.Now()

    // Extract request ID from metadata if present
    var requestID string
    if md, ok := metadata.FromIncomingContext(ctx); ok {
        if vals := md.Get("x-request-id"); len(vals) > 0 {
            requestID = vals[0]
        }
    }

    // Call the actual handler
    resp, err := handler(ctx, req)

    // Log after the call – captures real duration and status
    code := codes.OK
    if err != nil {
        code = status.Code(err)
    }

    log.Printf("method=%s request_id=%s code=%s duration=%v",
        info.FullMethod, requestID, code, time.Since(start).Round(time.Millisecond))

    return resp, err
}

// Recovery interceptor – converts panics into Internal gRPC errors
func recoveryInterceptor(
    ctx context.Context,
    req any,
    info *grpc.UnaryServerInfo,
    handler grpc.UnaryHandler,
) (resp any, err error) {
    defer func() {
        if r := recover(); r != nil {
            log.Printf("panic in %s: %v", info.FullMethod, r)
            err = status.Errorf(codes.Internal, "internal server error")
        }
    }()
    return handler(ctx, req)
}

// Wire interceptors on the server
func main() {
    lis, _ := net.Listen("tcp", ":50051")

    srv := grpc.NewServer(
        grpc.ChainUnaryInterceptor(
            recoveryInterceptor, // first: catch panics from all subsequent interceptors
            loggingInterceptor,  // second: log every call
        ),
    )

    pb.RegisterProductServiceServer(srv, newProductServer())
    log.Println("gRPC server with interceptors on :50051")
    srv.Serve(lis)
}
```

```
// Client: send metadata with every call
func callWithMetadata(client pb.ProductServiceClient, requestID string) {
    // Attach metadata to the context – server can read it via metadata.FromIncomingContext
    ctx := metadata.AppendToOutgoingContext(
        context.Background(),
        "x-request-id", requestID,
    )

    // Deadline: this request must complete in 2 seconds
    ctx, cancel := context.WithTimeout(ctx, 2*time.Second)
    defer cancel()

    p, err := client.GetProduct(ctx, &pb.GetProductRequest{Id: 1})
    if err != nil {
        if status.Code(err) == codes.DeadlineExceeded {
            fmt.Printf("[%s] deadline exceeded\n", requestID)
        }
        return
    }
    fmt.Printf("[%s] got: %s\n", requestID, p.Name)
}
```

Exercise

Build: Add the logging interceptor to your server. It must log: method name, `x-request-id` from metadata (or `-` if absent), gRPC status code, and duration in milliseconds.

Input: Call `GetProduct` from a client that sets `x-request-id: req-123` in metadata. Call it again without the metadata header.

Output:

```
method=/product.v1.ProductService/GetProduct request_id=req-123 code=OK duration=1ms
method=/product.v1.ProductService/GetProduct request_id=- code=NotFound duration=0ms
```

Acceptance: Write a test that calls the server via `bufconn` and checks that the log output contains the method name and status code. Run `go test ./...`.

Interview

- What is the difference between a unary interceptor and a streaming interceptor?
- Why must you propagate the incoming context to downstream calls rather than creating a new one?
- How does gRPC metadata differ from HTTP headers in terms of transport and API?

Source: [06-grpc-protobuf-and-service-contracts/05-schema-evolution-compat.md](#)

Unit 5 — Schema Evolution and Compatibility

Concept

Proto field numbers are the wire contract — they are embedded in every serialized message. Changing or reusing a field number after removing the original field causes silent data corruption: old clients read the new field's bytes and interpret them as the old type. Field numbers are permanent. Safe changes: add a new field with a new number, rename a field (names are not on the wire). Unsafe: change field type, reuse a retired number, change `repeated` to singular. Use `reserved` to prevent future accidents when removing fields.

Code

```
// product.proto – version 1 (original)
message Product {
  int64 id = 1;
  string name = 2;
  double price = 3;
  int32 stock = 4;
}
```

```
// product.proto – version 2 (safe evolution: new field added)
message Product {
  int64 id = 1;
  string name = 2;
  double price = 3;
  int32 stock = 4;
  string category = 5; // NEW: safe – uses a new field number
}

// What happens when old client receives a v2 message:
// - Fields 1-4 are decoded normally (same numbers, same types)
// - Field 5 (category) is UNKNOWN to the old client – it is silently ignored
// - Old client sees a valid Product with category == "" (zero value)
// This is backward compatible.
```

```
// product.proto – version 3 (removing a field safely)
message Product {
  int64 id = 1;
  string name = 2;
  double price = 3;
  // stock removed – MUST reserve the number to prevent accidental reuse
  reserved 4;
  reserved "stock"; // reserve the name too (best practice)

  string category = 5;
  // WRONG (do not do this): reusing field 4 for a new field
  // string sku = 4; // would decode old stock bytes as a string – silent corruption
}
```

```
// What breaks compatibility – do not do these:
message BadEvolution {
  int64 id = 1;
  string name = 2;
  // WRONG: changed field 3 from double to string
  string price = 3; // old clients expect double bytes – silent corruption

  // WRONG: reused field 4 after removing stock
  string sku = 4; // old clients decode sku bytes as int32 stock
}
```

```
// Verifying backward compatibility in a test:
// Serialize a v2 Product (with category), deserialize as v1 Product.
// Fields 1-4 must decode correctly. Field 5 (category) must be silently ignored.

func TestBackwardCompatibility(t *testing.T) {
    // v2 message with category set
    v2 := &productv2.Product{
        Id:      1,
        Name:    "Widget",
        Price:   9.99,
        Stock:   100,
        Category: "electronics",
    }

    // Serialize to bytes
    data, err := proto.Marshal(v2)
    if err != nil {
        t.Fatal(err)
    }

    // Deserialize as v1 (no category field)
    v1 := &productv1.Product{}
    if err := proto.Unmarshal(data, v1); err != nil {
        t.Fatal(err)
    }

    // v1 fields must be intact — category is ignored
    if v1.Id != 1 || v1.Name != "Widget" || v1.Price != 9.99 {
        t.Errorf("backward compat broken: %+v", v1)
    }

    // v1 does not have Category field — this is expected
}
```

Exercise

Build: Add a `category` string field as field number 5 to your `Product` proto. Generate the new code. Write a backward compatibility test: serialize a v2 `Product` (with category set), deserialize it using the v1 generated struct (without the category field), assert that fields 1-4 are still correct.

Input: `Product{id:1, name:"Widget", price:9.99, stock:100, category:"electronics"}` serialized as v2.

Output: When deserialized as v1: `id=1, name="Widget", price=9.99, stock=100`. Category is ignored without error.

Acceptance: Test passes. Add a `reserved 6;` line to the proto for a field you plan to add later — this documents the intent. Run `go test ./...`.

Interview

- Why is reusing a proto field number after removing the original field so dangerous?
- A colleague wants to rename field `price` to `unit_price` in the proto. Is this safe? What changes at the wire level versus the API level?
- What does `reserved` do in a proto file, and why should you add it when removing a field?

Source: `06-grpc-protobuf-and-service-contracts/06-capstone-multi-service-grpc-flow.md`

Unit 6 — Multi-service gRPC choreography capstone sketch

Architect narrative—even if trimmed executable slice:

```
api façade service
payment microservice collaborator
inventory microservice collaborator communicating via grpc (local docker-compose optional—not mandatory if resource constrained articulate honestly)
```

Cross-cutting behaviours embed intentionally

timeouts • naive bounded retry scaffolding (justify caution overlapping idempotency area forward) • trace hook placeholders • propagated cancellation scenarios

Produce written sequence diagram-esque bullet timeline narrating synchronous chain fragility hotspots contrasted hypothetical asynchronous escalation patterns previewing Areas 12+/13.

Interview expectation: cohesive story **without insisting finished production hardening artefacts** unrealistic at learning boundary—articulate deltas explicitly matured later.

Phase — 07-serialization-http-grpc-tradeoffs

Directory: 07-serialization-http-grpc-tradeoffs/

Source: 07-serialization-http-grpc-tradeoffs/00-quick-reference.md

Quick Reference — Serialization

JSON struct tags

```
json:"name" // rename field  json:"name,omitempty" // skip if zero/nil/empty  json:"- " // always skip
```

Common pitfalls

- Unexported fields are silently ignored
- nil slice → null; empty slice → [] (know the difference)
- time.Time: marshals to RFC3339 by default
- Integers > 2⁵³ lose precision in JS — use string

Protobuf field rules

- Never reuse or change a field number
- Safe: add new field, rename field, add new message
- Unsafe: change field type, change field number, remove required

When to use what

JSON: external APIs, webhooks, config files, logs Proto: internal RPC, high-throughput pipelines

Source: 07-serialization-http-grpc-tradeoffs/01-json-protobuf-boundaries-internal-vs-external.md

Unit 1 — JSON vs Protobuf: Where Each Belongs

Concept

JSON is for external APIs — browsers, mobile clients, third-party integrations. It is human-readable, self-describing, and universally supported without tooling. Protobuf is for internal service-to-service communication. It is compact, schema-enforced, and generates typed code for both sides of the wire. Using JSON over internal gRPC loses type safety and the compact encoding. Exposing raw proto over a public HTTP API breaks discoverability — clients cannot inspect or document it without your .proto files.

Code

```
package main

import (
    "encoding/json"
    "fmt"
    "log"

    "google.golang.org/protobuf/proto"

    pb "github.com/example/orders/gen/orderpb"
)

// OrderJSON is the external API representation.
type OrderJSON struct {
    ID          string `json:"id"`
    CustomerID  string `json:"customer_id"`
    Total       float64 `json:"total"`
    Status      string `json:"status"`
}

func main() {
    // JSON encoding — for external consumers
    order := OrderJSON{
        ID:          "ord_123",
        CustomerID:  "cust_456",
        Total:       99.99,
        Status:      "pending",
    }
    jsonBytes, err := json.Marshal(order)
    if err != nil {
        log.Fatal(err)
    }
    fmt.Printf("JSON (%d bytes): %s\n", len(jsonBytes), jsonBytes)
    // JSON (67 bytes): {"id":"ord_123","customer_id":"cust_456","total":99.99,"status":"pending"}

    // Protobuf encoding — for internal services
    pbOrder := &pb.Order{
        Id:          "ord_123",
        CustomerId:  "cust_456",
        Total:       99.99,
        Status:      "pending",
    }
    pbBytes, err := proto.Marshal(pbOrder)
    if err != nil {
        log.Fatal(err)
    }
    fmt.Printf("Proto (%d bytes): binary, not human-readable\n", len(pbBytes))
    // Proto (~32 bytes): roughly 50% smaller, no field names transmitted
}
```

Exercise

Build: An encoding comparison tool for an Order struct. **Input:** A hardcoded order with 5 fields: id, customer_id, item count (int), total (float64), status. **Output:** Two files written to disk — `order.json` and `order.pb` — plus a printed line: `JSON: 87 bytes | Proto: 38 bytes | Savings: 56%`. **Acceptance:** Both files exist. `cat order.json` shows readable JSON. `cat order.pb` shows garbage binary. The savings percentage in the output is mathematically correct.

Interview

- Why would you never expose a proto-only endpoint on a public REST API?
- If JSON is slower and larger than protobuf, why is it still the right choice for external APIs?

- A teammate wants to use `encoding/json` over an internal gRPC call “because it’s easier to debug.” What is the actual cost of that decision?
-

Source: `07-serialization-http-grpc-tradeoffs/02-json-encoding-patterns-and-gotchas.md`

Unit 2 — JSON Encoding Patterns and Gotchas

Concept

`json.Encoder` writes directly to an `io.Writer` — use it for HTTP responses to avoid allocating the full body in memory. `json.Decoder` reads from an `io.Reader` — use it for request bodies. Three common gotchas trip up production code: a nil slice marshals to `null`, not `[]` — clients break on this; unexported struct fields are silently ignored with no error; `time.Time` marshals to RFC3339 by default, which is correct but must be consistent across your API.

Code

```
package main

import (
    "encoding/json"
    "fmt"
    "net/http"
    "time"
)

type CreateOrderRequest struct {
    Items      []string `json:"items"`
    CreatedAt  time.Time `json:"created_at"`
    note      string    // unexported – silently ignored by json
}

type OrderResponse struct {
    ID         string    `json:"id"`
    Items      []string  `json:"items"`
    CreatedAt  time.Time `json:"created_at"`
}

func handleCreateOrder(w http.ResponseWriter, r *http.Request) {
    var req CreateOrderRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "invalid JSON", http.StatusBadRequest)
        return
    }

    // Gotcha: nil slice vs empty slice
    // If client sends {"items": null}, req.Items is nil here.
    // If client sends {"items": []}, req.Items is an empty non-nil slice.
    if req.Items == nil {
        http.Error(w, `"items" must not be null`, http.StatusBadRequest)
        return
    }

    resp := OrderResponse{
        ID:         "ord_789",
        Items:      req.Items,    // non-nil: marshals to []
        CreatedAt:  time.Now().UTC(),
    }

    w.Header().Set("Content-Type", "application/json")
    // Encoder writes directly to ResponseWriter – no intermediate []byte allocation
    if err := json.NewEncoder(w).Encode(resp); err != nil {
        // At this point headers are sent; we can only log
        fmt.Printf("encode error: %v\n", err)
    }
}

func nilVsEmpty() {
    var nilSlice []string    // nil
    emptySlice := []string{} // non-nil, empty

    nilJSON, _ := json.Marshal(nilSlice)
    emptyJSON, _ := json.Marshal(emptySlice)

    fmt.Println(string(nilJSON)) // null – breaks most JS clients
    fmt.Println(string(emptyJSON)) // [] – correct
}
```

Exercise

Build: An HTTP handler that accepts `POST /orders` with body `{"items": [...], "created_at": "..."} .` **Input:** Three test requests: (1) valid body with items, (2) `{"items": null, ...}`, (3) `{"items": [], ...}`. **Output:** Request 1 returns 200. Request 2 returns 400 with message `"items" must not be null`. Request 3 returns 200 (empty slice is valid). **Acceptance:** Add `omitempty` to the `Items` field tag and re-run — verify request 2 now returns 200 (the bug is masked). Remove `omitempty`. Explain in a comment why `omitempty` is dangerous on required slice fields.

Interview

- What is the difference between `json.Marshal` and `json.NewEncoder(w).Encode` for HTTP responses?
 - A client receives `"items": null` from your API and their JS code crashes. What went wrong on the server?
 - Why does `time.Time` marshal correctly without a custom marshaler, and what format does it use?
-

Source: `07-serialization-http-grpc-tradeoffs/03-protobuf-encoding-and-wire-format-basics.md`

Unit 3 — Protobuf Encoding and Wire Format Basics

Concept

Protobuf encodes each field as a tag-value pair where the tag is `(field_number << 3) | wire_type`. Field names never go over the wire — only numbers. This is why field numbers are sacred: if you change field number 1 from `id` to `user_id`, every existing encoded message in storage or in-flight will decode incorrectly. Adding a new optional field with a new number is safe — old decoders simply skip unknown field numbers, which gives forward compatibility for free.

Code

```
package main

import (
    "fmt"
    "log"

    "google.golang.org/protobuf/proto"

    pb "github.com/example/events/gen/eventpb"
)

// Proto definition for reference:
//
// message Event {
//     string id      = 1;
//     string type     = 2;
//     bytes  payload = 3;
//     int64  ts       = 4;
//     // string source = 5; // added later
// }

func main() {
    // Encode with original 4-field message
    original := &pb.Event{
        Id:      "evt_001",
        Type:    "order.created",
        Payload: []byte(`{"order_id":"ord_123"}`),
        Ts:      1716000000,
    }

    data, err := proto.Marshal(original)
    if err != nil {
        log.Fatal(err)
    }
    fmt.Printf("Encoded: %d bytes\n", len(data))

    // Forward compatibility: decode old bytes with new struct that has Source field 5.
    // The new field will be zero-valued — old bytes decode cleanly.
    newMsg := &pb.EventV2{} // same fields 1-4 + Source at field 5
    if err := proto.Unmarshal(data, newMsg); err != nil {
        log.Fatal(err)
    }

    fmt.Printf("ID:      %s\n", newMsg.Id)           // "evt_001" — preserved
    fmt.Printf("Type:    %s\n", newMsg.Type)         // "order.created" — preserved
    fmt.Printf("Source:  %s\n", newMsg.Source)       // "" — zero value, not an error
}
```

Exercise

Build: A forward-compatibility demonstration. **Input:** Define an `Event` proto with fields: `id` (1), `type` (2), `payload` (3), `timestamp` (4). Marshal one event to bytes and save to `event_v1.pb`. **Output:** Define `EventV2` with the same 4 fields plus `source string = 5`. Unmarshal `event_v1.pb` bytes into an `EventV2`. Print all fields — the first 4 should have their values, `source` should be empty string. **Acceptance:** No error on unmarshal. All original fields print correctly. Now try changing field 2's number from 2 to 6 in your V2 struct and unmarshal again — observe that `type` is now empty. Document why in a comment.

Interview

- Why are protobuf field numbers immutable once a schema is in production?
- What happens when a new decoder receives a message with an unknown field number?
- Why can't you rely on field names for forward compatibility the way you might with JSON?

Unit 4 — Choosing Serialization: Decision Matrix

Concept

The choice between JSON and protobuf is about audience and contract enforcement, not performance. External consumers (browsers, mobile apps, third parties) need JSON — it is self-describing, requires no tooling, and is debuggable with any HTTP client. Internal services need protobuf — the schema is the contract, type mismatches fail at compile time, and the encoding is compact. Never choose based on raw speed alone. A gateway service translates between the two: proto internally, JSON externally.

Code

```
// Serialization decision matrix
//
// | Scenario                                | Format   | Reason                                                                 |
// |-----|-----|-----|
// | Public REST API (browsers/mobile) | JSON    | Self-describing, no client tooling needed |
// | Internal gRPC service call         | Protobuf| Type-safe, compact, schema-enforced      |
// | Kafka event payload (internal)     | Protobuf| Schema registry, compact, versioned      |
// | Kafka event payload (external)     | JSON    | Consumer has no .proto files             |
// | Stored DB blob                     | Protobuf| Compact, schema evolution via field numbers |
// | Log/audit trail                     | JSON    | Must be human-readable without tooling   |

package main

import (
    "encoding/json"
    "net/http"

    "google.golang.org/protobuf/proto"

    pb "github.com/example/orders/gen/orderpb"
)

// Gateway: receives JSON from client, calls internal service via proto.
func orderGatewayHandler(w http.ResponseWriter, r *http.Request) {
    // 1. Decode external JSON request
    var req struct {
        CustomerID string `json:"customer_id"`
        ItemIDs    []string `json:"item_ids"`
    }
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "bad request", http.StatusBadRequest)
        return
    }

    // 2. Translate to internal proto and call downstream service
    pbReq := &pb.CreateOrderRequest{
        CustomerId: req.CustomerID,
        ItemIds:    req.ItemIDs,
    }
    pbBytes, _ := proto.Marshal(pbReq)
    _ = pbBytes // send to internal gRPC service

    // 3. Translate proto response back to JSON for the client
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(map[string]string{"status": "created"})
}
```

Exercise

Build: A written serialization strategy (in comments in a `.go` file) for three systems, plus working gateway translation code.

Input: Three scenarios: (a) public REST API for a mobile app, (b) internal order→inventory service call over gRPC, (c) event payload published to Kafka for internal consumers. **Output:** For each scenario: chosen format, one-sentence justification, and any trade-offs. Plus a gateway function that decodes a JSON `PlaceOrderRequest` and encodes it as a proto

`InternalOrderRequest`. **Acceptance:** Justifications address audience, not just performance. Gateway compiles and the translation is lossless — all fields from JSON appear in the proto output.

Interview

- A teammate says “just use JSON everywhere, it's simpler.” What are the concrete costs at scale?
 - When would you store event payloads in Kafka as JSON instead of protobuf?
 - What does a gateway service's job look like when it sits between a JSON public API and a proto internal service?
-

Phase — 08-persistence-sql-postgresql-and-sqlx

Directory: 08-persistence-sql-postgresql-and-sqlx/

Source: 08-persistence-sql-postgresql-and-sqlx/00-quick-reference.md

Quick Reference — PostgreSQL & sqlx

Connect

```
db, err := sqlx.Connect("pgx", os.Getenv("DATABASE_URL")) db.SetMaxOpenConns(25) db.SetMaxIdleConns(5)
db.SetConnMaxLifetime(5 * time.Minute)
```

Query patterns

```
db.GetContext(ctx, &dest, query, args...) // one row db.SelectContext(ctx, &dest, query, args...) // many rows db.ExecContext(ctx,
query, args...) // no rows returned db.NamedExecContext(ctx, query, struct{}) // named params
```

Struct tags

```
type User struct { ID int db:"id" Email string db:"email" }
```

NULL handling

```
type NullableUser struct { Bio sql.NullString db:"bio" } // check: u.Bio.Valid before u.Bio.String
```

Transaction

```
tx, _ := db.BeginTx(ctx, nil) defer tx.Rollback() // ... work ... tx.Commit()
```

Error sentinel

```
errors.Is(err, sql.ErrNoRows) // not found
```

Source: 08-persistence-sql-postgresql-and-sqlx/01-scope-persistence-capstone-order-system.md

Unit 1 — Scope: Postgres-Backed Order System

What You Are Building

By the end of this module you will have a working Postgres-backed order system with four tables: `users`, `products`, `orders`, `order_items`. The system uses `sqlx` for queries, a repository pattern for DB access, and transactions for multi-step writes.

What Each Unit Covers

Unit	Topic	Deliverable
2	Schema design with constraints	<code>schema.sql</code> with all 4 tables
3	sqlx query patterns	<code>GetContext</code> , <code>SelectContext</code> , <code>NamedExecContext</code>
4	Repository layer	Interface + Postgres implementation
5	Transactions	Atomic order placement with rollback

The Data Model

```
users
  id          UUID PRIMARY KEY
  email       TEXT UNIQUE NOT NULL
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()

products
  id          UUID PRIMARY KEY
  name        TEXT NOT NULL
  price       NUMERIC(10,2) NOT NULL CHECK (price > 0)
  stock       INT NOT NULL DEFAULT 0 CHECK (stock >= 0)

orders
  id          UUID PRIMARY KEY
  user_id     UUID NOT NULL REFERENCES users(id)
  status      TEXT NOT NULL DEFAULT 'pending'
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()

order_items
  id          UUID PRIMARY KEY
  order_id    UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE
  product_id  UUID NOT NULL REFERENCES products(id)
  quantity    INT NOT NULL CHECK (quantity > 0)
  unit_price  NUMERIC(10,2) NOT NULL CHECK (unit_price > 0)
```

What Success Looks Like

You can call `PlaceOrder(ctx, userID, items)` and: - A row appears in `orders` - Rows appear in `order_items` for each item - If any item insert fails, the entire order is absent from the DB - You can retrieve the order with all items by ID using a single query

No Scope Creep

This module does not cover: ORMs, query builders, connection pooling tuning, or schema migrations. Those are later units. Build the fundamentals correctly first.

Source: `08-persistence-sql-postgresql-and-sqlx/02-relational-modeling-constraints.md`

Unit 2 — Relational Modeling and Constraints

Concept

Constraints enforce business rules at the database level — they survive application bugs, bypassed code paths, and direct SQL access. `CHECK` encodes business invariants like price must be positive. `NOT NULL` marks required fields so the DB rejects incomplete records, not just your application code. `REFERENCES` enforces foreign key integrity so orphaned records are impossible. `DEFAULT` provides sensible initial state without requiring every `INSERT` to specify every column. Constraints are documentation that the database enforces.

Code

```
-- schema.sql

CREATE EXTENSION IF NOT EXISTS "pgcrypto";

CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       TEXT UNIQUE NOT NULL,
  name        TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE products (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  price       NUMERIC(10,2) NOT NULL CHECK (price > 0),
  stock       INT NOT NULL DEFAULT 0 CHECK (stock >= 0),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE orders (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id),
  status      TEXT NOT NULL DEFAULT 'pending'
              CHECK (status IN ('pending', 'confirmed', 'shipped', 'cancelled')),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_orders_user_id ON orders(user_id);

CREATE TABLE order_items (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id    UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  product_id  UUID NOT NULL REFERENCES products(id),
  quantity    INT NOT NULL CHECK (quantity > 0),
  unit_price  NUMERIC(10,2) NOT NULL CHECK (unit_price > 0)
);

CREATE INDEX idx_order_items_order_id ON order_items(order_id);
```

Exercise

Build: A product catalog schema — `products` and `categories` tables. **Input:** `products` has: `id`, `name`, `price`, `stock`, `category_id`. `categories` has: `id`, `name`. **Output:** A `catalog_schema.sql` file with both tables. Add: `CHECK (price > 0)`, `CHECK (stock >= 0)`, a foreign key from `products.category_id` to `categories.id`, and an index on `category_id`. **Acceptance:** Run `psql -f catalog_schema.sql` — no errors. Try `INSERT INTO products (name, price, ...) VALUES ('Widget', -5.00, ...)` — verify it fails with a constraint violation. Try inserting a product with a non-existent `category_id` — verify it fails with a foreign key violation.

Interview

- Why define `CHECK (price > 0)` in the schema instead of validating in the application layer?
 - What does `ON DELETE CASCADE` do on `order_items.order_id`, and when is it appropriate?
 - A product's stock hits -3 after a race condition in your application. Which constraint would have prevented this?
-

Source: 08-persistence-sql-postgresql-and-sqlx/03-sqlx-query-patterns.md

Unit 3 — sqlx Query Patterns

Concept

`sqlx` adds struct scanning on top of `database/sql` without hiding SQL from you. Three methods cover most use cases: `GetContext` fetches exactly one row and returns `sql.ErrNoRows` if nothing matches — wrap this as your domain `ErrNotFound`. `SelectContext` fetches multiple rows into a slice. `NamedExecContext` maps a struct's fields to named SQL parameters using `db:` tags, which prevents argument-order bugs in multi-column inserts. Always pass a context — it carries deadlines and cancellation signals that abort in-flight queries.


```

package repository

import (
    "context"
    "database/sql"
    "errors"
    "time"

    "github.com/jmoiern/sqlx"
)

var ErrNotFound = errors.New("not found")

type Order struct {
    ID          string    `db:"id"`
    UserID      string    `db:"user_id"`
    Status      string    `db:"status"`
    CreatedAt   time.Time `db:"created_at"`
}

type OrderRepo struct {
    db *sqlx.DB
}

func NewOrderRepo(db *sqlx.DB) *OrderRepo {
    return &OrderRepo{db: db}
}

// FindByID returns one order or ErrNotFound.
func (r *OrderRepo) FindByID(ctx context.Context, id string) (*Order, error) {
    var order Order
    err := r.db.GetContext(ctx, &order,
        `SELECT id, user_id, status, created_at FROM orders WHERE id = $1`, id)
    if errors.Is(err, sql.ErrNoRows) {
        return nil, ErrNotFound
    }
    if err != nil {
        return nil, err
    }
    return &order, nil
}

// ListByUserID returns all orders for a user. Returns empty slice, not nil, if none.
func (r *OrderRepo) ListByUserID(ctx context.Context, userID string) ([]Order, error) {
    orders := []Order{} // initialize to empty slice, not nil
    err := r.db.SelectContext(ctx, &orders,
        `SELECT id, user_id, status, created_at FROM orders WHERE user_id = $1 ORDER BY created_at
DESC`,
        userID)
    return orders, err
}

// Create inserts an order and returns the generated ID.
func (r *OrderRepo) Create(ctx context.Context, userID string) (*Order, error) {
    order := &Order{UserID: userID, Status: "pending"}
    rows, err := r.db.NamedExecContext(ctx,
        `INSERT INTO orders (user_id, status) VALUES (:user_id, :status) RETURNING id, created_at`,
        order)
    if err != nil {
        return nil, err
    }
    defer rows.Close()
    return order, nil
}

```

Exercise

Build: An `OrderRepository` with three methods matching the signatures above. **Input:** A running Postgres DB with the schema from Unit 2. **Output:** Working queries for `FindByID` (one row), `ListByUserID` (many rows), `Create` (insert with named params). Map `sql.ErrNoRows` to your own `ErrNotFound` in `FindByID`. **Acceptance:** Write a `main.go` that calls all three. `FindByID` with a fake UUID returns `ErrNotFound`. `ListByUserID` for a user with no orders returns an empty slice (not `nil` — check with `len == 0` and `result != nil`). `Create` returns an order with a populated ID.

Interview

- What is the difference between `GetContext` and `SelectContext` in `sqlx`?
 - Why does `sql.ErrNoRows` need to be mapped to a domain error before leaving the repository?
 - Why use `NamedExecContext` over positional parameters for a 6-column INSERT?
-

Source: `08-persistence-sql-postgresql-and-sqlx/04-repository-layer-ownership.md`

Unit 4 — Repository Layer Ownership

Concept

A repository wraps database access behind an interface. The service calls the interface and never imports `sqlx` or `database/sql` directly. The interface lives in the service package — it expresses what the service needs, not what the database can do. The implementation lives in a separate `repository` package. This separation makes the service testable with a mock and makes the storage technology swappable without touching business logic.


```

// domain/order.go – shared types, no imports from service or repository
package domain

import "time"

type Order struct {
    ID          string
    UserID      string
    Status      string
    CreatedAt   time.Time
}

// ---

// service/order_service.go – interface defined here, owned by the service
package service

import (
    "context"
    "github.com/example/app/domain"
)

// OrderRepository is what the service needs. Lives in the service package.
type OrderRepository interface {
    FindByID(ctx context.Context, id string) (*domain.Order, error)
    ListByUserID(ctx context.Context, userID string) ([]domain.Order, error)
    Create(ctx context.Context, userID string) (*domain.Order, error)
}

type OrderService struct {
    orders OrderRepository
}

func NewOrderService(orders OrderRepository) *OrderService {
    return &OrderService{orders: orders}
}

func (s *OrderService) GetOrder(ctx context.Context, id string) (*domain.Order, error) {
    return s.orders.FindByID(ctx, id)
}

// ---

// repository/postgres_order.go – implementation, imports sqlx
package repository

import (
    "context"
    "database/sql"
    "errors"

    "github.com/jmoirn/sqlx"
    "github.com/example/app/domain"
)

type PostgresOrderRepo struct {
    db *sqlx.DB
}

func NewPostgresOrderRepo(db *sqlx.DB) *PostgresOrderRepo {
    return &PostgresOrderRepo{db: db}
}

func (r *PostgresOrderRepo) FindByID(ctx context.Context, id string) (*domain.Order, error) {
    var row struct {
        ID      string `db:"id"`
    }

```

```

    UserID string `db:"user_id"`
    Status string `db:"status"`
}
err := r.db.GetContext(ctx, &row, `SELECT id, user_id, status FROM orders WHERE id = $1`, id)
if errors.Is(err, sql.ErrNoRows) {
    return nil, ErrNotFound
}
if err != nil {
    return nil, err
}
return &domain.Order{ID: row.ID, UserID: row.UserID, Status: row.Status}, nil
}

```

Exercise

Build: An `OrderRepository` interface and `PostgresOrderRepo` implementation with four methods: `FindByID`, `Create`, `ListByUserID`, `UpdateStatus`. **Input:** The interface lives in `service/`. The implementation lives in `repository/`. Domain types live in `domain/`. **Output:** The service package imports `domain` but never imports `repository` or `sqlx`. The `repository` package imports `sqlx` and `domain` but never imports `service`. **Acceptance:** Draw the import graph in a comment in `service/order_service.go`. Run `go build ./...` — no import cycles. Confirm that `service.OrderService` can be constructed with a hand-rolled mock that satisfies `OrderRepository` in a test file.

Interview

- Why should the repository interface live in the service package, not the repository package?
 - What does it mean for the service to “not know what database it’s using”?
 - If you replace Postgres with DynamoDB next quarter, which files change and which do not?
-

Source: `08-persistence-sql-postgresql-and-sqlx/05-transactions-and-failure.md`

Unit 5 — Transactions and Failure

Concept

A transaction groups multiple operations so they all succeed or all fail together. In `sqlx`: call `BeginTxx` to start, run your operations on the transaction, then `Commit`. If anything fails, call `Rollback`. The safe pattern is to `defer tx.Rollback()` immediately after `BeginTxx` — after a successful `Commit`, `Rollback` is a no-op and does nothing. If you forget the `defer` and an early return skips `Rollback`, the transaction holds locks until it times out.

Code

```
package repository

import (
    "context"
    "fmt"

    "github.com/jmoiern/sqlx"
    "github.com/example/app/domain"
)

type OrderItem struct {
    ProductID string
    Quantity  int
    UnitPrice float64
}

func (r *PostgresOrderRepo) PlaceOrder(
    ctx context.Context,
    userID string,
    items []OrderItem,
) (*domain.Order, error) {
    tx, err := r.db.BeginTx(ctx, nil)
    if err != nil {
        return nil, fmt.Errorf("begin tx: %w", err)
    }
    defer tx.Rollback() // no-op if Commit succeeds

    // Step 1: insert the order
    var orderID string
    err = tx.QueryRowContext(ctx,
        `INSERT INTO orders (user_id, status) VALUES ($1, 'pending') RETURNING id`,
        userID,
    ).Scan(&orderID)
    if err != nil {
        return nil, fmt.Errorf("insert order: %w", err) // Rollback fires via defer
    }

    // Step 2: insert each item — if any fails, the whole order rolls back
    for i, item := range items {
        _, err = tx.ExecContext(ctx,
            `INSERT INTO order_items (order_id, product_id, quantity, unit_price)
            VALUES ($1, $2, $3, $4)`,
            orderID, item.ProductID, item.Quantity, item.UnitPrice,
        )
        if err != nil {
            return nil, fmt.Errorf("insert item %d: %w", i, err) // Rollback fires via defer
        }
    }

    if err := tx.Commit(); err != nil {
        return nil, fmt.Errorf("commit: %w", err)
    }

    return &domain.Order{ID: orderID, UserID: userID, Status: "pending"}, nil
}
```

Exercise

Build: A `PlaceOrder` function that inserts an order and its items atomically. **Input:** A user ID and a slice of 3 order items. On the third item, deliberately return an error (simulate a constraint violation with an invalid product ID). **Output:** The function returns an error. The `orders` table has zero rows for this attempted order. The `order_items` table has zero rows for this order ID. **Acceptance:** After the failed call, query `SELECT COUNT(*) FROM orders` — count is unchanged. Query `SELECT COUNT(*)`

`FROM order_items` — also unchanged. Then run it with valid items and verify both tables have the expected rows. Confirm `defer tx.Rollback()` is always present — remove it and show what happens with an early return.

Interview

- Why is `defer tx.Rollback()` safe to call after a successful `Commit`?
 - A developer forgets to call `Rollback` on error. What is the operational impact?
 - What does “atomicity” mean for the `PlaceOrder` operation in concrete terms?
-

Source: `08-persistence-sql-postgresql-and-sqlx/06-locking-and-consistency-races.md`

Unit 6 — Locking flavours & concurrency hazards (lost updates narrative)

Interpret **optimistic** (often version counters / conditional updates) versus **pessimistic** (`SELECT ... FOR UPDATE` conceptual awareness) interplay trade spectrum.

Practice contention scenario:

two simultaneous purchasers dwindling scarce inventory provoking race glimpses—implement guard strategy variant articulating anomaly prevention claims honestly referencing isolation knowledge bounds.

Interview nuance distinguishing application-level reconciliation vs leaning DB primitives consciously—ethical complexity admission encouraged vs hand-wavy “DB handles everything” mythology debunk politely.

Source: `08-persistence-sql-postgresql-and-sqlx/07-schema-migrations-operational-hygiene.md`

Unit 7 — Migrations hygiene: repeatable `up` / `down` rehearsal discipline

Operational truth:

Production abhors improvised `ALTER` improvisations undocumented—migrate scripts reviewed like auth code guarding money.

Practice migration adding column illustrative:

```
orders.country (nullable rollout strategy pondering expand-contract glimpses bridging zero-downtime awareness external cross-program references optionally)
```

Rehearse downgrade path—even if ephemeral dev DB nuking acceptable—you still author reversible intent expressing operational seriousness.

Interview prompts: articulate coordination with application dual-read/write phases when uncompromisable uptime constraints realistic—defer deep pattern mastery referencing external dedicated units if enriched elsewhere purposely.

Source: `08-persistence-sql-postgresql-and-sqlx/08-query-ownership-dashboard-reads.md`

Unit 8 — Query ownership readability: resisting `SELECT *` convenience rot

Articulate dashboards / analytics queries maintaining **explicit column projections** safeguarding forward schema evolution breakage detection earlier.

Practice constructing representative aggregate queries powering operational insights—`revenue` snapshot & `top orders` illustrative—even if seeded fixture data artificially small.

Enumerate observability interplay hooking hypothetical slow-query log inspection mindset bridging Ops program cross-links optionally.

Interview dimension: distinguishing **operational querying** burdens vs transactional hot path query tuning priorities balancing honestly.

Source: 08-persistence-sql-postgresql-and-sqlx/09-connection-pool-tuning-awareness.md

Unit 9 — Connection pool configuration awareness (database/sql tuning mental hooks)

Tune cognitively—even if qualitative:

```
SetMaxOpenConns
SetMaxIdleConns
ConnMaxLifetime
```

Relate bursts of concurrent handlers + goroutines to pool starvation vs Postgres resource exhaustion interplay narrative—not numeric perfection yet.

Interview expectation: verbally distinguish misblaming DB vs mistaken pool undersizing hypotheses methodically narrowing production incidents hypothetical walkthrough style.

Source: 08-persistence-sql-postgresql-and-sqlx/10-handler-service-repository-persistence-shape.md

Unit 10 — Layered persistence integration recap (transport → service → repository → DB)

Formalise repeatable chain aligning earlier HTTP layering Area 05 with matured persistence artefacts—prevent routing SQL leaking upward regressions creeping during deadline crunch fictions ethically acknowledged.

Conduct deliberate refactor extracting stray raw queries embedded historically—document mechanical steps + detected smell indicators future you recognises faster.

Interview consolidation mini drill: trace single `CreateOrder` request path enumerating failure surfaces per hop without peeking notes—muscle memory rehearsal target.

Source: 08-persistence-sql-postgresql-and-sqlx/11-capstone-order-system-integration.md

Unit 11 — order-system/ persistence capstone stitching failures intentionally

Mandatory lifecycle coverage sketch:

```
create order
pay order
cancel order
list orders (+ filtered variants optional)
```

Fault injection rehearsals (document recovery reasoning)

Craft scenarios:

```
transaction rollback correctness after nested failure invocation
locking race resurfacing interplay lessons unit6 repairs verification
purposefully malformed / conflicting migration practising detection + rollback choreography reverting safely
```

Interview checklist aggregator

transactions • repository delineation • migration strategy • locking consistency • rollback narratives • pools • sqlx explicit query ownership • interplay with architectural boundaries forward Area 11

Deliverable emphasises retrospective write-ups—**“why remediation structurally healed class of defect”**—not ephemeral hotfix mysticism anecdotes.

Phase — 09-testing-engineering-go-services

Directory: 09-testing-engineering-go-services/

Source: 09-testing-engineering-go-services/00-quick-reference.md

Quick Reference — Testing

Run commands

go test ./... # all tests go test -v -run TestFoo ./... # specific, verbose go test -race ./... # race detection go test -bench=. -benchmem # benchmarks go test -fuzz=FuzzFoo -fuzztime=30s

Table-driven template

```
cases := []struct{ name string; input X; want Y }{ ... } for _, tc := range cases { t.Run(tc.name, func(t *testing.T) { ... }) }
```

t.Parallel() — safe when

- test uses local vars only (no shared state)
- tc := tc // capture loop var (pre-Go 1.22)

gomock

```
ctrl := gomock.NewController(t) mock.EXPECT().Method(gomock.Any()).Return(val, nil).Times(1)
```

testcontainers

```
wait.ForListeningPort("5432/tcp") t.Cleanup(func() { container.Terminate(ctx) })
```

Source: 09-testing-engineering-go-services/01-table-driven-unit-tests.md

Unit 1 — Table-Driven Unit Tests

Concept

Table-driven tests define all test cases in a slice, loop over them, and run each as a named subtest with `t.Run`. One test function covers the happy path, edge cases, and error cases — no copy-pasted test functions that drift apart over time. When a case fails, the subtest name tells you immediately which scenario broke. Adding a new case means one new struct literal, not a new function.


```

package pricing_test

import (
    "errors"
    "testing"

    "github.com/example/app/pricing"
)

func TestCalculateDiscount(t *testing.T) {
    tests := []struct {
        name      string
        price      float64
        discountPct float64
        want      float64
        wantErr    error
    }{
        {
            name:      "10% off standard price",
            price:     100.0,
            discountPct: 10,
            want:      90.0,
        },
        {
            name:      "zero discount returns original price",
            price:     50.0,
            discountPct: 0,
            want:      50.0,
        },
        {
            name:      "100% discount returns zero",
            price:     75.0,
            discountPct: 100,
            want:      0.0,
        },
        {
            name:      "negative price is an error",
            price:     -10.0,
            discountPct: 5,
            wantErr:    pricing.ErrInvalidPrice,
        },
        {
            name:      "discount over 100 is an error",
            price:     100.0,
            discountPct: 110,
            wantErr:    pricing.ErrInvalidDiscount,
        },
    }

    for _, tc := range tests {
        t.Run(tc.name, func(t *testing.T) {
            got, err := pricing.CalculateDiscount(tc.price, tc.discountPct)
            if tc.wantErr != nil {
                if !errors.Is(err, tc.wantErr) {
                    t.Errorf("got err %v, want %v", err, tc.wantErr)
                }
                return
            }
            if err != nil {
                t.Fatalf("unexpected error: %v", err)
            }
            if got != tc.want {
                t.Errorf("got %v, want %v", got, tc.want)
            }
        })
    }
}

```

```
}  
}
```

Exercise

Build: Table-driven tests for a `Product` domain type with `IsAvailable() bool` and `NewProduct(name string, price float64, stock int) (*Product, error)`. **Input:** Test cases covering: valid product, price=0 (error), price negative (error), stock=-1 (error), empty name (error), stock=0 with `IsAvailable()=false`, stock=5 with `IsAvailable()=true`. **Output:** All cases run as named subtests. Zero copy-pasted test functions. **Acceptance:** Run `go test -v ./...` — subtest names are human-readable like `TestNewProduct/empty_name_is_error`. Introduce a deliberate bug in `NewProduct` (allow negative price) — exactly one subtest fails and its name identifies the case.

Interview

- Why does using `t.Run` for each case make failures easier to diagnose than multiple test functions?
- What is the advantage of putting the error cases in the same table as the success cases?
- A new developer wants to add a test for price=0.01 (valid boundary). How many lines do they write?

Source: `09-testing-engineering-go-services/02-integration-tests-and-testcontainers.md`

Unit 2 — Integration Tests and Testcontainers

Concept

Integration tests run against a real database. They catch things mocks cannot: constraint violations, query syntax errors, index behavior, and transaction semantics. Testcontainers starts a real Postgres container for the test, applies your migrations, runs the tests, and tears everything down. The test binary controls the container lifecycle — no external setup required, no shared state between runs.


```

//go:build integration

package repository_test

import (
    "context"
    "testing"

    "github.com/jmoirn/sqlx"
    _ "github.com/lib/pq"
    "github.com/testcontainers/testcontainers-go"
    "github.com/testcontainers/testcontainers-go/modules/postgres"
    "github.com/testcontainers/testcontainers-go/wait"

    "github.com/example/app/repository"
)

func TestOrderRepository_Create_Integration(t *testing.T) {
    ctx := context.Background()

    // Start a real Postgres container
    pgContainer, err := postgres.RunContainer(ctx,
        testcontainers.WithImage("postgres:16-alpine"),
        postgres.WithDatabase("testdb"),
        postgres.WithUsername("test"),
        postgres.WithPassword("test"),
        testcontainers.WithWaitStrategy(
            wait.ForLog("database system is ready to accept connections").
                WithOccurrence(2),
        ),
    )
    if err != nil {
        t.Fatalf("start container: %v", err)
    }
    t.Cleanup(func() { pgContainer.Terminate(ctx) })

    // Connect
    connStr, _ := pgContainer.ConnectionString(ctx, "sslmode=disable")
    db, err := sqlx.Connect("postgres", connStr)
    if err != nil {
        t.Fatalf("connect: %v", err)
    }

    // Apply migration
    _, err = db.Exec(`
        CREATE TABLE IF NOT EXISTS orders (
            id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
            user_id UUID NOT NULL,
            status TEXT NOT NULL DEFAULT 'pending',
            created_at TIMESTAMPTZ NOT NULL DEFAULT now()
        )
    `)
    if err != nil {
        t.Fatalf("migrate: %v", err)
    }

    repo := repository.NewPostgresOrderRepo(db)

    // Test: create order, find it by ID
    userID := "550e8400-e29b-41d4-a716-446655440000"
    order, err := repo.Create(ctx, userID)
    if err != nil {
        t.Fatalf("create: %v", err)
    }
    if order.ID == "" {
        t.Fatal("expected non-empty ID")
    }
}

```

```
}

found, err := repo.FindByID(ctx, order.ID)
if err != nil {
    t.Fatalf("find: %v", err)
}
if found.UserID != userID {
    t.Errorf("got user_id %s, want %s", found.UserID, userID)
}
}
```

Exercise

Build: An integration test for `OrderRepository.Create` using Testcontainers. **Input:** A test with `//go:build integration` tag. Use a real Postgres container with your schema from Unit 2. **Output:** Test creates an order, retrieves it by ID, verifies all fields match (`user_id`, `status="pending"`, `created_at` is recent). **Acceptance:** Run `go test -tags=integration -v ./...` — test passes. Run without the tag — test is not compiled and skipped. Run with `-count=3` — all three runs pass (no shared state leaking between runs).

Interview

- What does Testcontainers give you that an in-memory SQLite database cannot?
- Why use `//go:build integration` instead of always running these tests?
- A colleague says “just mock the DB in integration tests.” What is wrong with that?

Source: `09-testing-engineering-go-services/03-gomock-and-interface-seams.md`

Unit 3 — gomock and Interface Seams

Concept

Mock the interface boundary to test the service layer in isolation from the database. Generate mocks with `mockgen` — never write them by hand. Set expectations with `EXPECT()` to declare what calls should happen, how many times, and what they return. If the service makes an unexpected call, the test fails immediately with a clear message. This catches logic errors: calling the repo when you should have used a cache, or calling it twice when once is correct.

Code

```
//go:generate mockgen -source=order_repository.go -destination=mocks/mock_order_repository.go

package service_test

import (
    "context"
    "errors"
    "testing"

    "github.com/golang/mock/gomock"

    "github.com/example/app/domain"
    "github.com/example/app/service"
    mock_service "github.com/example/app/service/mocks"
)

func TestOrderService_GetOrder(t *testing.T) {
    ctrl := gomock.NewController(t)
    // ctrl.Finish() is called automatically in Go 1.14+ via t.Cleanup

    mockRepo := mock_service.NewMockOrderRepository(ctrl)
    svc := service.NewOrderService(mockRepo)
    ctx := context.Background()

    t.Run("order found", func(t *testing.T) {
        want := &domain.Order{ID: "ord_1", UserID: "usr_1", Status: "pending"}

        // Expect exactly one call with this ID, return the order
        mockRepo.EXPECT().
            FindByID(ctx, "ord_1").
            Return(want, nil).
            Times(1)

        got, err := svc.GetOrder(ctx, "ord_1")
        if err != nil {
            t.Fatalf("unexpected error: %v", err)
        }
        if got.ID != want.ID {
            t.Errorf("got %s, want %s", got.ID, want.ID)
        }
    })

    t.Run("order not found", func(t *testing.T) {
        mockRepo.EXPECT().
            FindByID(ctx, "ord_missing").
            Return(nil, service.ErrNotFound).
            Times(1)

        _, err := svc.GetOrder(ctx, "ord_missing")
        if !errors.Is(err, service.ErrNotFound) {
            t.Errorf("got %v, want ErrNotFound", err)
        }
    })
}
```

Exercise

Build: Tests for `UserService.GetUser` that has both a cache and a repository. **Input:** `UserService` holds a `UserRepository` (mock) and a `UserCache` (mock). `GetUser` checks cache first, falls through to repo on miss, then stores in cache. **Output:** Two test cases: (1) cache hit — repo must NOT be called (use `Times(0)`), (2) cache miss — repo IS called exactly once, result is stored in cache (`cache.Set` is called once). **Acceptance:** Both tests pass. Remove the cache check from `GetUser` — the cache hit test fails because the repo is called unexpectedly. This proves the mock verified the interaction.

Interview

- What does `Times(1)` on a mock expectation verify?
 - Why generate mocks from interfaces rather than writing them by hand?
 - When should you use `AnyTimes()` instead of `Times(1)`, and what risk does it carry?
-

Source: `09-testing-engineering-go-services/04-benchmarks-load-and-fuzz-hooks.md`

Unit 4 — Benchmarks, Load Tests, and Fuzz Hooks

Concept

Benchmarks measure performance using `BenchmarkX(b *testing.B)` — the test runner calls your function `b.N` times and adjusts N until timing stabilizes. Use `-benchmem` to show allocations per operation, which is often more important than raw speed. Fuzz tests (`FuzzX`) generate random inputs to find panics and crashes in parsing and decoding code. Both are standard library features — no external tools needed.


```

package serialization_test

import (
    "bytes"
    "encoding/json"
    "strings"
    "testing"

    "github.com/example/app/domain"
)

var order = domain.Order{
    ID:      "ord_123",
    UserID:  "usr_456",
    Status:  "pending",
}

// BenchmarkJSONMarshal measures allocation cost of json.Marshal.
func BenchmarkJSONMarshal(b *testing.B) {
    b.ReportAllocs()
    for i := 0; i < b.N; i++ {
        _, _ = json.Marshal(order)
    }
}

// BenchmarkJSONMarshalWithBuffer reuses a buffer to reduce allocations.
func BenchmarkJSONMarshalWithBuffer(b *testing.B) {
    b.ReportAllocs()
    var buf bytes.Buffer
    enc := json.NewEncoder(&buf)
    for i := 0; i < b.N; i++ {
        buf.Reset()
        _ = enc.Encode(order)
    }
}

// BenchmarkStringConcat vs Builder – classic allocation demo.
func BenchmarkStringConcat(b *testing.B) {
    b.ReportAllocs()
    for i := 0; i < b.N; i++ {
        s := ""
        for j := 0; j < 10; j++ {
            s += "word "
        }
        _ = s
    }
}

func BenchmarkStringBuilder(b *testing.B) {
    b.ReportAllocs()
    for i := 0; i < b.N; i++ {
        var sb strings.Builder
        for j := 0; j < 10; j++ {
            sb.WriteString("word ")
        }
        _ = sb.String()
    }
}

// FuzzParseOrderID finds crashes in order ID parsing.
func FuzzParseOrderID(f *testing.F) {
    // Seed corpus
    f.Add("ord_123")
    f.Add("")
    f.Add("ord_" + strings.Repeat("x", 1000))
}

```

```
f.Fuzz(func(t *testing.T, input string) {
    // Must not panic for any input
    _, _ = parseOrderID(input)
})
}
```

Exercise

Build: Benchmarks for JSON marshal/unmarshal of an Order struct with 6 fields. **Input:** Benchmark 1: `json.Marshal` with no buffer reuse. Benchmark 2: `json.NewEncoder` with a `bytes.Buffer` reset between iterations. **Output:** Run `go test -bench=. -benchmem -benchtime=3s ./...`. Show output with `ns/op`, `B/op`, `allocs/op` for both benchmarks. **Acceptance:** The buffer-reuse benchmark shows fewer `allocs/op` than the plain marshal benchmark. Document the difference in a comment. Run with `-benchtime=10x` (10 iterations fixed) to understand the flags.

Interview

- What does `allocs/op` tell you that `ns/op` does not?
 - Why does resetting a `bytes.Buffer` reduce allocations compared to calling `json.Marshal` directly?
 - When would you use a fuzz test over a table-driven test?
-

Source: `09-testing-engineering-go-services/05-capstone-coverage-balanced-suite.md`

Unit 5 — Capstone: Coverage and a Balanced Test Suite

Concept

A balanced test suite has unit tests (fast, many, mock dependencies) and integration tests (slow, few, real DB). They do not duplicate each other — unit tests verify logic, integration tests verify DB behavior. Coverage percentage is a proxy metric, not a goal. 70% coverage of real business behavior is more valuable than 100% coverage of trivial getters. High coverage with weak assertions is worse than lower coverage with strong ones.

Code

```
# Makefile

.PHONY: test-unit test-integration test-all coverage

# Fast — runs on every save
test-unit:
    go test -race -count=1 ./...

# Slow — runs in CI, requires DB
test-integration:
    go test -tags=integration -race -count=1 -timeout=120s ./...

# Both
test-all: test-unit test-integration

# Coverage report — unit tests only
coverage:
    go test -coverprofile=coverage.out ./...
    go tool cover -html=coverage.out -o coverage.html
    go tool cover -func=coverage.out | grep total
```

```
# Target test distribution for the order system:
#
# Unit tests (service layer, mocked deps):      ~15 tests, run in <1s
#   - OrderService.PlaceOrder: invalid user, empty items, success
#   - PricingService.Calculate: discount bounds, zero price
#   - StatusMachine.Transition: valid transitions, invalid transitions
#
# Integration tests (repository, real DB):      ~5 tests, run in ~10s
#   - OrderRepo.Create: round-trip, fields preserved
#   - OrderRepo.FindByID: not found returns ErrNotFound
#   - OrderRepo.PlaceOrder: transaction rollback on item failure
#
# Benchmarks:                                  ~2, run on demand
#   - BenchmarkOrderRepo_FindByID
#   - BenchmarkJSONMarshal_OrderResponse
```

Exercise

Build: A complete test suite for your order system across three categories. **Input:** The order system from modules 8-9. **Output:** (1) 3 unit tests for service logic using mocked repositories. (2) 2 integration tests for repository against a real DB via Testcontainers. (3) 1 benchmark for the most-called query (`FindByID`). **Acceptance:** `make test-unit` passes in under 2 seconds. `make test-integration` passes with Testcontainers running. `make coverage` shows at least 60% coverage of the service package. The integration tests are excluded from `make test-unit` (confirm via `-tags` absence).

Interview

- What is the difference between a unit test and an integration test in your order system?
 - Why would 100% unit test coverage with all dependencies mocked give you false confidence?
 - A new developer asks “should I write a unit test or integration test for this query?” How do you answer?
-

Phase — 10-testing-advanced-contracts-integration-and-mocks

Directory: 10-testing-advanced-contracts-integration-and-mocks/

Source: 10-testing-advanced-contracts-integration-and-mocks/00-quick-reference.md

Quick Reference — Advanced Testing

Subtest parallel capture (pre-1.22)

```
for _, tc := range cases { tc := tc // REQUIRED: loop var capture t.Run(tc.name, func(t *testing.T) { t.Parallel() }) }
```

t.Helper() — always call in assertion helpers

```
func assertEquals(t *testing.T, got, want int) { t.Helper() if got != want { t.Errorf(...) } }
```

Mock vs stub distinction

// Stub: returns canned data, no expectations // Mock: asserts interactions (gomock EXPECT) // Use mocks sparingly — integration tests > mocks

Contract testing

// Consumer defines contract (expected request/response shape) // Provider verifies it on their side // Tooling: pact-go

Flake prevention

// Never sleep in tests — use wait strategies // Isolate state: each test gets its own DB schema or transaction // Use t.Cleanup for guaranteed teardown

Source: 10-testing-advanced-contracts-integration-and-mocks/01-table-driven-tests-and-subtest-discipline.md

Unit 1 — Table-Driven Tests and Subtest Discipline

Concept

`t.Parallel()` on a subtest runs it concurrently with other parallel subtests — this speeds up slow test suites and exposes data races. `t.Helper()` marks a function as a test helper so that when it calls `t.Errorf`, the failure points to the caller's line, not the helper's line. In Go versions before 1.22, loop variables are shared across iterations — always capture `tc := tc` before calling `t.Run` with `t.Parallel()` to avoid all subtests running with the last loop value.

Code

```
package validation_test

import (
    "testing"

    "github.com/example/app/validation"
)

// assertNoError is a helper — t.Helper() makes failures point to the caller.
func assertNoError(t *testing.T, err error) {
    t.Helper()
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
}

func assertError(t *testing.T, err error, want error) {
    t.Helper()
    if err == nil {
        t.Fatal("expected error, got nil")
    }
}

func TestValidateEmail(t *testing.T) {
    tests := []struct {
        name    string
        input    string
        wantErr bool
    }{
        {name: "valid email", input: "user@example.com", wantErr: false},
        {name: "missing @", input: "userexample.com", wantErr: true},
        {name: "empty string", input: "", wantErr: true},
        {name: "unicode local part", input: "用🔍@example.com", wantErr: false},
        {name: "very long email", input: string(make([]byte, 300)) + "@x.com", wantErr: true},
    }

    for _, tc := range tests {
        tc := tc // capture loop variable — required before Go 1.22
        t.Run(tc.name, func(t *testing.T) {
            t.Parallel() // run subtests concurrently

            err := validation.ValidateEmail(tc.input)
            if tc.wantErr {
                assertError(t, err, nil)
            } else {
                assertNoError(t, err)
            }
        })
    }
}
```

Exercise

Build: Convert 3 of your existing sequential tests to parallel subtests with `t.Helper()` assertion functions. **Input:** Three existing test functions from your order system (pick ones that make sense to parallelize — no shared mutable state). **Output:** Each uses `t.Parallel()`. Each uses a helper function with `t.Helper()`. Loop variable is captured before `t.Parallel()`. **Acceptance:** Run `go test -race ./...` — zero data races reported. Remove `tc := tc` from one test — run `go test -race` again — observe the race or the wrong case running. Re-add it.

Interview

- Why must you capture `tc := tc` before calling `t.Parallel()` in a table-driven test in Go 1.21?
 - What is the visible difference in test output between a helper with `t.Helper()` and one without?
 - When is `t.Parallel()` a bad idea on a subtest?
-

Source: `10-testing-advanced-contracts-integration-and-mocks/02-contract-testing-producer-consumer-safety.md`

Unit 2 — Contract Testing: Producer-Consumer Safety

Concept

Contract testing verifies that a consumer's expectations of a provider are actually met. The consumer writes down what requests it sends and what response shape it expects. The provider runs tests against those expectations. This prevents breaking changes — renaming a JSON field breaks the consumer contract test immediately, not when services are deployed to staging and someone notices. It is lighter than full end-to-end tests and faster to run in CI.

Code

```
package contract_test

import (
    "encoding/json"
    "net/http"
    "net/http/httptest"
    "testing"

    "github.com/example/app/handler"
)

// Consumer's expectation: GET /products/:id returns {id, name, price}
// If the producer renames "name" to "product_name", this test fails.

type productConsumerExpectation struct {
    ID      string `json:"id"`
    Name    string `json:"name"` // consumer expects this field name
    Price   float64 `json:"price"`
}

func TestProductHandler_ContractGet(t *testing.T) {
    h := handler.NewProductHandler(stubProductRepo{})
    srv := httptest.NewServer(h)
    defer srv.Close()

    resp, err := http.Get(srv.URL + "/products/prod_1")
    if err != nil {
        t.Fatalf("request failed: %v", err)
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        t.Fatalf("got status %d, want 200", resp.StatusCode)
    }

    var got productConsumerExpectation
    if err := json.NewDecoder(resp.Body).Decode(&got); err != nil {
        t.Fatalf("decode: %v", err)
    }

    // Contract assertions – shape, not implementation details
    if got.ID == "" {
        t.Error("contract violation: 'id' field is empty or missing")
    }
    if got.Name == "" {
        t.Error("contract violation: 'name' field is empty or missing")
    }
    if got.Price <= 0 {
        t.Error("contract violation: 'price' must be positive")
    }
}

// stubProductRepo satisfies the handler's repository interface for contract tests.
type stubProductRepo struct{}

func (s stubProductRepo) FindByID(_ string) (*domain.Product, error) {
    return &domain.Product{ID: "prod_1", Name: "Widget", Price: 9.99}, nil
}
```

Exercise

Build: A contract test for `ProductService.GetProduct` that runs against the real HTTP handler. **Input:** The handler serves `GET /products/:id` and returns `{id, name, price}`. **Output:** A contract test struct with exactly the fields the consumer expects. The test fetches from a `httptest.Server` and asserts each field is present and valid. **Acceptance:** Test passes. Now rename `name` to `product_name` in the handler's response struct — the contract test fails with a clear message identifying the broken field. Revert and confirm it passes again.

Interview

- What does a contract test verify that a unit test of the handler cannot?
- Why does the consumer define the contract, not the producer?
- Contract tests and integration tests both test real behavior. What is the difference?

Source: `10-testing-advanced-contracts-integration-and-mocks/03-integration-tests-and-database-semantics.md`

Unit 3 — Integration Tests and Database Semantics

Concept

Integration tests should test actual database behavior — constraint violations, NULL handling, unique index enforcement, and concurrent writes. Do not mock the DB in integration tests; that defeats the purpose. Use a transaction per test and roll it back at the end to keep tests isolated without truncating tables between every case. The rollback approach is fast and leaves the schema intact for inspection if a test fails mid-run.

Code

```
//go:build integration

package repository_test

import (
    "context"
    "testing"

    "github.com/jmoiern/sqlx"
    "github.com/example/app/repository"
)

// newTestTx starts a transaction and registers rollback on test cleanup.
// Each test gets an isolated view of the DB.
func newTestTx(t *testing.T, db *sqlx.DB) *sqlx.Tx {
    t.Helper()
    tx, err := db.Beginx()
    if err != nil {
        t.Fatalf("begin tx: %v", err)
    }
    t.Cleanup(func() { tx.Rollback() }) // always rolls back — isolates the test
    return tx
}

func TestUserRepo_DuplicateEmail(t *testing.T) {
    tx := newTestTx(t, testDB) // testDB set up in TestMain

    repo := repository.NewUserRepoWithTx(tx)

    _, err := repo.Create(context.Background(), "alice@example.com")
    if err != nil {
        t.Fatalf("first insert: %v", err)
    }

    _, err = repo.Create(context.Background(), "alice@example.com")
    if err == nil {
        t.Fatal("expected error on duplicate email, got nil")
    }
    // Verify the error is a unique constraint violation, not a generic error
    if !repository.IsUniqueViolation(err) {
        t.Errorf("got error type %T, want unique violation", err)
    }
}

func TestOrderRepo_InvalidUserIDFailsForeignKey(t *testing.T) {
    tx := newTestTx(t, testDB)
    repo := repository.NewOrderRepoWithTx(tx)

    _, err := repo.Create(context.Background(), "00000000-0000-0000-0000-000000000000")
    if err == nil {
        t.Fatal("expected FK violation, got nil")
    }
    if !repository.IsForeignKeyViolation(err) {
        t.Errorf("got error %v, want FK violation", err)
    }
}
```

Exercise

Build: Three integration tests using the transaction-per-test isolation pattern. **Input:** A real Postgres DB with your schema from Module 8, Unit 2. **Output:** Test 1: inserting duplicate email returns a unique constraint violation. Test 2: inserting an order with a non-existent user_id fails with a foreign key violation. Test 3: two goroutines concurrently decrement the same product's stock —

verify the final stock is consistent (no negative value, no lost update). **Acceptance:** All three tests use `t.Cleanup` rollback. After each test, the DB is in the same state as before. Test 3 requires a `SELECT ... FOR UPDATE` or a `CHECK (stock >= 0)` constraint to be present.

Interview

- Why roll back in a test cleanup instead of truncating tables?
 - What behavior would you miss if you mocked the DB in these three tests?
 - How do you test that a concurrent write produces a consistent result?
-

Source: `10-testing-advanced-contracts-integration-and-mocks/04-testcontainers-realism-and-flake-hygiene.md`

Unit 4 — Testcontainers Realism and Flake Hygiene

Concept

Flaky tests are false negatives — they undermine trust in the entire suite. The most common causes in container-based tests are: using `time.Sleep` to wait for the container to be ready (wrong — the wait time varies), sharing state between test cases, and non-deterministic ordering. Fix flakiness with proper wait strategies that check for specific log output or port readiness, and use `TestMain` for a single shared container across the test package.

Code

```
//go:build integration

package repository_test

import (
    "context"
    "os"
    "testing"

    "github.com/jmoiern/sqlx"
    _ "github.com/lib/pq"
    "github.com/testcontainers/testcontainers-go"
    "github.com/testcontainers/testcontainers-go/modules/postgres"
    "github.com/testcontainers/testcontainers-go/wait"
)

var testDB *sqlx.DB

// TestMain starts one container shared across all tests in this package.
// This is faster than starting a container per test.
func TestMain(m *testing.M) {
    ctx := context.Background()

    // WRONG: time.Sleep(2 * time.Second) – fragile on slow CI
    // RIGHT: wait strategy based on actual readiness signal

    pgContainer, err := postgres.RunContainer(ctx,
        testcontainers.WithImage("postgres:16-alpine"),
        postgres.WithDatabase("testdb"),
        postgres.WithUsername("test"),
        postgres.WithPassword("test"),
        // Wait until Postgres logs "ready to accept connections" twice
        // (once for template DB, once for the actual DB)
        testcontainers.WithWaitStrategy(
            wait.ForLog("database system is ready to accept connections").
                WithOccurrence(2),
        ),
    )
    if err != nil {
        panic("start container: " + err.Error())
    }
    defer pgContainer.Terminate(ctx)

    connStr, _ := pgContainer.ConnectionString(ctx, "sslmode=disable")
    testDB, err = sqlx.Connect("postgres", connStr)
    if err != nil {
        panic("connect: " + err.Error())
    }

    // Apply schema once for the whole package
    if _, err := testDB.Exec(schemaSQL); err != nil {
        panic("migrate: " + err.Error())
    }

    // Run all tests, capture exit code
    code := m.Run()

    testDB.Close()
    os.Exit(code)
}
```

Exercise

Build: A `TestMain` setup with a shared Postgres container using proper wait strategies. **Input:** Your existing integration test file from Unit 2 of this module (which may have a per-test container setup). **Output:** Refactor to a single container in `TestMain`. Replace any `time.Sleep` with `wait.ForLog`. Each test uses `newTestTx` for isolation (from Unit 3). **Acceptance:** Run `go test -tags=integration -count=10 -v ./...` — all 10 runs pass. The total run time should be under 30 seconds for 5 integration tests. Add `fmt.Println("sleeping") + time.Sleep(3*time.Second)` before `ConnectString` — observe it breaks. Remove it.

Interview

- Why is `wait.ForLog` more reliable than `time.Sleep` for container readiness?
 - Why use a single container in `TestMain` instead of one container per test?
 - A test passes locally but fails 20% of the time in CI. What are the three most likely causes?
-

Source: `10-testing-advanced-contracts-integration-and-mocks/05-mock-ownership-and-test-doubles-discipline.md`

Unit 5 — Mock Ownership and Test Doubles Discipline

Concept

Mock only what you own — interfaces you defined in your own packages. Never mock standard library types or third-party client structs directly. Wrapping a third-party client in your own interface gives you a seam you control. A stub returns fixed data but verifies nothing. A mock verifies interactions. Use mocks sparingly — overuse creates tests that mirror implementation details, which break on every refactor even when behavior is correct.

Code

```
// payment/gateway.go – your interface wrapping the third-party client
package payment

import "context"

// PaymentGateway is the interface you own.
// The third-party SDK is hidden behind it.
type PaymentGateway interface {
    Charge(ctx context.Context, amount float64, token string) (string, error)
    Refund(ctx context.Context, chargeID string) error
}

// stripeGateway implements PaymentGateway using the Stripe SDK.
// This is the only place the third-party package is imported.
type stripeGateway struct {
    client *stripe.Client // third-party – not mockable directly
}

func NewStripeGateway(apiKey string) PaymentGateway {
    return &stripeGateway{client: stripe.New(apiKey)}
}

func (g *stripeGateway) Charge(ctx context.Context, amount float64, token string) (string, error) {
    // ... calls stripe SDK
}

// ---
// In tests, mock PaymentGateway, not *stripe.Client

//go:generate mockgen -source=gateway.go -destination=mocks/mock_gateway.go

package service_test

import (
    "context"
    "testing"

    "github.com/golang/mock/gomock"
    mock_payment "github.com/example/app/payment/mocks"
    "github.com/example/app/service"
)

func TestOrderService_PlaceOrder_ChargesPayment(t *testing.T) {
    ctrl := gomock.NewController(t)
    mockGateway := mock_payment.NewMockPaymentGateway(ctrl)

    // The service should call Charge exactly once with the correct amount
    mockGateway.EXPECT().
        Charge(gomock.Any(), 99.99, "tok_test").
        Return("ch_123", nil).
        Times(1)

    svc := service.NewOrderService(mockGateway)
    _, err := svc.PlaceOrder(context.Background(), "tok_test", 99.99)
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
}
```

Exercise

Build: A `PaymentGateway` interface wrapping a fictional third-party client, with a mock-based service test. **Input:** A `CheckoutService.Checkout(ctx, cartID, paymentToken)` that calls `PaymentGateway.Charge`. The gateway interface has `Charge` and `Refund`. **Output:** (1) The `PaymentGateway` interface in the `payment` package. (2) A generated mock. (3) Two service tests: successful checkout (Charge called once), failed charge (Charge returns error, service returns error, Refund is NOT called). **Acceptance:** Mocks are generated with `go generate`. Service tests pass. The `stripeGateway` struct satisfies `PaymentGateway` — verified in an integration test file with `var _ payment.PaymentGateway = (*payment.stripeGateway)(nil)`.

Interview

- Why wrap a third-party client in your own interface instead of using the third-party's interface directly?
- What is the difference between a stub and a mock in concrete terms?
- A test suite has 200 mocked function calls spread across 50 test files. A developer refactors the service — 40 tests break. What is the root cause?

Source: `10-testing-advanced-contracts-integration-and-mocks/06-capstone-repository-integration-harness.md`

Unit 6 — Capstone: repository integration harness (confidence over line count)

Build a thin but real integration harness that proves `sqlx/Postgres-backed repositories` behave with real DB semantics—not only mocks.

Recommended checks (adapt to scope):

```
UNIQUE constraint violation maps to typed domain errors (no string contains hacks)
explicit transaction rollback on forced failure smoke path
migration apply on blank DB repeatable (or documented alternative fixture strategy)
performance sanity optional micro-benchmark not mandatory core here—confidence first
```

Write a half-page rationale: **why these tests earned CI time.**

Phase — 11-caching-redis-and-invalidations

Directory: 11-caching-redis-and-invalidations/

Source: 11-caching-redis-and-invalidations/00-quick-reference.md

Quick Reference — Redis & Caching

go-redis v9 basics

```
rdb := redis.NewClient(&redis.Options{Addr: "localhost:6379"}) rdb.Set(ctx, key, value, ttl) val, err := rdb.Get(ctx, key).Result()
errors.Is(err, redis.Nil) // cache miss check
```

Common commands

```
rdb.Del(ctx, key) rdb.Exists(ctx, key) rdb.Expire(ctx, key, ttl) rdb.Incr(ctx, key) rdb.HSet(ctx, key, field, value) rdb.HGet(ctx, key,
field)
```

Pipeline (batch, 1 RTT)

```
pipe := rdb.Pipeline() pipe.Set(ctx, "a", 1, time.Minute) cmds, _ := pipe.Exec(ctx)
```

Cache-aside flow

1. GET from cache → hit: return
2. miss: load from DB
3. SET in cache with TTL
4. return

Stampede prevention

singleflight.Group — collapses concurrent misses to 1 DB call

TTL discipline

- User sessions: 30min–24h
 - Reference data: 5–15min
 - Rate limit counters: 1min exact
 - Never: no TTL on mutable data
-

Source: 11-caching-redis-and-invalidations/01-redis-cache-strategies-enterprise-mindset.md

Unit 1 — Redis Cache Strategies

Concept

Cache-aside is the most common pattern: check the cache, on a miss load from DB, then populate the cache. The application owns the cache interaction. Write-through writes to both cache and DB on every update — cache is never stale but writes are slower. Cache-only is rare and only appropriate when data loss is acceptable. TTLs are mandatory on every key — caching without expiry creates stale data that lives forever and corrupts production reads.

Code

```
package service

import (
    "context"
    "encoding/json"
    "errors"
    "fmt"
    "time"

    "github.com/redis/go-redis/v9"
    "github.com/example/app/domain"
)

const productTTL = 5 * time.Minute

type ProductService struct {
    repo ProductRepository
    cache *redis.Client
}

// Cache-aside: check cache → miss → load DB → populate cache
func (s *ProductService) GetProduct(ctx context.Context, id string) (*domain.Product, error) {
    key := fmt.Sprintf("product:%s", id)

    // 1. Check cache
    data, err := s.cache.Get(ctx, key).Bytes()
    if err == nil {
        var p domain.Product
        if err := json.Unmarshal(data, &p); err == nil {
            return &p, nil // cache hit
        }
    }
    if !errors.Is(err, redis.Nil) {
        // Redis error – log and fall through to DB (degraded mode)
        fmt.Printf("cache get error: %v\n", err)
    }

    // 2. Cache miss – load from DB
    product, err := s.repo.FindByID(ctx, id)
    if err != nil {
        return nil, err
    }

    // 3. Populate cache – always with TTL
    if data, err := json.Marshal(product); err == nil {
        s.cache.Set(ctx, key, data, productTTL) // best-effort, ignore error
    }

    return product, nil
}

// Write-through: update DB and cache together
func (s *ProductService) UpdateProduct(ctx context.Context, p *domain.Product) error {
    if err := s.repo.Update(ctx, p); err != nil {
        return err
    }
    key := fmt.Sprintf("product:%s", p.ID)
    data, _ := json.Marshal(p)
    return s.cache.Set(ctx, key, data, productTTL).Err()
}
```


Exercise

Build: Cache-aside for `ProductService.GetProduct` using a mock repository and a mock Redis client. **Input:** Call `GetProduct` twice with the same product ID. **Output:** First call: repo `FindByID` is called once, result stored in cache. Second call: repo `FindByID` is NOT called (cache hit). **Acceptance:** Use a counter on the mock repo to verify call count. First call count=1, second call count=1 (unchanged). Add a test where Redis returns an error — verify the service falls through to the DB and does not return an error to the caller.

Interview

- What is the difference between cache-aside and write-through in terms of who controls cache population?
 - A cache key has no TTL. What happens over time in production?
 - Why is “fall through to DB on Redis error” the correct behavior instead of returning an error?
-

Source: `11-caching-redis-and-invalidations/02-go-redis-client-patterns.md`

Unit 2 — go-redis Client Patterns

Concept

`go-redis/v9` is the standard Redis client for Go. Always configure pool size and connection timeouts explicitly — the defaults are too permissive for production. `redis.Nil` is returned from `Get` when a key does not exist — it is not an error, it is a miss, and you must handle it separately from actual errors. Pipelines batch multiple commands into one round trip, which matters when you are setting 10 keys at once.

Code

```
package cache

import (
    "context"
    "errors"
    "fmt"
    "time"

    "github.com/redis/go-redis/v9"
)

func NewClient(addr string) *redis.Client {
    return redis.NewClient(&redis.Options{
        Addr:      addr,
        PoolSize:  20,           // max connections in pool
        MinIdleConns: 5,        // keep warm
        DialTimeout: 2 * time.Second,
        ReadTimeout: 1 * time.Second,
        WriteTimeout: 1 * time.Second,
    })
}

// Get returns the value, ErrMiss if key does not exist, or an error.
var ErrMiss = errors.New("cache miss")

func Get(ctx context.Context, rdb *redis.Client, key string) ([]byte, error) {
    val, err := rdb.Get(ctx, key).Bytes()
    if errors.Is(err, redis.Nil) {
        return nil, ErrMiss // not an error – key does not exist
    }
    return val, err
}

// Set stores a value with a mandatory TTL.
func Set(ctx context.Context, rdb *redis.Client, key string, val []byte, ttl time.Duration) error {
    if ttl <= 0 {
        return errors.New("ttl must be positive – never cache without expiry")
    }
    return rdb.Set(ctx, key, val, ttl).Err()
}

// Pipeline: set multiple keys in one round trip.
func BulkSet(ctx context.Context, rdb *redis.Client, pairs map[string][]byte, ttl time.Duration) error {
    pipe := rdb.Pipeline()
    for k, v := range pairs {
        pipe.Set(ctx, k, v, ttl)
    }
    cmds, err := pipe.Exec(ctx)
    if err != nil {
        return fmt.Errorf("pipeline exec: %w", err)
    }
    for _, cmd := range cmds {
        if cmd.Err() != nil {
            return fmt.Errorf("pipeline cmd %s: %w", cmd.Name(), cmd.Err())
        }
    }
    return nil
}
```

Exercise

Build: A Redis-backed rate limiter: `Allow(ctx, userID string) (bool, error)`. **Input:** Each call increments a counter for the user. Limit: 5 requests per 10 seconds. On the first increment, set a 10-second expiry. **Output:** `Allow` returns `true` if under limit, `false` if over. Uses `INCR` and `EXPIRE` commands. The `EXPIRE` is only set when the key is new (`count == 1`) to avoid resetting the window on every call. **Acceptance:** Call `Allow` 6 times for the same user. First 5 return true. Sixth returns false. Wait for TTL to expire (or manually delete the key), call again — returns true. Test with `redis.NewClient` pointing at a local Redis instance.

Interview

- What does `redis.Nil` mean, and why do you handle it separately from other errors?
 - Why set `PoolSize` explicitly instead of using the default?
 - Why must `EXPIRE` only be called when the counter is 1 (new key) in a rate limiter?
-

Source: `11-caching-redis-and-invalidations/03-cache-aside-pattern-implementation.md`

Unit 3 — Cache-Aside Pattern Implementation

Concept

Cache-aside means the application is responsible for both reading and writing the cache. On a cache miss, the application loads from the DB and then writes to the cache. The cache is never written to on updates — only invalidated. This is the right pattern when reads vastly outnumber writes and you can tolerate brief staleness. Cache degradation is required: if Redis is unavailable, the service must continue operating against the DB, with a warning logged.


```

package service

import (
    "context"
    "encoding/json"
    "errors"
    "fmt"
    "log/slog"
    "time"

    "github.com/redis/go-redis/v9"
    "github.com/example/app/domain"
)

type ProductService struct {
    repo ProductRepository
    cache *redis.Client
    log    *slog.Logger
}

func (s *ProductService) GetProduct(ctx context.Context, id string) (*domain.Product, error) {
    key := productKey(id)

    // Try cache first
    data, err := s.cache.Get(ctx, key).Bytes()
    switch {
    case err == nil:
        // Cache hit – deserialize and return
        var p domain.Product
        if jsonErr := json.Unmarshal(data, &p); jsonErr != nil {
            s.log.WarnContext(ctx, "cache unmarshal failed", "key", key, "err", jsonErr)
            // Fall through to DB on corrupt cache data
        } else {
            return &p, nil
        }

    case errors.Is(err, redis.Nil):
        // Expected miss – load from DB

    default:
        // Redis is unhealthy – log warning, fall through to DB
        s.log.WarnContext(ctx, "redis unavailable, falling back to DB",
            "err", err, "key", key)
    }

    // Load from DB
    product, err := s.repo.FindByID(ctx, id)
    if err != nil {
        return nil, err
    }

    // Populate cache – best-effort, never fail the request on cache write error
    if data, marshalErr := json.Marshal(product); marshalErr == nil {
        if setErr := s.cache.Set(ctx, key, data, 5*time.Minute).Err(); setErr != nil {
            s.log.WarnContext(ctx, "cache set failed", "key", key, "err", setErr)
        }
    }

    return product, nil
}

func productKey(id string) string {
    return fmt.Sprintf("product:v1:%s", id)
}

```

Exercise

Build: The `ProductService.GetProduct` method with full cache degradation handling. **Input:** Three scenarios to test: (1) Redis returns the value (cache hit), (2) Redis returns `redis.Nil` (miss, load DB), (3) Redis returns a connection error (degraded, load DB, log warning). **Output:** All three scenarios return a product without error. Scenario 3 logs a warning. In scenarios 2 and 3, the repo is called exactly once. **Acceptance:** Stop Redis locally, run your service — it still returns products from the DB. Restart Redis — it caches again on the next request. Verify by inspecting `KEYS product:*` in `redis-cli` after the second call.

Interview

- What should happen to the request when Redis is completely unavailable?
 - Why use a versioned key prefix like `product:v1:` instead of just `product:?`
 - A cache write fails silently. Is that correct behavior? When would it not be?
-

Source: `11-caching-redis-and-invalidations/04-invalidation-strategies-and-ttl-discipline.md`

Unit 4 — Invalidation Strategies and TTL Discipline

Concept

Delete-on-write is the simplest correct invalidation: when you update an entity, delete its cache key. The next read repopulates from the DB. Never update the cache on write — that creates race conditions between writers. Stampede prevention solves a different problem: when a popular key expires, many goroutines miss simultaneously and all try to load from the DB.

`singleflight.Group` ensures only one goroutine does the DB fetch while the rest wait and reuse the result.

Code

```
package service

import (
    "context"
    "encoding/json"
    "fmt"
    "time"

    "golang.org/x/sync/singleflight"
    "github.com/redis/go-redis/v9"
    "github.com/example/app/domain"
)

type UserService struct {
    repo UserRepository
    cache *redis.Client
    sf     singleflight.Group
}

// UpdateUser updates the DB and deletes the cache key.
func (s *UserService) UpdateUser(ctx context.Context, u *domain.User) error {
    if err := s.repo.Update(ctx, u); err != nil {
        return err
    }
    // Delete-on-write – never update the cache here
    s.cache.Del(ctx, userKey(u.ID))
    return nil
}

// GetUser uses singleflight to prevent cache stampede.
func (s *UserService) GetUser(ctx context.Context, id string) (*domain.User, error) {
    key := userKey(id)

    // Check cache first (outside singleflight – fast path)
    if data, err := s.cache.Get(ctx, key).Bytes(); err == nil {
        var u domain.User
        if json.Unmarshal(data, &u) == nil {
            return &u, nil
        }
    }

    // singleflight: only one goroutine loads from DB for this key at a time
    v, err, _ := s.sf.Do(key, func() (interface{}, error) {
        u, err := s.repo.FindByID(ctx, id)
        if err != nil {
            return nil, err
        }
        data, _ := json.Marshal(u)
        s.cache.Set(ctx, key, data, 10*time.Minute)
        return u, nil
    })
    if err != nil {
        return nil, err
    }
    return v.(*domain.User), nil
}

func userKey(id string) string {
    return fmt.Sprintf("user:v1:%s", id)
}
```

Exercise

Build: A stampede simulation with and without singleflight. **Input:** A `UserService.GetUser` with a mock repo. Simulate a cache miss for `user:1`. Launch 10 goroutines concurrently, all calling `GetUser("user:1")`. **Output:** Without singleflight: repo `FindByID` is called 10 times (observe with a counter). With singleflight: repo `FindByID` is called exactly 1 time. All 10 callers receive the correct result. **Acceptance:** Use an `atomic.Int64` counter on the mock repo to count calls. Without singleflight: counter ≥ 2 (likely 10). With singleflight: counter == 1. Run the test with `-race` — no data races.

Interview

- Why delete the cache key on write instead of updating it?
 - What is a cache stampede and why does it happen exactly when you least want it (high traffic + cold cache)?
 - What does the third return value from `singleflight.Do` tell you, and when does it matter?
-

Phase — 12-backend-architecture-ownership-and-design

Directory: 12-backend-architecture-ownership-and-design/

Source: 12-backend-architecture-ownership-and-design/00-quick-reference.md

Quick Reference — Backend Architecture

Package naming rules

- Name by WHAT it owns, not by what it does (not “utils”, “helpers”)
- One concept per package
- Avoid import cycles — use interfaces to break them

Layer dependency direction

cmd → service → domain (types only) ↓ repository (implements service interfaces)

Interface placement

// Define interfaces WHERE THEY ARE CONSUMED (service layer) // Implement them WHERE DATA LIVES (repository layer) // This makes service layer independently testable

DI: manual wiring in main()

repo := postgres.NewRepo(db) svc := service.New(repo) h := handler.New(svc) // If this doesn't compile, your graph is broken

Domain model rules

- No framework imports in domain/
 - Pure Go types + methods
 - Business logic lives here, not in handlers
-

Source: 12-backend-architecture-ownership-and-design/01-scope-architecture-capstone-ecommerce-backend.md

Unit 1 — Scope: Refactoring to Layered Architecture

What You Are Building

Take the working order system from Module 8 and refactor it into a properly layered e-commerce backend. By the end of this module you will have: domain types with business logic, a service layer that owns business rules, a repository layer for storage access, and an HTTP handler layer. No business logic in handlers. No HTTP types in services. No database imports in domain.

The Four Layers

```
handler/           - HTTP only: parse request, call service, write response
  order_handler.go

service/           - business rules only: validation, orchestration, interfaces
  order_service.go
  interfaces.go     - repository and cache interfaces live here

domain/            - types and invariants only: no external imports
  order.go
  product.go

repository/        - database only: sqlx, SQL queries, row mapping
  postgres_order.go
  postgres_product.go
```

Dependency Rules

```
handler  → service  ✓
handler  → domain   ✓
service  → domain   ✓
service  → repository interface (defined in service/) ✓
repository → domain ✓
repository → sqlx    ✓

handler  → repository ✗ (skip the service – bypass business rules)
service  → handler    ✗ (upward dependency)
domain   → anything   ✗ (domain is the core – no external imports)
```

What Each Unit Covers

Unit	Topic	Deliverable
2	Package naming	Audit and fix util soup
3	Layer discipline	Import graph verification
4	Manual DI	Wire everything in main()
5	Domain modeling	Order with behavior, not just data

What Success Looks Like

You can add a new business rule (orders over \$1000 require manual review) by changing only `service/order_service.go`. The handler and repository do not change. The rule is testable with mocks — no DB required.

Source: `12-backend-architecture-ownership-and-design/02-package-ownership-vs-util-soup.md`

Unit 2 — Package Ownership vs Util Soup

Concept

Package names should describe what they own, not what they do. `util`, `helpers`, `common`, and `misc` are symptoms of missing domain concepts — they are where code goes when you haven't decided who owns it. A package should have one reason to change: if you add a new feature and you're editing `util/`, the package name is wrong. Each function in `util` belongs in a specific domain package that owns that concept.

Code

```
// BAD: util package accumulates unrelated things
// util/util.go

package util

import (
    "fmt"
    "time"
    "strings"
)

func FormatMoney(amount float64) string { return fmt.Sprintf("%.2f", amount) }
func IsValidEmail(s string) string      { return strings.Contains(s, "@") }
func ParseOrderDate(s string) time.Time { /* ... */ }
func TruncateText(s string, n int) string { /* ... */ }

// ---

// GOOD: each function belongs to the package that owns the concept

// money/money.go — owns currency formatting and arithmetic
package money

func Format(amount float64) string { return fmt.Sprintf("%.2f", amount) }

// validation/email.go — owns input validation rules
package validation

func IsValidEmail(s string) bool { return strings.Contains(s, "@") && len(s) > 3 }

// domain/order.go — owns order parsing logic
package domain

func ParseOrderDate(s string) (time.Time, error) { return time.Parse(time.RFC3339, s) }

// display/text.go — owns presentation formatting
package display

func Truncate(s string, n int) string {
    if len(s) <= n { return s }
    return s[:n] + "..."
}
```

Exercise

Build: A package audit and refactor of your `go-lab/` directory. **Input:** Find every function in packages named `util`, `helpers`, `common`, or `misc`. **Output:** Move at least 3 functions to correctly named packages. Each function's new home should be named after the concept it belongs to, not what it does. **Acceptance:** Run `go build ./...` after the move — no compilation errors. The old package either no longer exists or contains only functions that genuinely belong together. Write a one-sentence comment at the top of each new package file: `// Package money owns currency arithmetic and formatting.`

Interview

- A package named `util` has grown to 800 lines. What does that tell you about the design?
 - How do you decide which package a function belongs in?
 - Two packages both need the same helper function. Is the answer to put it in `common`?
-

Unit 3 — Clean Architecture Layers and Dependency Discipline

Concept

Dependencies point inward: handler depends on service, service depends on domain and repository interfaces, domain depends on nothing. The domain package is the innermost layer — it has no external imports. When a layer needs something from an outer layer, it defines an interface that the outer layer implements. This inversion is what makes each layer independently testable and swappable.

Code

```
// Directory structure with import annotations:
//
// domain/          ← no imports from other layers
//   order.go       imports: stdlib only
//
// service/         ← imports domain only
//   order_service.go imports: domain, stdlib
//   interfaces.go   imports: domain, context – defines OrderRepository interface
//
// repository/      ← imports domain, sqlx
//   postgres_order.go imports: domain, sqlx, database/sql
//
// handler/         ← imports service, domain
//   order_handler.go imports: service, domain, net/http

// service/interfaces.go – service defines what it needs, not what exists
package service

import (
    "context"
    "github.com/example/app/domain"
)

// OrderRepository is defined by the service, satisfied by the repository package.
type OrderRepository interface {
    FindByID(ctx context.Context, id string) (*domain.Order, error)
    Create(ctx context.Context, userID string) (*domain.Order, error)
    UpdateStatus(ctx context.Context, id, status string) error
}

// ---

// domain/order.go – no external imports, only stdlib
package domain

import "errors"

var ErrOrderEmpty = errors.New("order has no items")

type Order struct {
    ID      string
    UserID  string
    Items   []OrderItem
    Status  string
}

// AddItem enforces domain rules – no DB, no HTTP
func (o *Order) AddItem(item OrderItem) error {
    if o.Status != "pending" {
        return errors.New("cannot add items to a non-pending order")
    }
    o.Items = append(o.Items, item)
    return nil
}

// ---

// Compile-time check: verify PostgresOrderRepo satisfies service.OrderRepository
// Put this in repository/postgres_order.go
var _ service.OrderRepository = (*PostgresOrderRepo)(nil)
```

Exercise

Build: Draw the dependency graph for your e-commerce service and verify it in code. **Input:** Your existing order system packages. **Output:** An ASCII dependency graph in a comment at the top of `service/order_service.go`. A compile-time interface check in `repository/postgres_order.go`. **Acceptance:** Run `go build ./...` — compile-time check passes. Use `grep -r "\"github.com/example/app/service\"" domain/` — result is empty (domain imports nothing from service). Use `grep -r "\"github.com/example/app/handler\"" service/` — result is empty (service imports nothing from handler).

Interview

- Why does the repository interface live in the service package instead of the repository package?
 - What breaks if `domain/order.go` imports `net/http`?
 - How do you verify at compile time that a struct satisfies an interface?
-

Source: `12-backend-architecture-ownership-and-design/04-manual-di-and-compilation-safety.md`

Unit 4 — Manual Dependency Injection and Compilation Safety

Concept

Wire all dependencies in `main()`. Constructors accept their dependencies as parameters — if something is missing, the code will not compile. There is no need for a DI framework in most Go services: explicit constructor injection is clearer, faster to trace, and catches missing dependencies at compile time. The wiring in `main()` documents the entire dependency graph of your service in one readable function.


```
// main.go — entire dependency graph visible in one place

package main

import (
    "log"
    "net/http"
    "os"
    "time"

    "github.com/jmoirn/sqlx"
    _ "github.com/lib/pq"
    "github.com/redis/go-redis/v9"

    "github.com/example/app/handler"
    "github.com/example/app/repository"
    "github.com/example/app/service"
)

func main() {
    // 1. Infrastructure
    db, err := sqlx.Connect("postgres", os.Getenv("DATABASE_URL"))
    if err != nil {
        log.Fatalf("db connect: %v", err)
    }
    db.SetMaxOpenConns(25)
    db.SetMaxIdleConns(5)
    db.SetConnMaxLifetime(5 * time.Minute)

    rdb := redis.NewClient(&redis.Options{
        Addr: os.Getenv("REDIS_URL"),
    })

    // 2. Repositories — depend on *sqlx.DB
    userRepo := repository.NewPostgresUserRepo(db)
    productRepo := repository.NewPostgresProductRepo(db)
    orderRepo := repository.NewPostgresOrderRepo(db)

    // 3. Services — depend on repository interfaces
    userSvc := service.NewUserService(userRepo, rdb)
    productSvc := service.NewProductService(productRepo, rdb)
    orderSvc := service.NewOrderService(orderRepo, productSvc)

    // 4. Handlers — depend on services
    userHandler := handler.NewUserHandler(userSvc)
    productHandler := handler.NewProductHandler(productSvc)
    orderHandler := handler.NewOrderHandler(orderSvc)

    // 5. Router
    mux := http.NewServeMux()
    mux.Handle("/users/", userHandler)
    mux.Handle("/products/", productHandler)
    mux.Handle("/orders/", orderHandler)

    // 6. Server
    srv := &http.Server{
        Addr:      ":" + os.Getenv("PORT"),
        Handler:    mux,
        ReadTimeout: 10 * time.Second,
        WriteTimeout: 30 * time.Second,
    }

    log.Printf("listening on %s", srv.Addr)
    if err := srv.ListenAndServe(); err != nil {
        log.Fatal(err)
    }
}
```



```
}  
}
```

Exercise

Build: Add an `EmailSender` dependency to `OrderService` that sends a confirmation email on order placement. **Input:** `EmailSender` is an interface in `service/interfaces.go` with one method: `Send(ctx, to, subject, body string) error`. `NewOrderService` now requires it as a parameter. **Output:** Update `main()` to construct and pass the email sender. Use a `service.NoopEmailSender` for local development (logs instead of sending). **Acceptance:** Remove the `EmailSender` from `NewOrderService`'s constructor call in `main()` — the code does not compile. Add it back — it compiles. The wiring in `main()` is the only place that changed.

Interview

- What does “compilation safety” mean in the context of dependency injection?
- Why is explicit wiring in `main()` preferred over a DI framework for most Go services?
- A new developer joins and wants to understand how the service is assembled. Where do you point them?

Source: `12-backend-architecture-ownership-and-design/05-domain-modelling-ddd-elements.md`

Unit 5 — Domain Modelling: Behavior, Not Just Data

Concept

A domain model contains business rules, not just data. An `Order` is not just a struct with fields — it has invariants (you cannot add an item to a completed order), behavior (total calculation), and state transitions (pending → confirmed → shipped). Domain methods enforce rules so they cannot be bypassed. If the rule lives only in the service layer, it can be skipped by calling the repository directly. If it lives in the domain type, it cannot.


```

// domain/order.go

package domain

import (
    "errors"
    "time"
)

type OrderStatus string

const (
    StatusPending   OrderStatus = "pending"
    StatusConfirmed OrderStatus = "confirmed"
    StatusShipped   OrderStatus = "shipped"
    StatusCancelled OrderStatus = "cancelled"
)

var (
    ErrOrderNotPending   = errors.New("order is not in pending state")
    ErrOrderEmpty        = errors.New("order has no items")
    ErrInvalidTransition = errors.New("invalid status transition")
)

type OrderItem struct {
    ProductID string
    Quantity  int
    UnitPrice float64
}

type Order struct {
    ID          string
    UserID      string
    Status      OrderStatus
    Items       []OrderItem
    CreatedAt   time.Time
}

// AddItem enforces the invariant: cannot modify a non-pending order.
func (o *Order) AddItem(item OrderItem) error {
    if o.Status != StatusPending {
        return ErrOrderNotPending
    }
    if item.Quantity <= 0 {
        return errors.New("quantity must be positive")
    }
    o.Items = append(o.Items, item)
    return nil
}

// Confirm transitions the order to confirmed. Requires at least one item.
func (o *Order) Confirm() error {
    if o.Status != StatusPending {
        return ErrInvalidTransition
    }
    if len(o.Items) == 0 {
        return ErrOrderEmpty
    }
    o.Status = StatusConfirmed
    return nil
}

// Total calculates the order value. Pure function – no DB, no network.
func (o *Order) Total() float64 {
    var total float64
    for _, item := range o.Items {

```

```
        total += float64(item.Quantity) * item.UnitPrice
    }
    return total
}
```

Exercise

Build: A `CartService` with domain-enforced rules. **Input:** A `Cart` domain type with: `AddItem`, `Checkout()` (`*Order`, `error`). **Output:** `Checkout` must enforce: (1) cannot checkout an empty cart, (2) item prices are snapshotted at checkout time (the returned `Order` stores the price at the moment of checkout, not a reference to live product data). **Acceptance:** Write tests: empty cart checkout returns `ErrCartEmpty`. Checkout with items returns an `Order` with unit prices equal to the prices at checkout time (not zero, not from a live DB call). Change a product's price after checkout — the `Order`'s item prices are unaffected.

Interview

- Why put `AddItem` validation in the domain type instead of the service?
 - What is the difference between a domain method and a service method?
 - The `Total()` method has no error return. Why is that appropriate?
-

Source: `12-backend-architecture-ownership-and-design/06-aggregate-boundary-tradeoffs.md`

Unit 6 — Aggregate boundary tension interrogation sharpening trade coherence

Dangerous cohesion collapse archetype caricature lumping disjoint lifecycles—`Orders + Inventory + Coupons + Reviews` casually unified provoking oversized transactional gravity wells hindering iterative scaling.

Articulate narrower aggregate boundaries conscientiously—even if implying eventual conversational synchronisation choreography previewing CQRS saga interplay forward areas.

Produce trade-off memo confronting reviewer hypothetical challenging boundary tightness accusing over-segmentation—defend succinctly evidencing behavioural independence statistics qualitative if lacking quantitative backlog metrics early lifecycle honesty.

Source: `12-backend-architecture-ownership-and-design/07-cqrs-read-vs-write-shape.md`

Unit 7 — CQRS read vs write model divergence literacy (purposeful—not reflexive duplication)

Articulate caricature pitfalls:

Repositories attempting identical shapes satisfying divergent analytic read patterns producing accidental coupling throttling iterative schema evolution velocities.

Separate conceptual paths responsibly:

```
Command handling mutating aggregates persisting optimised write-centric schema shapes
Versus query surfaces projecting denormalised / dashboard-friendly aggregates consciously accepting du
plication taxation only when evidenced necessity emerges—not speculative future-proof fetish premature
ly.
```

Practice divergence sketch

Demonstrate differentiated read model powering dashboard metrics distinct relational shapes vs normalized transaction tables—articulate syncing strategy high-level glimpsing event-driven projections later bridging distributed area optionally—not mandatory exhaustive implementation fidelity now.

Interview nuance disclaimers emphasising CQRS operational complexity tax honesty counterbalanced scaling wins articulation precision anti-dogmatic balanced maturity signal.

Source: `12-backend-architecture-ownership-and-design/08-architectural-composition-recap.md`

Unit 8 — Architectural composition revisit (scaling mindset beyond micro snippet examples)

Elevate fundamentals composition motifs—embedding collaborators / assembling services orchestrating orthogonal capabilities—preventing creeping inheritance impulses resurrecting OO habits misaligned pragmatic Go codebase sustainability reviews.

Conduct cross-cutting collaborator audit enumerating duplication risks vs justified reuse boundaries balancing consciously.

Brief oral rehearsal contrasting PHP inheritance-heavy historical reflex vs Go assembling explicit small interfaces interplay bridging earlier foundational Area 01 spiral intentionally closing conceptual loop matured scale now.

Source: `12-backend-architecture-ownership-and-design/09-strategic-package-realignment.md`

Unit 9 — Strategic package refactors guarding compile-time cohesion signals

Conduct deliberate reorganisation manoeuvre improving discovery / import graph clarity—even if mechanically incremental diff emphasising readability over aesthetic folder naming artistry shallowly judged.

Enumerate risks large-scale moves incur merge conflict penalties—articulate phased migration strategies balancing reviewer fatigue vs incremental drift accumulation hazards truthfully nuanced.

Interview prompt: versioning internal shared modules hypothetical multi-team concurrency constraints glimpsing forward organisational scaling pressures beyond solo learning scope honesty encouraged.

Source: `12-backend-architecture-ownership-and-design/10-cross-boundary-integration-rules.md`

Unit 10 — Cross-boundary rule enforcement: domain purity vs pragmatic Go pragmatism balance

Danger examples:

Polluting pure domain structs with omnipresent JSON / DB tags collapsing separation advantages inadvertently—exceptions permissible when consciously enumerated with reversible migration thoughts scheduled.

Maintain converter layer discipline mapping domain ↔ persistence ↔ transport payloads intentionally—not accidental silent aliasing divergence corrupting behavioural guarantees silently.

Conduct audit pass identifying leaky boundary offenders proposing remediation slices prioritising review blast radius ethically.

Source: `12-backend-architecture-ownership-and-design/11-errors-as-architecture-signals.md`

Unit 11 — Architectural error narratives signalling deeper systemic health than status codes superficially flashing

Elevate naive `return err` sprawl critiques matured earlier layering explicit domain error taxonomies aligning observability classifications guiding operator triage distinctions previewing reliability engineering interplay optional cross-disciplinary bridging.

Articulate interplay between sentinel business conflicts vs infra transient outage classifications shaping retry policies thoughtfully bridging distributed Areas forward intentionally referencing hook words only minimally now.

Conduct mini matrix mapping error classes → corrective actions referencing hypothetical pager responsibilities maturity signal interviewers relish concise clarity.

Source: 12-backend-architecture-ownership-and-design/12-tradeoff-studies-hyperscale-hypothetical.md

Unit 12 — Hyperscale hypothetical trade study (reasoning gymnastics without fake precision numerology)

Synthetic scenario authoring brief historically:

Imagine **≈5M monthly active-ish users** hypothetically & **≈100M orders** illustrative scale—articulate nuanced reasoning toggling optional accelerators:

CQRS divergence necessity? Read replica introduction vs caching amplification layering? Aggregate splits reconnaissance adjusting transactional contention hotspots? Operational cost sensitivities ethically acknowledged honesty vs buzzword binge shallow matrix spam lacking falsifiable claims humility signal mature hire trait.

Produce bullet trade matrix + explicit assumption ledger + falsification metrics you'd gather before rewriting architecture radically—intellectual humility prized.

Source: 12-backend-architecture-ownership-and-design/13-capstone-ecommerce-backend-integration.md

Unit 13 — ecommerce-backend/ architecture capstone integration consolidation

Mandatory functional arcs synthesising motifs:

```
order creation workflows
payments transitions + idempotency awareness bridging forward reminders
cancellation pathways reconciling inventory states consistently
dashboard read surfaces practising CQRS divergence earlier conceptual seeds if even lightweight specialised queries distinct shapes—demonstrating intentional duplication vs premature generalisation contrasts honestly
```

Documentation expectations emphasising WHY manifesto excerpts

Enumerate ownership rationale bullet clusters + boundary confrontation resolutions + conscientious shortcut admissions paired mitigation timers intellectual honesty signalling senior-like transparency balanced ambition vs pragmatism.

Oral rehearsal checklist aggregator bridging entire Area 11 interview lenses previously scattered—simulate uninterrupted guided tour narrative without peeking scaffolding notes muscle memory forging target.

Phase — 13-distributed-systems-resilience-and-messaging

Directory: 13-distributed-systems-resilience-and-messaging/

Source: 13-distributed-systems-resilience-and-messaging/00-quick-reference.md

Quick Reference — Distributed Systems Resilience

http.Client — always configure timeouts

```
client := &http.Client{
    Timeout: 10 * time.Second,
    Transport: &http.Transport{
        DialContext: (&net.Dialer{Timeout: 3*time.Second}).DialContext,
        ResponseHeaderTimeout: 5 * time.Second,
        MaxIdleConnsPerHost: 10,
    },
}
```

Retry rules

- Only retry idempotent operations
- Use exponential backoff with jitter
- Cap retries (3-5 max) with max backoff (10s)
- Check ctx.Done() between retries
- Don't retry: 400-499, context.Canceled

Idempotency key pattern

1. Client generates UUID key per logical operation
2. Server stores key + result on first execution
3. Server returns cached result on duplicate key
4. Key TTL: 24h–7d depending on operation

Failure categories

- Transient: retry (network timeout, 503)
- Permanent: don't retry (400, 404, auth failure)
- Unknown: retry once with backoff, then fail

Circuit breaker (conceptual)

Closed → (failures > threshold) → Open → (timeout) → Half-open → test

Source: 13-distributed-systems-resilience-and-messaging/01-scope-payment-system-micro-slices.md

Unit 1 — Scope: Payment System Micro-Slices

Concept

This module builds a payment processing client that handles the real failure modes of distributed systems. You will implement proper HTTP client timeouts, exponential backoff with jitter, and idempotency keys. By the end you have a client that can safely retry failed payment requests without double-charging a customer.

Code

```
// What you will build across this module:
//
// PaymentClient
//   - http.Client with explicit Timeout and Transport settings
//   - withRetry: exponential backoff, jitter, max attempts, context cancellation
//   - Charge(ctx, idempotencyKey, amount) – safe to call multiple times
//
// PaymentServer (test double)
//   - Accepts idempotency key in header
//   - Returns cached result on duplicate key
//   - Sleeps or fails on demand to simulate real gateway behavior
//
// By end of module the following holds:
//   - A 10s slow server gets a timeout error in ~5s
//   - A server that fails 3/5 times still succeeds after retry
//   - Calling Charge twice with the same key charges exactly once

package payments // placeholder – each unit fills this in
```

Exercise

Build: Nothing yet — this is orientation. Read the scope, then sketch on paper: what are the three things that can go wrong between client and payment gateway? **Input:** Your own reasoning **Output:** Three failure scenarios written down: (1) server never responds, (2) server responds with error, (3) server succeeds but response is lost **Acceptance:** You can explain why scenario 3 requires idempotency keys but scenarios 1 and 2 do not

Interview

- What is the difference between a timeout and a retry?
- Why is it unsafe to retry a POST to a payment gateway without an idempotency key?
- Name two things you must configure on `http.Client` to prevent goroutine leaks on slow servers.

Source: `13-distributed-systems-resilience-and-messaging/02-partial-failure-reality-check.md`

Unit 2 — Partial Failure Reality Check

Concept

In distributed systems, a request can succeed on the server but the response never arrives. The client sees a network error and doesn't know if the server processed it or not. This is the fundamental problem that requires idempotency. Naive retry on any error causes double-processing — the server ran the work twice.

Code

```
package main

import (
    "errors"
    "fmt"
    "math/rand"
)

var processed int // counts how many times the charge actually ran

// simulateCharge processes 80% of the time but always returns a network
// error 20% of the time — even when the work was done.
func simulateCharge(amount int) error {
    processed++ // work is done before we know if the response succeeds
    if rand.Float64() < 0.20 {
        return errors.New("network error: response lost")
    }
    return nil
}

// naiveRetry retries on every error — demonstrates double-processing.
func naiveRetry(amount int) (int, error) {
    calls := 0
    for i := 0; i < 5; i++ {
        calls++
        if err := simulateCharge(amount); err == nil {
            return calls, nil
        }
    }
    return calls, errors.New("all attempts failed")
}

func main() {
    const trials = 1000
    totalProcessed := 0
    totalSuccessResponses := 0

    for i := 0; i < trials; i++ {
        processed = 0
        _, err := naiveRetry(100)
        totalProcessed += processed
        if err == nil {
            totalSuccessResponses++
        }
    }

    fmt.Printf("Trials: %d\n", trials)
    fmt.Printf("Total times charge actually ran: %d\n", totalProcessed)
    fmt.Printf("Total successful responses: %d\n", totalSuccessResponses)
    fmt.Printf("Double-charge ratio: %.2f\n", float64(totalProcessed)/float64(totalSuccessResponses))
}
```

Exercise

Build: Run the program above and capture the output. **Input:** 1000 trials of naiveRetry calling simulateCharge **Output:** Print how many times the simulated charge executed vs how many times the caller received a success response. Show the ratio.

Acceptance: The double-charge ratio is greater than 1.0 — proving that naive retry on any error causes more charges than successful responses

Interview

- A client retries a payment POST after a timeout. The server processed the first request. What happens?
 - What is the difference between “at-most-once” and “at-least-once” delivery?
 - Name the only piece of information a client needs to detect a duplicate before resending.
-

Source: `13-distributed-systems-resilience-and-messaging/03-http-clients-timeout-mandates.md`

Unit 3 — HTTP Clients: Timeout Mandates

Concept

`http.DefaultClient` has no timeouts — a slow server will block your goroutine forever. Configure `Timeout` on `http.Client` for the full round-trip budget, and set connection and TLS timeouts on the `Transport`. Use `context` for per-request cancellation on top of client-level timeouts. Both layers are needed: client timeout is a hard ceiling, context lets callers cancel early.

Code

```
package main

import (
    "context"
    "fmt"
    "net/http"
    "time"
)

// Danger: DefaultClient has zero timeouts – never use for outbound calls.
var _ = http.DefaultClient

// newPaymentClient returns an http.Client with all timeout layers configured.
func newPaymentClient() *http.Client {
    transport := &http.Transport{
        DialContext: (&net.Dialer{
            Timeout: 3 * time.Second, // TCP connect timeout
            KeepAlive: 30 * time.Second,
        }).DialContext,
        TLSHandshakeTimeout: 5 * time.Second,
        ResponseHeaderTimeout: 10 * time.Second, // time to first byte
        MaxIdleConns: 100,
        MaxIdleConnsPerHost: 10,
        IdleConnTimeout: 90 * time.Second,
    }
    return &http.Client{
        Timeout: 5 * time.Second, // total request budget (overrides all above)
        Transport: transport,
    }
}

func chargeWithTimeout(ctx context.Context, client *http.Client, url string) error {
    req, err := http.NewRequestWithContext(ctx, http.MethodPost, url, nil)
    if err != nil {
        return fmt.Errorf("build request: %w", err)
    }
    resp, err := client.Do(req)
    if err != nil {
        return fmt.Errorf("do request: %w", err) // context.DeadlineExceeded or url.Error with timeout
    }
    defer resp.Body.Close()
    return nil
}
```

Exercise

Build: A `PaymentGatewayClient` with a 5s `Timeout`. Point it at a test server that sleeps 10s before responding. **Input:** One HTTP POST to the slow test server **Output:** The client returns an error containing “timeout” or “deadline exceeded” in approximately 5 seconds **Acceptance:** `time.Since(start) < 6*time.Second` and `err != nil` — the client did not wait 10s

Interview

- What is the difference between `http.Client.Timeout` and `ResponseHeaderTimeout` on the Transport?
 - Why is `http.DefaultClient` dangerous in production services?
 - If you set a 5s client timeout and also pass a context with a 2s deadline, which one wins?
-

Source: `13-distributed-systems-resilience-and-messaging/04-retry-policies-with-backoff.md`

Unit 4 — Retry Policies with Backoff

Concept

Retry with exponential backoff: first retry after 100ms, double each time, cap at 30s, max 5 attempts. Add jitter (random $\pm 20\%$) to prevent thundering herd — without it, all clients retry at the same instant and amplify the outage. Only retry transient errors (timeouts, 5xx); never retry 4xx or non-retryable business errors.

Code

```
package main

import (
    "context"
    "errors"
    "fmt"
    "math/rand"
    "net/http"
    "time"
)

func isRetryable(err error) bool {
    var urlErr *url.Error
    if errors.As(err, &urlErr) && urlErr.Timeout() {
        return true
    }
    return false
}

func isRetryableStatus(code int) bool {
    return code == http.StatusTooManyRequests ||
        code == http.StatusBadGateway ||
        code == http.StatusServiceUnavailable ||
        code == http.StatusGatewayTimeout
}

func withRetry(ctx context.Context, fn func() error) error {
    const (
        maxAttempts = 5
        base        = 100 * time.Millisecond
        cap         = 30 * time.Second
    )
    var err error
    for attempt := 0; attempt < maxAttempts; attempt++ {
        err = fn()
        if err == nil {
            return nil
        }
        if !isRetryable(err) {
            return err // non-retryable – stop immediately
        }
        if attempt == maxAttempts-1 {
            break
        }
        // exponential backoff with ±20% jitter
        backoff := base * (1 << attempt)
        if backoff > cap {
            backoff = cap
        }
        jitter := time.Duration(rand.Int63n(int64(backoff) / 5))
        if rand.Intn(2) == 0 {
            jitter = -jitter
        }
        wait := backoff + jitter

        select {
        case <-ctx.Done():
            return ctx.Err()
        case <-time.After(wait):
        }
    }
    return fmt.Errorf("all %d attempts failed: %w", maxAttempts, err)
}
```

Exercise

Build: Test `withRetry` with a mock function that fails 3 times then succeeds. **Input:** A mock that returns a retryable error on attempts 1–3 and nil on attempt 4 **Output:** Print each attempt number and timestamp. Show that total attempts = 4 and the time between calls increases. **Acceptance:** (1) total attempts = 4, (2) each wait is longer than the previous, (3) passing a cancelled context stops retries immediately without waiting for the next backoff

Interview

- Why add jitter to backoff? What is thundering herd?
 - What is the difference between a retryable and a non-retryable error? Give one example of each.
 - If max attempts = 5 and base = 100ms, what is the maximum wait time before the 5th attempt (before jitter)?
-

Source: `13-distributed-systems-resilience-and-messaging/05-idempotency-keys-payments-double-charge-story.md`

Unit 5 — Idempotency Keys: The Double-Charge Story

Concept

An idempotency key is a UUID the client generates per logical operation. The server stores the key plus the result. On retry with the same key, the server returns the cached result without re-executing. Set a TTL on the key (e.g., 24h) — after that, the same key is treated as a new request. The client must generate the key before the first attempt and reuse it on every retry.


```

package main

import (
    "errors"
    "fmt"
    "sync"
    "time"
)

type ChargeResult struct {
    TransactionID string
    Amount        int
    ProcessedAt   time.Time
}

type idempotencyRecord struct {
    result      ChargeResult
    expiresAt   time.Time
}

// PaymentServer is a test double with idempotency key support.
type PaymentServer struct {
    mu      sync.Mutex
    store   map[string]idempotencyRecord
    charges int // counts real executions
}

func (s *PaymentServer) Charge(idempotencyKey string, amount int) (ChargeResult, error) {
    s.mu.Lock()
    defer s.mu.Unlock()

    // Check for existing result.
    if rec, ok := s.store[idempotencyKey]; ok {
        if time.Now().Before(rec.expiresAt) {
            return rec.result, nil // cached – not re-executed
        }
    }

    // Execute the charge.
    s.charges++
    result := ChargeResult{
        TransactionID: fmt.Sprintf("txn-%d", s.charges),
        Amount:        amount,
        ProcessedAt:   time.Now(),
    }

    s.store[idempotencyKey] = idempotencyRecord{
        result:      result,
        expiresAt:   time.Now().Add(24 * time.Hour),
    }
    return result, nil
}

func main() {
    server := &PaymentServer{store: make(map[string]idempotencyRecord)}
    key := "idem-key-abc123"

    r1, _ := server.Charge(key, 100)
    r2, _ := server.Charge(key, 100) // retry with same key

    fmt.Printf("First response: txnID=%s\n", r1.TransactionID)
    fmt.Printf("Second response: txnID=%s\n", r2.TransactionID)
    fmt.Printf("Charges executed: %d\n", server.charges)

    if r1.TransactionID != r2.TransactionID {
        panic(errors.New("idempotency broken: different transaction IDs"))
    }
}

```



```
}  
}
```

Exercise

Build: Send the same charge request twice with the same idempotency key. Add a counter on the mock gateway that increments only when a real charge runs (not on cache hit). **Input:** Two calls to `Charge` with identical idempotency key and amount **Output:** Both calls return the same `TransactionID`. `server.charges == 1`. **Acceptance:** `r1.TransactionID == r2.TransactionID` and `server.charges == 1` — charge executed once, second call returned cached result

Interview

- Where should the client generate the idempotency key — before or after the first attempt?
 - What happens if a server stores idempotency keys forever with no TTL?
 - Can two different operations share the same idempotency key? What goes wrong?
-

Source: `13-distributed-systems-resilience-and-messaging/06-queue-decoupling-rabbitmq-basics.md`

Unit 6 — RabbitMQ queue decoupling: move work off the critical path ethically

Contrast overloaded synchronous fan-out caricature pushing email/analytics/notifications inline until checkout latency becomes user-facing hostage situation.

Responsible outline:

```
checkout critical path ⇒ enqueue durable intent message ⇒ worker draining asynchronously respecting  
consumers operational realities acknowledging new tail latency failure classes appearing honestly
```

Practice

Wire a modest publisher/consumer with explicit ack/nack reasoning—understand poison messages preview next DLQ discussions.

Interview expectation: verbally contrast queue introduction tax (operability, duplication handling) vs latency wins / blast-radius containment virtues balanced non-absolutism.

Source: `13-distributed-systems-resilience-and-messaging/07-delivery-semantics-at-most-once-least-once.md`

Unit 7 — Delivery semantics: at-most-once, at-least-once, and “exactly-once” marketing vs engineering honesty

Distributed messaging rarely matches marketing slogans. Your job is to state what your system actually provides and what handlers must tolerate.

Semantics sketch

At-most-once

A message might never arrive. You must not rely on implicit delivery unless the business accepts loss.

At-least-once

Duplicates are possible unless you dedupe/idempotently process. RabbitMQ workloads are commonly in this territory unless you add strict dedupe semantics elsewhere.

“Exactly-once”

Often shorthand for engineered outcomes (idempotent handlers + dedupe keys + transactional outbox patterns), not an OS/network primitive you get for free.

Practice

Replay the same queued job twice deliberately. Confirm your consumer does not corrupt state (`INSERT` blindly without keys, naive counters exploding, duplicate side effects).

Interview prompts

- What does your broker actually guarantee vs what your application must guarantee?
 - Where do you dedupe—in DB, metadata store, keyed business tables?
-

Source: `13-distributed-systems-resilience-and-messaging/08-dead-letter-queues-recovery-flows.md`

Unit 8 — Dead-letter queues: escape infinite poisonous retry traps

Operational story: a malformed message or a deterministic bug retries forever—burning CPU, harming shared brokers, burying legitimate traffic.

Define a humane policy:

- bounded retries while errors look transient / unknown,
- then route poison messages into a **dead-letter queue** path (conceptual name varies by broker) for isolated inspection/replay tooling.

Document what operators need: payloads, timestamps, correlation headers, replay procedure that does not blindly double-charge (link idempotency, Unit 5).

Interview prompts

Difference between DLQ as “junk drawer” versus an operational cockpit with alerting and replay runbooks tied to compensation logic.

Source: `13-distributed-systems-resilience-and-messaging/09-sagas-compensating-actions.md`

Unit 9 — Sagas & compensating actions across services

Across service boundaries there is usually **no** single ACID transaction that can atomically commit `payment + inventory + email + loyalty points`. Coordination becomes an explicit workflow with **partial failure** baked in.

Core mental model

- **Forward actions** advance business state (`ReserveInventory` , `CapturePayment`).
- **Compensating actions** undo earlier forward steps best-effort when later steps fail (`ReleaseReservation` , `RefundPayment`), subject to strict business/legal rules—not every rollback is symmetrical or instant.

Design tasks:

1. Decide what “done” means for each step and what artefacts prove it (records, statuses, immutable events).
2. Make steps **retry-safe** using idempotency keys where external systems can double-apply ambiguous calls.
3. Document where you cannot compensate fully (often payment rails); pair with ops runbooks honest about manual intervention.

Compare coordination styles

- **Orchestration**: central component drives next steps explicitly (think workflow engine vibes or your own saga coordinator)—clear flow, coupling risk concentrated.
- **Choreography**: services react to signals/events—fewer choke points but harder reasoning about implicit contracts.

Interview prompt: sketch one concrete failure midpoint and enumerate compensations invoked in order—and what happens if a compensation fails.

Source: `13-distributed-systems-resilience-and-messaging/10-backpressure-queue-growth-throttling.md`

Unit 10 — Backpressure when producers outrun consumers

Problems appear as **queue depth growth**, goroutine buildup, latency balloons, RAM pressure—or worse, masking failures until cascading timeouts occur.

Signals to watch mentally (even qualitatively in a toy):

```
ingress rate sustained > worker drain capability
timeouts rising while CPUs look partially idle ⇒ blocking / IO / lock contention hypotheses
producer goroutines spawning without bound drowning scheduler fairness illusions ethically debunk polity
```

Intervention patterns (pick consciously)

- Bounded queues + rejecting/shedding load at the HTTP edge when saturated.
- Throttling callers (tokens, leaky buckets) before they enqueue unbounded speculative work.
- Scaling workers realistically (more CPU-bound vs IO-bound distinctions matter).
- Revisiting idempotency and retry aggression when storms self-inflict outages.

Interview drill: articulate where backpressure is applied (client, gateway, publisher, broker, consumer batch sizing) and the user-visible degradation you accept under overload.

Source: `13-distributed-systems-resilience-and-messaging/11-distributed-tracing-correlation-notes.md`

Unit 11 — Correlation IDs and minimal distributed tracing awareness

Failures cross service hops; plaintext “error happened” logs at each hop recreate detective novels nobody finishes.

Practice baseline

Attach and propagate minimally:

```
request id (edge assigned)
upstream trace-ish correlation token (whatever your org standardises—don't Cargo-cult a vendor name here blindly)
tenant/user identifiers ethically cautioned PII sensitivities—even learning exercises practise redaction instincts
```

Log fields should line up grepably across hops even before full OpenTelemetry depth—establish mental habit: **carry context into nested calls**.

Interview prompts

Cardinality explosion when logging high-cardinality labels indiscriminately.

Sampling intuition: you cannot economically retain every span forever—articulate pragmatic retention vs debuggability tensions honestly.

Source: 13-distributed-systems-resilience-and-messaging/12-chaos-micro-lab-catalog.md

Unit 12 — Chaos rehearsal catalogue (small, controlled fault injections)

Build intentional failure rehearsals—notebook/diagram-heavy is fine—to complement code:

Fault	Symptoms you expect	Fix story you must articulate
payment backend slow / flaky	timeout storms, cascading cancellation	tightened budgets + retry sanity + degraded UX policy
queue broker restart	backlog spike, duplicated deliveries surfaced	durable publisher confirms / consumer idempotency / DLQ path
worker crash mid-batch	orphaned partial writes unless transactional boundaries exist	transactional outbox glimpses responsibly optional advanced bridging
network partition caricature verbally	ambiguous success	idempotency + reconciliation queries patterns

Produce a miniature **timeline write-up**: detection → containment → remediation → preventative guard next iteration.

Interview mindset: rehearsals beat heroics—you can describe how you practise partial failure calmly.

Source: 13-distributed-systems-resilience-and-messaging/13-capstone-payment-system-integration.md

Unit 13 — Capstone: resilient payment-system/ checkout choreography

Compose an end-to-end story (code depth proportionate honest to your sandbox):

```
checkout
  → payment
  → inventory reservation / adjustment choreography
  → queue-backed notification acknowledgement path
```

Must deliberately exercise earlier motifs

timeouts • bounded retries • idempotency keys • RabbitMQ-ish queue semantics pragmatically • delivery honesty • DLQ emergence • saga/compensation narration • correlation identifiers propagation minimally • backpressure narrative when workers insufficient capacity honesty

Fault replay notebook

Enumerate duplicate requests, ambiguous timeouts (“did payment land?”), poison messages, flaky workers—recover with written rationale bridging **delivery semantics**, **handler idempotency**, and **human-runbook** pointers where automation cannot ethically replace judgement.

Interview rehearsal: summarise the system aloud as if onboarding a sceptical teammate—prioritise correctness arguments over toolchain trivia.

Phase — 14-workflow-durable-execution-and-temporal-concepts

Directory: 14-workflow-durable-execution-and-temporal-concepts/

Source: 14-workflow-durable-execution-and-temporal-concepts/00-quick-reference.md

Quick Reference — Temporal Workflows

Key types

```
workflow.Context          // NOT context.Context — deterministic
workflow.ExecuteActivity(ctx, fn, args...).Get(ctx, &result)
workflow.ExecuteChildWorkflow(ctx, fn, args...).Get(ctx, &result)
workflow.GetSignalChannel(ctx, signalName)
workflow.SetQueryHandler(ctx, queryName, handlerFn)
```

Activity options

```
workflow.WithActivityOptions(ctx, workflow.ActivityOptions{
    StartToCloseTimeout: 30*time.Second,
    RetryPolicy: &temporal.RetryPolicy{MaxAttempts: 3},
})
```

Workflow rules (determinism)

- NO: time.Now(), rand, goroutines, direct I/O
- YES: workflow.Now(), workflow.Go(), workflow.ExecuteActivity()
- All side effects go through activities

Error handling

```
temporal.NewNonRetryableApplicationError(msg, errType, cause)
// use for: invalid input, business rule violations
```

Worker registration

```
w := worker.New(client, taskQueue, worker.Options{})
w.RegisterWorkflow(MyWorkflow)
w.RegisterActivity(MyActivity) // or &ActivityStruct{}
```

Source: 14-workflow-durable-execution-and-temporal-concepts/01-durable-workflows-temporal-shape.md

Unit 1 — Durable Workflows: Temporal Shape

Concept

Durable execution means a workflow survives process restarts. Temporal records every step to an event history. On restart, it replays the history to restore state — your workflow function is re-run from the top, but `ExecuteActivity` calls return immediately from history instead of re-executing. This means your workflow function must be deterministic: same inputs always produce the same sequence of calls.

Code

```
// Fragile: in-process job queue loses state on restart.
type InMemoryJob struct {
    Steps []func() error
}

func runFragile(job InMemoryJob) error {
    for _, step := range job.Steps {
        if err := step(); err != nil {
            return err // on restart, progress is lost — starts from zero
        }
    }
    return nil
}

// Durable: Temporal workflow survives restart.
// On restart, Temporal replays the event history. Each ExecuteActivity call
// checks history first — if the activity already completed, the result is
// returned from history without calling the activity again.
func PaymentWorkflow(ctx workflow.Context, orderID string) error {
    ao := workflow.ActivityOptions{
        StartToCloseTimeout: 30 * time.Second,
    }
    ctx = workflow.WithActivityOptions(ctx, ao)

    // If process restarts here, Temporal replays history.
    // ProcessPaymentActivity is NOT called again if it already succeeded.
    if err := workflow.ExecuteActivity(ctx, ProcessPaymentActivity, orderID).Get(ctx, nil); err !=
nil {
        return err
    }
    return workflow.ExecuteActivity(ctx, SendConfirmationActivity, orderID).Get(ctx, nil)
}
```

Exercise

Build: Nothing to run yet. Identify which of these are safe inside a Temporal workflow function and explain why. **Input:** These five calls: `time.Now()`, `rand.Int()`, `os.ReadFile("config.json")`, `workflow.ExecuteActivity(ctx, MyActivity)`, `workflow.Now(ctx)` **Output:** Safe: `workflow.ExecuteActivity`, `workflow.Now(ctx)`. Unsafe: `time.Now()`, `rand.Int()`, `os.ReadFile()`. **Acceptance:** You can explain the rule: if the call produces a different result on replay than on first execution, it is not safe in a workflow

Interview

- What does Temporal do with the event history when a worker restarts?
- Why is calling `time.Now()` inside a workflow dangerous but `workflow.Now(ctx)` safe?
- What is an activity, and why does Temporal retry it instead of retrying the workflow?

Source: `14-workflow-durable-execution-and-temporal-concepts/02-temporal-workflow-and-activity-anatomy.md`

Unit 2 — Temporal Workflow and Activity Anatomy

Concept

A **workflow** is a durable, deterministic function — it must not perform I/O directly. An **activity** wraps all side effects and is the unit Temporal retries.

Code

```
// Workflow — deterministic, no I/O directly
func OrderWorkflow(ctx workflow.Context, orderID string) error {
    ao := workflow.ActivityOptions{
        StartToCloseTimeout: 30 * time.Second,
        RetryPolicy: &temporal.RetryPolicy{
            MaxAttempts: 3,
            InitialInterval: time.Second,
        },
    }
    ctx = workflow.WithActivityOptions(ctx, ao)

    var result PaymentResult
    err := workflow.ExecuteActivity(ctx, ProcessPaymentActivity, orderID).Get(ctx, &result)
    if err != nil {
        return err
    }

    return workflow.ExecuteActivity(ctx, SendConfirmationActivity, orderID, result).Get(ctx, nil)
}

// Activity — can do I/O, is retried by Temporal
func ProcessPaymentActivity(ctx context.Context, orderID string) (PaymentResult, error) {
    // real I/O here: call payment gateway
    return gateway.Charge(ctx, orderID)
}

// Register and start worker
w := worker.New(client, "orders-queue", worker.Options{})
w.RegisterWorkflow(OrderWorkflow)
w.RegisterActivity(ProcessPaymentActivity)
w.Run(worker.InterruptCh())
```

Concern	Workflow	Activity
Determinism	Required — replayed from history	Not required
I/O	Forbidden	Allowed
Retries	Managed by Temporal on failure	Yes, per RetryPolicy
Timeouts	ScheduleToClose, StartToClose	StartToClose, ScheduleToClose

Exercise

Build: An order fulfillment workflow with three activities in sequence: `ProcessPayment` → `ReserveInventory` → `SendConfirmation` **Input:** `OrderID string` passed to the workflow **Output:** Each activity prints what it did. If `ReserveInventory` returns an error, the workflow returns that error and `SendConfirmation` is never called. `ProcessPayment` is not retried for reservation failures. **Acceptance:** Run the worker, start the workflow via CLI (`tctl workflow run` or Temporal UI), observe all three activity executions in the Temporal UI event history. Force `ReserveInventory` to fail — verify the workflow fails and `SendConfirmation` does not appear in history.

Interview

- Why must workflows be deterministic? (Temporal replays history to reconstruct state after a crash.)
- What happens if you call `time.Now()` inside a workflow? (Non-determinism — use `workflow.Now()` instead.)
- Can an activity fail halfway through? What happens? (Temporal retries from the start of that activity invocation.)

Unit 3 — Temporal Signals and Queries

Concept

Signals push external events into a running workflow asynchronously. **Queries** read workflow state without mutating it — pure read, safe to call any time.

Code

```
// Signal: external event pushed into a running workflow
func OrderWorkflow(ctx workflow.Context, orderID string) error {
    cancelCh := workflow.GetSignalChannel(ctx, "cancel-order")

    selector := workflow.NewSelector(ctx)
    selector.AddReceive(cancelCh, func(c workflow.ReceiveChannel, more bool) {
        // handle cancellation signal
    })

    // workflow.Select blocks until a channel is ready
    selector.Select(ctx)
    return nil
}

// Send signal from outside
client.SignalWorkflow(ctx, workflowID, runID, "cancel-order", nil)

// Query: read workflow state without mutating it
workflow.SetQueryHandler(ctx, "get-status", func() (string, error) {
    return currentStatus, nil
})

// Execute query from outside
val, err := client.QueryWorkflow(ctx, workflowID, runID, "get-status")
```

	Signal	Query
Mutates state	Yes	No
Async	Yes	No (synchronous response)
Persisted in history	Yes	No
Use case	Cancel, approve, external event	Status check, audit read

Exercise

Build: An order workflow that waits for a `confirm` signal before processing payment. The workflow logs “waiting for confirm” when it starts, then “confirmed, processing payment” after the signal arrives. **Input:** Signal sent via `tctl workflow signal --name confirm` or Temporal UI **Output:** Workflow logs “waiting for confirm” immediately, then pauses. After signal: logs “confirmed, processing payment” and continues. **Acceptance:** Kill and restart the worker while the workflow is waiting for the signal — the workflow resumes correctly after the worker comes back up, and the signal sent before the restart is not lost

Interview

- What happens if a signal arrives before the workflow registers the channel? (Temporal buffers it — delivered when the channel is registered.)
- Why can't a query handler mutate state? (Queries are not recorded in history — mutations would break replay determinism.)

Unit 4 — Temporal Error Handling and Retries

Concept

Temporal distinguishes **retryable** errors (default — Temporal will retry per policy) from **non-retryable** errors (halt the retry loop immediately and fail the activity or workflow).

Code

```
// Non-retryable error — stops retry loop immediately
return temporal.NewNonRetryableApplicationError("invalid card", "CARD_INVALID", nil)

// Retryable (default) — Temporal retries per ActivityOptions.RetryPolicy
return fmt.Errorf("gateway timeout: %w", err)

// ActivityOptions retry policy
ao := workflow.ActivityOptions{
    StartToCloseTimeout: time.Minute,
    RetryPolicy: &temporal.RetryPolicy{
        InitialInterval:    time.Second,
        BackoffCoefficient: 2.0,
        MaximumInterval:    time.Minute,
        MaxAttempts:        5,
        NonRetryableErrorTypes: []string{"CARD_INVALID", "INSUFFICIENT_FUNDS"},
    },
}

// Heartbeat for long-running activities
func LongActivity(ctx context.Context) error {
    for i := 0; i < 1000; i++ {
        activity.RecordHeartbeat(ctx, i) // keeps activity alive
        if activity.GetInfo(ctx).Attempt > 1 {
            // resume from heartbeat details on retry
        }
    }
    return nil
}
```

Error type	Behaviour	Use case
<code>fmt.Errorf(...)</code>	Retryable	Transient failures, gateway timeouts
<code>temporal.NewNonRetryableApplicationError</code>	Not retried	Invalid input, business rule violations
<code>context.DeadlineExceeded</code>	Depends on policy	Timeout — usually retryable

Exercise

Build: A payment activity that returns `NonRetryableApplicationError` for card number “invalid” and a normal retryable error for card number “timeout”. **Input:** Card number string — use “invalid” to trigger non-retryable, “timeout” to trigger retryable **Output:** For “invalid”: workflow fails immediately with no retries. For “timeout”: activity retries 3 times with backoff, then workflow fails. **Acceptance:** Observe retry count in Temporal UI event history — “invalid” shows 1 attempt total; “timeout” shows exactly 3 attempts with increasing intervals between them

Interview

- What is `StartToCloseTimeout` vs `ScheduleToCloseTimeout` ? (Start-to-close: single attempt budget. Schedule-to-close: total budget across all retries.)
- When do you use heartbeats? (Long-running activities — without heartbeats, Temporal cannot detect a stuck worker until timeout.)

- Why mark an error non-retryable? (Retrying `CARD_INVALID` wastes time and charges the card again if the activity isn't idempotent.)
-

Phase — 15-messaging-advanced-kafka-and-event-ownership

Directory: 15-messaging-advanced-kafka-and-event-ownership/

Source: 15-messaging-advanced-kafka-and-event-ownership/00-quick-reference.md

Quick Reference — Kafka & Event Ownership

Producer (segmentio/kafka-go)

```
w := &kafka.Writer{
    Addr:      kafka.TCP("localhost:9092"),
    Topic:     "orders",
    Balancer:  &kafka.Hash{}, // consistent partition by key
}
w.WriteMessages(ctx, kafka.Message{Key: []byte(orderID), Value: payload})
```

Consumer with group

```
r := kafka.NewReader(kafka.ReaderConfig{
    Brokers: []string{"localhost:9092"},
    GroupID: "service-name",
    Topic:   "orders",
})
msg, _ := r.FetchMessage(ctx)
// process...
r.CommitMessages(ctx, msg) // AFTER processing — at-least-once
```

Ordering guarantee

```
// Within partition: guaranteed
// Across partitions: NO guarantee
// Use consistent key (userID, orderID) for per-entity ordering
```

Delivery semantics

```
at-most-once:  commit before processing (can lose)
at-least-once: commit after processing (can duplicate) ← default choice
exactly-once:  transactions on both ends (complex, Kafka-specific)
```

Patterns

```
Outbox: write event to DB table in same TX as data, poll+publish separately
Inbox:  store processed message IDs to detect and skip duplicates
```

Source: 15-messaging-advanced-kafka-and-event-ownership/01-scope-from-queue-to-log-event-backbone.md

Unit 1 — Scope: From Queue to Log — Event Backbone

Concept

This module builds an event-driven order processing pipeline. The producer is an order-service that publishes `OrderCreated` events to Kafka. The consumer is an inventory-service that processes them. By the end you have working segmentio/kafka-go producers and consumers, consumer group rebalancing, and the outbox pattern for reliable cross-service publication.

Code

```
// What you will build across this module:
//
// order-service (producer)
//   - Creates an order in the DB
//   - Writes an OrderCreated event to an outbox table in the same transaction
//   - A background poller reads the outbox and publishes to Kafka
//
// inventory-service (consumer)
//   - Consumer group: reads OrderCreated events from Kafka
//   - Checks inbox table before processing (idempotent)
//   - Commits offset only after successful processing
//
// By end of module the following holds:
//   - Order and event are always consistent (both saved or neither)
//   - Two consumer instances share the partition load
//   - Restarting a consumer replays only uncommitted messages

package main // placeholder — each unit fills this in
```

Exercise

Build: Nothing yet. Sketch on paper: what happens if order-service inserts an order to DB, then crashes before publishing to Kafka? **Input:** Your reasoning about the gap between DB write and Kafka publish **Output:** Identify the consistency problem. Write: which approach fixes it — (a) publish to Kafka then write to DB, (b) write to DB then publish to Kafka, (c) write both in one transaction using an outbox table **Acceptance:** You can explain why (a) and (b) both have failure windows but (c) does not

Interview

- What is the fundamental difference between a message queue and a Kafka topic?
- Why does Kafka guarantee ordering within a partition but not across partitions?
- What is the outbox pattern and which problem does it solve?

Source: 15-messaging-advanced-kafka-and-event-ownership/02-kafka-foundations-partitions-and-ordering.md

Unit 2 — Kafka Foundations: Partitions and Ordering

Concept

A Kafka topic is an append-only log split into partitions. Ordering is guaranteed within a partition, not across partitions. Messages with the same partition key always go to the same partition. Use a consistent partition key (e.g., `orderId` or `userId`) when order matters for that entity. A message published without a key is round-robin across partitions.

Code

```
package main

import (
    "context"
    "fmt"
    "github.com/segmentio/kafka-go"
)

func newWriter(brokers []string, topic string) *kafka.Writer {
    return &kafka.Writer{
        Addr:      kafka.TCP(brokers...),
        Topic:     topic,
        Balancer: &kafka.Hash{}, // consistent: same key → same partition
    }
}

func publishOrder(ctx context.Context, w *kafka.Writer, userID, orderID string, payload []byte) error {
    return w.WriteMessages(ctx, kafka.Message{
        Key:   []byte(userID), // partition key: all orders for same user go to same partition
        Value: payload,
        Headers: []kafka.Header{
            {Key: "orderID", Value: []byte(orderID)},
        },
    })
}

func readPartition(brokers []string, topic string, partition int) {
    r := kafka.NewReader(kafka.ReaderConfig{
        Brokers:   brokers,
        Topic:     topic,
        Partition: partition,
        MinBytes:  1,
        MaxBytes:  10e6,
    })
    defer r.Close()

    for {
        m, err := r.ReadMessage(context.Background())
        if err != nil {
            break
        }
        fmt.Printf("partition=%d offset=%d key=%s\n", m.Partition, m.Offset, m.Key)
    }
}
```

Exercise

Build: Publish 10 orders for 3 users (key = userID, e.g., “user-1”, “user-2”, “user-3”). Read them back directly from each partition.
Input: 10 messages with keys distributed across 3 users on a topic with 3 partitions **Output:** Print each message with its partition number and offset. Show that all messages for “user-1” are in the same partition. **Acceptance:** For each userID, all messages land in one partition. Within each partition, messages appear in the order they were published.

Interview

- A topic has 3 partitions. Two messages have the same key. Are they guaranteed to be in the same partition?
 - Can you reorder messages within a partition by replaying from an earlier offset?
 - What happens to ordering guarantees if you increase partition count on an existing topic?
-

Unit 3 — Consumer Groups: Rebalancing and Commit Semantics

Concept

A consumer group is multiple consumers sharing the work of consuming a topic. Each partition is consumed by exactly one consumer in the group at a time. Commit the offset after processing, not before (at-least-once delivery). On restart, the consumer re-reads from the last committed offset — so you may see the last message again. Your processing must be idempotent to handle this safely.

Code

```
package main

import (
    "context"
    "fmt"
    "github.com/segmentio/kafka-go"
)

func newGroupReader(brokers []string, topic, groupID string) *kafka.Reader {
    return kafka.NewReader(kafka.ReaderConfig{
        Brokers:  brokers,
        Topic:    topic,
        GroupID:  groupID, // enables consumer group — auto-assigns partitions
        MinBytes: 1,
        MaxBytes: 10e6,
        // CommitInterval: 0 means manual commit
    })
}

func consumeWithManualCommit(ctx context.Context, r *kafka.Reader, process func(kafka.Message) error) error {
    for {
        // FetchMessage does NOT commit the offset.
        m, err := r.FetchMessage(ctx)
        if err != nil {
            return err
        }

        if err := process(m); err != nil {
            // Do not commit — message will be re-delivered on next FetchMessage.
            fmt.Printf("processing failed, not committing offset %d: %v\n", m.Offset, err)
            continue
        }

        // Commit only after successful processing.
        // On crash before this line, the message is re-delivered.
        if err := r.CommitMessages(ctx, m); err != nil {
            return fmt.Errorf("commit offset %d: %w", m.Offset, err)
        }
    }
}
```

Exercise

Build: Start 2 consumers in the same group on a topic with 2 partitions. Publish 100 messages. Each consumer processes its assigned partition. Kill one consumer mid-run. **Input:** 100 messages published to a 2-partition topic. Two readers with the same

`GroupID`. **Output:** Total messages processed across both consumers = 100. After killing one consumer, the surviving consumer picks up the orphaned partition and processes the remaining messages. **Acceptance:** No message is skipped (total processed = 100). If your handler is idempotent, no message is double-processed after the rebalance.

Interview

- What is the difference between `FetchMessage` and `ReadMessage` in kafka-go?
 - If you crash after processing but before committing, what happens to that message?
 - Can two consumers in the same group read the same partition simultaneously?
-

Source: `15-messaging-advanced-kafka-and-event-ownership/04-outbox-pattern-reliable-cross-service-publication.md`

Unit 4 — Outbox Pattern: Reliable Cross-Service Publication

Concept

Write the event to an outbox table in the same DB transaction as your business data. A separate poller reads the outbox and publishes to Kafka. This guarantees the event is published if and only if the business operation committed. Without the outbox, a crash between DB write and Kafka publish leaves the downstream service permanently unaware.


```

package main

import (
    "context"
    "database/sql"
    "encoding/json"
    "fmt"
    "time"
)

type OrderCreatedEvent struct {
    OrderID string
    UserID  string
    Amount  int
}

// CreateOrder writes the order and the outbox event in one transaction.
// Either both are committed or neither is — no consistency gap.
func CreateOrder(ctx context.Context, db *sql.DB, orderID, userID string, amount int) error {
    tx, err := db.BeginTx(ctx, nil)
    if err != nil {
        return err
    }
    defer tx.Rollback()

    _, err = tx.ExecContext(ctx,
        "INSERT INTO orders (id, user_id, amount) VALUES ($1, $2, $3)",
        orderID, userID, amount)
    if err != nil {
        return fmt.Errorf("insert order: %w", err)
    }

    payload, _ := json.Marshal(OrderCreatedEvent{OrderID: orderID, UserID: userID, Amount: amount})
    _, err = tx.ExecContext(ctx,
        "INSERT INTO outbox (id, topic, key, payload, published) VALUES ($1, $2, $3, $4, false)",
        orderID, "orders.created", userID, payload)
    if err != nil {
        return fmt.Errorf("insert outbox: %w", err)
    }

    return tx.Commit()
}

// PollAndPublish reads unpublished outbox events and publishes them to Kafka.
// Runs in a background goroutine on a tick.
func PollAndPublish(ctx context.Context, db *sql.DB, publish func(key, payload []byte) error) error {
    rows, err := db.QueryContext(ctx,
        "SELECT id, key, payload FROM outbox WHERE published = false ORDER BY created_at LIMIT 100")
    if err != nil {
        return err
    }
    defer rows.Close()

    for rows.Next() {
        var id, key string
        var payload []byte
        if err := rows.Scan(&id, &key, &payload); err != nil {
            return err
        }
        if err := publish([]byte(key), payload); err != nil {
            return fmt.Errorf("publish %s: %w", id, err)
        }
    }
    _, err = db.ExecContext(ctx,
        "UPDATE outbox SET published = true, published_at = $1 WHERE id = $2",
        time.Now(), id)
    if err != nil {

```

```
        return err
    }
}
return rows.Err()
}
```

Exercise

Build: Simulate `CreateOrder` failing halfway through (commit returns an error). Then run a successful `CreateOrder`. Run `PollAndPublish` and check what appears in Kafka. **Input:** One failed order creation (rollback), one successful order creation. **Output:** After `PollAndPublish` runs, exactly one event appears in Kafka — the successful order. The failed order produces no event. **Acceptance:** Kafka message count = 1. The event's `OrderID` matches the successful order. `PollAndPublish` runs within 1s of the successful `CreateOrder` commit.

Interview

- What happens if `PollAndPublish` crashes after publishing to Kafka but before marking the row as published?
- Why must the outbox poller be idempotent?
- Can you use CDC (change data capture) instead of a polling outbox? What are the tradeoffs?

Source: `15-messaging-advanced-kafka-and-event-ownership/05-inbox-pattern-idempotent-consumer-safety.md`

Unit 5 — Inbox Pattern: Idempotent Consumer Safety

Concept

An inbox table stores the ID of every processed message. Before processing, check if the message ID is already in the inbox. If yes, skip it. This makes the consumer idempotent — safe to receive the same message twice. Insert the message ID and the processing result in the same transaction as your business write, so you never record “processed” without the work actually completing.

Code

```
package main

import (
    "context"
    "database/sql"
    "errors"
    "fmt"
)

var ErrAlreadyProcessed = errors.New("message already processed")

// ProcessWithInbox checks the inbox before processing and records
// the message ID in the same transaction as the business write.
func ProcessWithInbox(ctx context.Context, db *sql.DB, messageID string, process func(*sql.Tx) error) error {
    tx, err := db.BeginTx(ctx, nil)
    if err != nil {
        return err
    }
    defer tx.Rollback()

    // Check inbox – was this message already processed?
    var exists bool
    err = tx.QueryRowContext(ctx,
        "SELECT EXISTS(SELECT 1 FROM inbox WHERE message_id = $1)", messageID).Scan(&exists)
    if err != nil {
        return fmt.Errorf("inbox check: %w", err)
    }
    if exists {
        return ErrAlreadyProcessed // idempotent skip
    }

    // Run business logic inside the same transaction.
    if err := process(tx); err != nil {
        return fmt.Errorf("process: %w", err)
    }

    // Record message as processed in the same transaction.
    _, err = tx.ExecContext(ctx,
        "INSERT INTO inbox (message_id, processed_at) VALUES ($1, NOW())", messageID)
    if err != nil {
        return fmt.Errorf("inbox insert: %w", err)
    }

    return tx.Commit()
}
```

Exercise

Build: Publish the same message twice (same Kafka key and same payload). Your consumer calls `ProcessWithInbox` for each delivery. Add a counter that increments inside the `process` function. **Input:** Two Kafka messages with identical message ID (set via a header) **Output:** `process` is called once. The second call returns `ErrAlreadyProcessed`. Counter = 1. Inbox table has one row for that message ID. **Acceptance:** `counter == 1` and `inbox` table has exactly one row for the message ID after both deliveries are processed

Interview

- Why must you insert the inbox record in the same transaction as the business write?
- What happens if you check the inbox but insert it after the business write in a separate transaction?
- Is the inbox pattern sufficient on its own, or do you also need the outbox pattern?

Source: 15-messaging-advanced-kafka-and-event-ownership/06-poison-messages-dead-letter-and-replay-governance.md

Unit 6 — Poison scenarios, replay, operational governance—not endless retry loops

Poison message caricature deterministic bug or schema mismatch causing infinite catastrophic failure churn—must graduate to segregated remediation path analogous DLQ philosophies yet broker-specific knobs differ.

Operational responsibilities:

bounded retries distinguishing transient infra vs poisonous semantic failure
replay tooling auditable respecting idempotency + legal duplication constraints consciously
prevent unsafe mass replays multiplying financial inconsistencies responsibly

Kafka nuance teaser

Understand **offsets commit** interplay with poisonous partition stall—whole partition stuck if single bad record mishandled—contrasts naive queue acknowledging pattern assumptions.

Interview scenario: formulate policy table classifying retries vs skipping vs patching schema vs deploying fix-forward strategies.

Source: 15-messaging-advanced-kafka-and-event-ownership/07-event-driven-ownership-contracts-evolution-and-sagas-bridge.md

Unit 7 — Event-driven ownership: contracts, versioning, choreography discipline

Elevate storytelling beyond payloads:

Ownership concern	Behavioural question
Schema/version	Breaking vs additive evolution & dual publishing windows.
Causal boundaries	Which events must imply prior state guarantees?
Compensation interplay	How inventory corrective events interplay with saga failure arcs from Area 12.
Trace correlation	Propagate linkage IDs bridging Area 06 / 12 motifs intentionally.

Event-driven temptation anti-pattern: anarchic choreography without explicit **ownership matrix** collapsing debugging into distributed guesswork responsibly debunk verbally.

Produce mini **ownership table** hypothetical `payment.capture`, `inventory.reserve`, `inventory.compensated` delineating emitting team & consuming invariants verbally articulable interviewer satisfying clarity signal matured deliberately.

Source: 15-messaging-advanced-kafka-and-event-ownership/08-capstone-payment-event-inventory-narrative-lab.md

Unit 8 — Capstone narration: payment → verified event emission → inventory reaction

Produce written architecture & sequence justification (diagram optional) practising earlier motifs—not necessarily full infra cluster fidelity now.

Mandatory reasoning hooks (tick each explicitly in prose):

transactional persistence + outbox (or explicit refusal with compensating design + risks stated)
partitioning key coherence vs hotspots / skew honesty
inbox / idempotent consumer interplay OR alternative dedupe discipline stated
bounded retries vs poison isolation + replay governance story
Lag / backlog ageing metrics intuition (even if dashboards are stubs)
RabbitMQ vs Kafka trade matrix from your Areas 12 ↔ 14 story—no slogan absolutism

Deliver **short design note** citing:

Section	Requirement
Data path	Payment commit → emission → inventory acknowledgement.
Failure modes	Poison, duplication, partitioning skew, saga compensation tie-in at least verbally.
Interview compare	Structured bullet RabbitMQ vs Kafka without absolutism—state operational triggers choosing each.

Oral rehearsal: summarise end-to-end in ≤3 minutes calmly staff-style.

Phase — 16-performance-profiling-and-perf-lab

Directory: 16-performance-profiling-and-perf-lab/

Source: 16-performance-profiling-and-perf-lab/00-quick-reference.md

Quick Reference — Performance Profiling

Benchmark flags

```
go test -bench=.           # run all benchmarks
go test -bench=BenchmarkFoo # specific
go test -benchmem          # show allocations
go test -count=3           # run N times (check variance)
go test -cpuprofile cpu.out # save CPU profile
go test -memprofile mem.out # save memory profile
```

pprof HTTP endpoint

```
import _ "net/http/pprof"
go http.ListenAndServe("localhost:6060", nil)
```

```
# collect 30s CPU profile
go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30
# heap
go tool pprof http://localhost:6060/debug/pprof/heap
```

pprof commands

```
(pprof) top10      # top 10 by CPU/alloc
(pprof) web        # flame graph (needs graphviz)
(pprof) list funcName # annotate source
(pprof) peek funcName # callers + callees
```

Reading -benchmem output

```
BenchmarkFoo 1000000 1234 ns/op 128 B/op 3 allocs/op
                ↑           ↑           ↑
                nanosec    bytes/call  alloc count
```

Allocation reduction

```
strings.Builder > string concat in loop
sync.Pool for frequently allocated objects
preallocate slices: make([]T, 0, knownCap)
```

Source: 16-performance-profiling-and-perf-lab/01-scope-latency-ownership-measure-first.md

Unit 1 — Scope: Latency Ownership — Measure First

Concept

This module profiles a slow HTTP handler, finds the bottleneck, and fixes it with evidence. You will write benchmarks, collect pprof CPU and heap profiles, and make changes guided by data instead of intuition. The rule is: measure first, optimize second — the bottleneck is almost never where you think it is.

Code

```
// What you will build across this module:
//
// slowHandler (provided)
//   - An HTTP handler with multiple inefficiencies built in
//   - Unnecessary allocations in the hot path
//   - A blocking call where none is needed
//   - Redundant JSON encoding
//
// Your job, unit by unit:
//   1. Write benchmarks — establish a baseline ns/op and allocs/op
//   2. Add pprof — collect a CPU profile under load with hey or wrk
//   3. Read the flame graph — find the widest frame that is your code
//   4. Fix the top bottleneck — measure again to confirm improvement
//   5. Repeat for heap profile — find inuse_objects growth, fix it
//
// Target: reduce p99 latency by at least 40% using only profiling evidence

package perf // placeholder — each unit fills this in
```

Exercise

Build: Nothing yet. Read the scope and answer: what is the difference between a benchmark and a pprof profile? When do you need each? **Input:** Your reasoning **Output:** Write two sentences: (1) what a benchmark tells you that a profile does not, (2) what a profile tells you that a benchmark does not **Acceptance:** You can explain: benchmark = reproducible measurement of a specific code path; profile = where real CPU time goes during a realistic workload

Interview

- What does `-benchmem` show that `-bench` alone does not?
- Why should you never optimize code you haven't benchmarked first?
- Name two tools in the Go standard toolchain for measuring performance.

Source: 16-performance-profiling-and-perf-lab/02-benchmark-thinking-and-microbenchmark-hygiene.md

Unit 2 — Benchmark Thinking and Microbenchmark Hygiene

Concept

Benchmarks run until time stabilizes (`b.N` iterations, chosen by the test runner). `-benchmem` shows bytes/op and allocs/op — often more important than ns/op because allocations drive GC pauses. Run `-count=3` to check variance. A 10% difference with high variance is noise; a 3× difference with low variance is real.

Code

```
package bench_test

import (
    "strings"
    "testing"
)

// Slow: + operator creates a new string on every iteration.
func concatPlus(parts []string) string {
    result := ""
    for _, p := range parts {
        result += p
    }
    return result
}

// Fast: strings.Builder pre-allocates and appends in place.
func concatBuilder(parts []string) string {
    var b strings.Builder
    b.Grow(len(parts) * 10) // hint: avoids re-growth
    for _, p := range parts {
        b.WriteString(p)
    }
    return b.String()
}

var parts = make([]string, 100)

func init() {
    for i := range parts {
        parts[i] = "word"
    }
}

func BenchmarkConcatPlus(b *testing.B) {
    for i := 0; i < b.N; i++ {
        _ = concatPlus(parts)
    }
}

func BenchmarkConcatBuilder(b *testing.B) {
    for i := 0; i < b.N; i++ {
        _ = concatBuilder(parts)
    }
}

// Run: go test -bench=. -benchmem -count=3
// Expected output:
// BenchmarkConcatPlus-8      50000      24000 ns/op   45000 B/op   99 allocs/op
// BenchmarkConcatBuilder-8  500000      2400 ns/op    1024 B/op    2 allocs/op
```

Exercise

Build: Benchmark `json.Marshal` for a struct with 10 fields. Then encode the same struct using a pre-allocated `bytes.Buffer` and `json.NewEncoder`. **Input:** A struct with 10 string/int fields. Benchmark both approaches with `-benchmem -count=3`.

Output: Two benchmark results showing ns/op, B/op, allocs/op for each approach. **Acceptance:** The `json.NewEncoder` with pre-allocated buffer shows fewer allocs/op than `json.Marshal`. Explain why: `Marshal` allocates a new `[]byte`; `NewEncoder` writes into your buffer.

Interview

- What does `b.N` represent in a Go benchmark?
 - Why run with `-count=3` instead of just `-count=1`?
 - A benchmark shows 5000 ns/op and 20 allocs/op. Which should you optimize first for a high-QPS service?
-

Source: `16-performance-profiling-and-perf-lab/03-allocation-pressure-stack-heap-first-pass.md`

Unit 3 — Allocation Pressure: Stack, Heap, First Pass

Concept

Stack allocations are free — they are just a pointer move. Heap allocations require GC. A variable escapes to heap when it outlives its function (returned as pointer, captured in closure, stored in interface). Use `-gcflags="-m"` to see which variables escape and why. The goal is to keep hot-path variables on the stack.

Code

```
package main

// Case 1: returns int — stays on stack, no allocation.
func stackInt() int {
    x := 42
    return x // value copied out, x does not escape
}

// Case 2: returns *int — x escapes to heap.
// go build -gcflags="-m" prints: "moved to heap: x"
func heapPointer() *int {
    x := 42
    return &x // x must outlive the function — escapes
}

// Case 3: stored in interface{} — value is boxed on heap.
// go build -gcflags="-m" prints: "42 escapes to heap"
func interfaceBox() interface{} {
    return 42 // integer boxed into interface{} header — heap allocation
}

// Case 4: closure captures variable — captured vars escape to heap.
func closureCapture() func() int {
    x := 42
    return func() int { return x } // x captured, escapes to heap
}

func main() {
    _ = stackInt()
    _ = heapPointer()
    _ = interfaceBox()
    _ = closureCapture()
}

// Run: go build -gcflags="-m" ./...
// Look for lines containing "escapes to heap" or "moved to heap"
```

Exercise

Build: Run `go build -gcflags="-m" ./...` on your e-commerce service (or any service from a prior module). Find 3 variables that escape to heap. For one of them, rewrite the code to avoid the escape. Verify with `-benchmem` that `allocs/op` decreased. **Input:** Existing service code **Output:** Three escape analysis findings from the compiler output. Before/after

benchmark showing allocs/op for the function you fixed. **Acceptance:** The rewritten function shows fewer allocs/op in the benchmark. You can explain the reason for each of the three escapes you found.

Interview

- What is escape analysis and who performs it in Go?
 - Name three things that cause a variable to escape from stack to heap.
 - Why does storing a value in an `interface{}` cause a heap allocation?
-

Source: 16-performance-profiling-and-perf-lab/04-cpu-profiling-http-pprof-workflow.md

Unit 4 — CPU Profiling: HTTP pprof Workflow

Concept

The pprof HTTP endpoint exposes CPU, heap, goroutine, and mutex profiles. Collect a 30s CPU profile under load with `go tool pprof`. The flame graph shows where CPU time is spent — the widest frame at the top is where time is consumed most. Optimize the widest frame that is your code, not the standard library.

Code

```
package main

import (
    "log"
    "net/http"
    _ "net/http/pprof" // registers /debug/pprof/* handlers on DefaultServeMux
)

func main() {
    // Debug server — expose on a separate port, never on the public port.
    go func() {
        log.Println(http.ListenAndServe(":6060", nil))
    }()

    // Your application server.
    mux := http.NewServeMux()
    mux.HandleFunc("/", yourHandler)
    log.Fatal(http.ListenAndServe(":8080", mux))
}

// Collect a 30s CPU profile:
// go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30
//
// Open interactive browser UI:
// go tool pprof -http=:8081 http://localhost:6060/debug/pprof/profile?seconds=30
//
// Generate load while collecting (use hey or wrk):
// hey -n 10000 -c 50 http://localhost:8080/
//
// In the browser: View > Flame Graph
// The widest frame at the top is where your CPU time goes.
```

Exercise

Build: Add pprof to your API service from a prior module. Send 1000 requests with `hey` while collecting a 30s CPU profile. Open the profile in the browser flame graph. **Input:** 1000 HTTP requests against your running service. CPU profile collected during the load. **Output:** A flame graph showing the top CPU consumer. Write down: what is the widest frame that belongs to your code (not `runtime` or `net/http`)? **Acceptance:** You can point to a specific function in the flame graph and explain why it consumes CPU. If the top consumer is framework overhead, document that finding — that is a valid result.

Interview

- What is the pprof endpoint URL for a CPU profile?
 - Why should the debug server run on a different port from the application server?
 - What does a wide frame in a CPU flame graph mean?
-

Source: 16-performance-profiling-and-perf-lab/05-heap-profiling-live-vs-allocated-objects.md

Unit 5 — Heap Profiling: Live vs Allocated Objects

Concept

`inuse_objects` is what is alive right now. `alloc_objects` is total allocated since start — this measures GC pressure even if objects are collected quickly. Run `runtime.GC()` before taking a heap profile for a cleaner picture of what is actually live. A function that allocates a lot but GCs quickly is less harmful than one with long-lived allocations.

Code

```
package main

import (
    "net/http"
    "runtime"
    _ "net/http/pprof"
)

// Simulate a leak: allocating a large slice on every request and keeping
// a reference in a global slice prevents GC from collecting them.
var leak [][]byte

func leakyHandler(w http.ResponseWriter, r *http.Request) {
    buf := make([]byte, 1<<20) // 1 MB per request
    leak = append(leak, buf)    // retains reference — never collected
    w.WriteHeader(http.StatusOK)
}

// Non-leaky: allocate, use, let go — GC collects after each request.
func cleanHandler(w http.ResponseWriter, r *http.Request) {
    buf := make([]byte, 1<<20)
    _ = buf // used, then eligible for GC
    w.WriteHeader(http.StatusOK)
}

// Force GC then take a heap profile snapshot:
// curl http://localhost:6060/debug/pprof/heap?gc=1 > heap1.prof
// go tool pprof -http=:8081 heap1.prof
//
// Wait 10s, repeat:
// curl http://localhost:6060/debug/pprof/heap?gc=1 > heap2.prof
//
// In the browser: switch between "inuse_space" and "alloc_space" views.
// inuse_space grows over time with leakyHandler but stays flat with cleanHandler.
```

Exercise

Build: Add `leakyHandler` to a test server. Send 50 requests. Collect heap profiles 10s apart. Compare `inuse_objects` between the two profiles. **Input:** 50 POST requests to `/leak`. Two heap profiles taken 10s apart. **Output:** Profile 1 and Profile 2 `inuse_space` values. Show the growth. Then switch to `cleanHandler`, repeat — show `inuse_space` stays flat. **Acceptance:** With `leakyHandler`: `inuse_space` in profile 2 is larger than in profile 1. With `cleanHandler`: `inuse_space` is approximately equal between profiles.

Interview

- What is the difference between `inuse_objects` and `alloc_objects` in a heap profile?
 - Why run `runtime.GC()` before taking a heap snapshot?
 - A heap profile shows high `alloc_space` but low `inuse_space`. Is there a memory leak?
-

Source: `16-performance-profiling-and-perf-lab/06-garbage-collector-allocation-pressure-and-pauses.md`

Unit 6 — Garbage collector thinking: allocation pressure & pause awareness

Go's GC is **non-magical**: more allocation churn generally means more GC work (CPU + latency variance), even if individual pauses are often short relative to classic stop-the-world horror stories.

Practice

Allocate ~100k short-lived objects in tight loops under bench harness—watch **GC-related cost** indirectly via:

```
-benchmem
cpu + heap profiles (previous units)
optional GODEBUG=gctrace=1 textual logs (interpret qualitatively; don't overfit local laptop numbers)
```

Lab

Explain in prose what **allocation pressure** means for tail latency at API scale—not average-only dashboards delusions.

Interview prompts

Generational GC myths misapplied to Go—stay factual with current runtime docs when disputing trivia.

Trade-off: micro-alloc elimination vs readability—optimise only with evidence (Area intent).

Source: `16-performance-profiling-and-perf-lab/07-memory-leak-patterns-goroutines-slices-channels.md`

Unit 7 — Memory “leak” thinking in Go: goroutines, slices, channels

Go can **retain memory** without classic malloc/free leaks:

Pattern	Symptom sketch
Goroutine leak	forgotten <code>ctx</code> cancellation / blocked sends / waiting forever
Slice backing array retention	tiny visible slice holding giant underlying array alive
Channel / map caches	unbounded growth “feature”

Practice

Seed an intentional **goroutine leak** (ignore `Done` + blocking channel), observe process RSS climb / goroutine count via `pprof` goroutine profile or runtime metrics patterns.

Fix with cancellation + join discipline cross-linking Area `03/04`.

Interview expectation

Differentiate **unreachable cycles** (GC reclaim) vs **reachable accidental retention** (profiler shows live objects still referenced).

Source: 16-performance-profiling-and-perf-lab/08-slice-and-map-performance-patterns.md

Unit 8 — Slice & map performance: capacity, growth, bucket physics (high level)

Slice preallocation:

```
s := make([]T, 0, 1000) // fewer reallocations vs blind append growth
```

Maps: growth & rehash events exist—avoid hot loops recreating maps unintentionally; pre-size when approximate cardinality known.

Lab

Benchmark **prealloc** vs **naive append** and a **map heavy** micro workload; narrate `B/op` + `allocs/op` shifts.

Interview prompts

When preallocation misleads (overalloc memory permanently for tiny inputs).

Map iteration order non-determinism unrelated but often adjacent confusion—don't conflate with perf story unless relevant.

Source: 16-performance-profiling-and-perf-lab/09-concurrency-performance-and-worker-scaling-laws.md

Unit 9 — Concurrency performance: more goroutines ≠ faster

Measure:

```
worker pool sizes: 5 vs 50 vs 500
```

on **CPU-bound** vs **IO-bound** toy workloads—observe contention, scheduling overhead, and throughput collapse when oversubscribed CPU work.

Lab deliverable

Plot a small qualitative table of outcomes (even handwritten) and explain how they relate to coarse parallel speedup intuition (CPU-bound saturation vs IO-bound overlap)—without treating a laptop benchmark like a mathematical proof.

Source: 16-performance-profiling-and-perf-lab/10-json-and-serialization-hot-path-benchmarks.md

Unit 10 — JSON / hot-path serialization performance benching (`encoding/json` first)

Bench:

```
small structs vs medium vs large payloads  
Decoder reuse vs naive patterns where applicable thoughtfully  
alternative codecs only as optional appendix if curious—maintain honesty verifying semantics compatibility trade-offs consciously
```

Lab

Document where JSON becomes bottleneck vs database/network overshadowing guesses—articulate investigative question ordering for “slow endpoints” bridging next latency ownership unit.

Interview focus: schema shape & field count ripple effects—not just “JSON slow” slogans.

Source: 16-performance-profiling-and-perf-lab/11-latency-ownership-and-bottleneck-triage.md

Unit 11 — Latency ownership: triage CPU / memory / GC / DB / network systematically

Synthetic production scenario:

You receive “this API is slow.” Forbidden answer: random micro-optimisation gut feelings.

Checklist mental walkthrough

Layer	Quick honest probes
CPU	profile hot functions
Alloc/GC	- benchmem , heap profile, alloc pressure
DB	query plans / connection pool waits (cross Area 08)
Network	timeouts, retries storm, payload bloat
Queues	backlog age / consumer lag (Areas 12 / 14)

Practice

Introduce intentional slow dependency (sleeping fake client) AND independent CPU hog path—prove you can separate symptoms using tools rather than story-telling.

Deliverable

Short **incident note template**: symptom → hypothesis → validation evidence → fix → trade-off.

Source: 16-performance-profiling-and-perf-lab/12-profiling-incident-synthetic-lab.md

Unit 12 — Profiling incident lab (synthetic): CPU pegged, memory climbing, workers stalling

Compose multi-signal chaos in perf-lab/ :

CPU ~95% busy spin or pathological algorithm
memory growth via leak pattern(s)
worker pool starved or oversubscribed scenario mixing IO + CPU skew

You must:

profile (CPU + heap at minimum)
identify dominant contributors
patch with measured before/after bench/prof snapshot commentary
document trade-offs (readability loss? bounded buffers?)

Interview rehearsal: narrate investigation without opening laptop—prove muscle memory.

Source: 16-performance-profiling-and-perf-lab/13-capstone-payment-worker-performance-lab.md

Unit 13 — Capstone: `payment-worker/` performance integration

Build (or extend) `payment-worker/` exercising:

```
worker pool + queue consumption + retry + timeout discipline (bridge Areas 12/14 concepts lightly)
```

Then inject **deliberate pathologies**:

```
allocation / GC pressure scenario  
goroutine leak scenario  
memory retention via slice backing array trick (educational)
```

Use `pprof` + **benchmarks** to detect and fix—document **WHY** fixes work structurally.

Interview consolidation checklist (must speak smoothly):

```
pprof CPU/heap  
benchmark discipline  
GC / allocation pressure intuition  
leak classes  
slice prealloc trade-offs  
stack vs heap first-pass literacy (deepening Area 16/23)
```

Phase — 17-go-runtime-internals-foundations-for-profilers

Directory: 17-go-runtime-internals-foundations-for-profilers/

Source: 17-go-runtime-internals-foundations-for-profilers/00-quick-reference.md

Quick Reference — Go Runtime Internals

GMP model

```
G = goroutine (user-space, lightweight ~2KB stack)
M = OS thread (managed by runtime)
P = processor context (GOMAXPROCS count, default=nCPU)
```

GOMAXPROCS

```
runtime.GOMAXPROCS(n)    // set P count
GOMAXPROCS=1             // useful to eliminate scheduling noise in benchmarks
```

Escape analysis

```
go build -gcflags="-m" ./...    // see what escapes to heap
go build -gcflags="-m -m" ./... // verbose (shows reason)
```

GC tuning

```
GOGC=100      // default: GC when heap 2× live data
GOGC=200      // less frequent GC, more memory
GOMEMLIMIT=512MiB // Go 1.19+: memory ceiling
```

Execution trace

```
import "runtime/trace"
trace.Start(f); defer trace.Stop()
go tool trace trace.out
```

Blocking categories

```
// Cooperatively yields to scheduler:
channel ops, syscalls, runtime.Gosched(), time.Sleep()
// Does NOT yield (burns P):
pure CPU loops, math-heavy code
```

Unit 1 — GMP Scheduler Model and Blocking Intuition

Concept

G = goroutine (user-space, starts at 2KB stack, grows as needed). M = OS thread (managed by the runtime). P = logical processor (GOMAXPROCS, default = number of CPUs). When a goroutine blocks on I/O, its M is released and another goroutine takes over the P. CPU-bound goroutines must yield cooperatively — in Go 1.14+, the runtime preempts them with signals so other goroutines still get CPU time.

Code

```
package main

import (
    "fmt"
    "runtime"
    "sync"
    "time"
)

func cpuBound(id int, wg *sync.WaitGroup) {
    defer wg.Done()
    start := time.Now()
    sum := 0
    for i := 0; i < 100_000_000; i++ {
        sum += i
    }
    fmt.Printf("goroutine %d done in %v (sum=%d)\n", id, time.Since(start), sum)
}

func main() {
    const n = 4

    for _, procs := range []int{1, n} {
        runtime.GOMAXPROCS(procs)
        var wg sync.WaitGroup
        start := time.Now()
        for i := 0; i < n; i++ {
            wg.Add(1)
            go cpuBound(i, &wg)
        }
        wg.Wait()
        fmt.Printf("GOMAXPROCS=%d total time: %v\n\n", procs, time.Since(start))
    }
}
```

Exercise

Build: Run the program above. Set `GOMAXPROCS=1`, then `GOMAXPROCS=4` (or match your CPU count). Run 4 CPU-bound goroutines each doing 100 million iterations. **Input:** 4 goroutines, each doing 100M additions **Output:** Total wall time for each GOMAXPROCS setting **Acceptance:** With GOMAXPROCS=4, total time is approximately 4× faster than GOMAXPROCS=1. The speedup confirms the work is CPU-bound and parallel.

Interview

- What is a P in the GMP model and why does its count matter for CPU-bound work?
- What happens to the M (OS thread) when a goroutine blocks on a network read?

- A program has GOMAXPROCS=2 and 100 goroutines. How many goroutines run simultaneously?

Source: 17-go-runtime-internals-foundations-for-profilers/02-stacks-heaps-and-escape-analysis-instructs.md

Unit 2 — Stacks, Heaps, and Escape Analysis Instincts

Concept

Stack: per-goroutine, automatic, fast — allocating is a pointer move. Heap: GC-managed, slower — used when a value must outlive its function. Escape analysis decides where each variable lives at compile time. You can see every decision with `go build -gcflags="-m"`. The goal: keep hot-path variables on the stack.

Code

```
package main

// stackReturn: value copied out — x stays on stack.
// -gcflags="-m": no escape message
func stackReturn() int {
    x := 42
    return x
}

// heapReturn: pointer returned — x escapes to heap.
// -gcflags="-m": "moved to heap: x"
func heapReturn() *int {
    x := 42
    return &x
}

// interfaceEscape: integer stored in interface{} — boxed on heap.
// -gcflags="-m": "42 escapes to heap"
func interfaceEscape() interface{} {
    return 42
}

// closureEscape: captured variable escapes to heap.
// -gcflags="-m": "moved to heap: x"
func closureEscape() func() int {
    x := 42
    return func() int { return x }
}

// sliceGrowth: append may move slice to heap if it grows.
func sliceGrowth() []int {
    s := make([]int, 0, 10)
    for i := 0; i < 100; i++ {
        s = append(s, i) // grows beyond cap=10 → heap
    }
    return s
}

// Run: go build -gcflags="-m" ./...
// Each escape is explained by one of the patterns above.
```

Exercise

Build: Write 5 functions with different escape patterns. Before running `-gcflags="-m"`, predict which variables escape for each function. **Input:** 5 functions: one returning int, one returning *int*, one accepting *interface{}*, one using a closure, one returning a slice from a function **Output:** Your predictions vs the actual `-gcflags="-m"` output for each function **Acceptance:*** Your

predictions are correct for at least 4 of 5. For any wrong prediction, write one sentence explaining why the compiler made a different decision.

Interview

- Why is returning a pointer from a function sufficient to cause a heap allocation?
 - Does every interface conversion cause a heap allocation? Give a counterexample.
 - A goroutine's stack starts at 2KB. What happens when it grows beyond that?
-

Source: `17-go-runtime-internals-foundations-for-profilers/03-gc-basics-under-allocation-pressure.md`

Unit 3 — GC Basics Under Allocation Pressure

Concept

Go's GC is concurrent — it runs alongside your program with short stop-the-world pauses. GC triggers when the heap grows to 2× live data (GOGC=100). `GOMEMLIMIT` (Go 1.19+) sets a hard memory ceiling — when approached, GC runs more frequently regardless of GOGC. High allocation rate = frequent GC = latency spikes.

Code

```
package main

import (
    "fmt"
    "runtime"
)

func printMemStats(label string) {
    var m runtime.MemStats
    runtime.ReadMemStats(&m)
    fmt.Printf("%s: HeapAlloc=%dKB NumGC=%d PauseTotalNs=%dms\n",
        label,
        m.HeapAlloc/1024,
        m.NumGC,
        m.PauseTotalNs/1e6,
    )
}

func allocate(n int) {
    for i := 0; i < n; i++ {
        _ = make([]byte, 1024) // 1KB per iteration
    }
}

func main() {
    printMemStats("before")
    allocate(100_000)
    printMemStats("after")
    runtime.GC()
    printMemStats("after GC")
}

// Experiment with environment variables:
// GOGC=100 (default) - GC when heap doubles
// GOGC=200 - GC less often, more memory used, fewer pauses
// GOGC=off - no GC - maximum throughput, unbounded memory
// GOMEMLIMIT=128MiB - hard ceiling - GC runs aggressively when approaching limit
//
// Run:
// GOGC=off go run main.go
// GOGC=100 go run main.go
// GOGC=off GOMEMLIMIT=64MiB go run main.go
```

Exercise

Build: Write a benchmark that allocates 1KB slices in a tight loop (`b.N` iterations). Run it three ways and record throughput (ns/op). **Input:** `go test -bench=. -count=3` with `GOGC=100`, `GOGC=off`, and `GOGC=off GOMEMLIMIT=64MiB` **Output:** ns/op for each configuration. Which is fastest? What happens with `GOMEMLIMIT` when you allocate more than the limit? **Acceptance:** `GOGC=off` is fastest (no GC pauses). `GOGC=off` with `GOMEMLIMIT` shows GC kicking in once memory approaches the limit, raising ns/op. You can explain why.

Interview

- What does `GOGC=100` mean in plain English?
 - Why might `GOGC=off` in production be dangerous without `GOMEMLIMIT`?
 - What is a stop-the-world GC pause and how long are typical Go GC pauses?
-

Unit 4 — Execution Trace Overview: go tool trace

Concept

Execution trace records goroutine scheduling, GC events, and syscall timings with microsecond precision. It is more detailed than pprof — it shows blocking events and goroutine lifecycle, not just where CPU time goes. It is expensive to collect — use it for specific problem investigations (goroutine starvation, unexpected blocking, GC pause spikes), not as always-on instrumentation.

Code

```
package main

import (
    "os"
    "runtime/trace"
)

func main() {
    f, _ := os.Create("trace.out")
    defer f.Close()

    trace.Start(f)
    defer trace.Stop()

    // Your work here – the trace captures everything that happens.
    doWork()
}

// Open the trace in the browser:
// go tool trace trace.out
//
// Navigate to:
// "Goroutine analysis" – see per-goroutine blocking time
// "View trace"         – timeline of all goroutines, GC events, syscalls
// "Stop-the-world"     – GC pause durations
//
// For HTTP services, collect trace via pprof endpoint:
// curl http://localhost:6060/debug/pprof/trace?seconds=5 > trace.out
// go tool trace trace.out
```

Exercise

Build: Add trace collection to your API service. Send 100 requests while collecting a 5s trace. Open it in the browser. **Input:** 100 HTTP requests against the running service. 5s trace collected via the pprof endpoint. **Output:** Find and record: (1) how long the longest GC stop-the-world pause is, (2) which goroutines block the longest, (3) how many goroutines are active at peak load. **Acceptance:** You can navigate to “Goroutine analysis” and identify the top blocking goroutine. You can find at least one GC event in the timeline view and read its pause duration.

Interview

- What is the difference between a pprof CPU profile and an execution trace?
 - Why is trace collection more expensive than pprof profiling?
 - Name two events visible in an execution trace that are not visible in a CPU profile.
-

Unit 5 — Capstone: Slow Service to Evidence Fix

Concept

Debugging performance = hypothesis → measure → fix → measure again. Never guess. The fix that seems obvious is often not the bottleneck. This capstone gives you a deliberately slow handler with multiple inefficiencies — your job is to find each one with profiling evidence before touching the code.

Code

```
package main

import (
    "encoding/json"
    "net/http"
    "strings"
    _ "net/http/pprof"
)

type Item struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Price int    `json:"price"`
}

// slowHandler has three intentional inefficiencies.
// Your job is to find them with pprof before fixing them.
func slowHandler(w http.ResponseWriter, r *http.Request) {
    // Inefficiency 1: string concatenation in a loop (allocates N times)
    var names string
    for i := 0; i < 100; i++ {
        names += fmt.Sprintf("item-%d,", i)
    }

    // Inefficiency 2: redundant JSON encode (result discarded)
    items := make([]Item, 100)
    for i := range items {
        items[i] = Item{ID: i, Name: fmt.Sprintf("item-%d", i), Price: i * 10}
    }
    _, _ = json.Marshal(items) // encoded but discarded — wasted work

    // Inefficiency 3: allocates new encoder on every request instead of reusing
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(items)
}

func main() {
    go http.ListenAndServe(":6060", nil) // pprof
    http.HandleFunc("/items", slowHandler)
    http.ListenAndServe(":8080", nil)
}
```

Exercise

Build: Use pprof to identify the top 3 bottlenecks in `slowHandler`. Fix each one. Show before/after benchmark numbers. **Input:** `slowHandler` as written above. Load from `hey -n 5000 -c 20 http://localhost:8080/items`. **Output:** For each bottleneck: (1) which pprof frame pointed to it, (2) what the fix is, (3) before/after ns/op and allocs/op from the benchmark. **Acceptance:** After all 3 fixes: allocs/op is reduced by at least 50% and ns/op improves by at least 30% compared to the original. Each fix is backed by a before/after benchmark — no guessing.

Interview

- How do you benchmark an HTTP handler in Go without starting a real server?
 - What does `httptest.NewRecorder()` give you?
 - If a CPU profile shows most time in `runtime.mallocgc`, what does that tell you?
-

Phase — 18-go-runtime-deep-dive-staff-level

Directory: 18-go-runtime-deep-dive-staff-level/

Source: 18-go-runtime-deep-dive-staff-level/00-quick-reference.md

Quick Reference — Go Runtime Deep Dive

sync.Pool usage pattern

```
var pool = sync.Pool{New: func() any { return &MyObj{} }}
obj := pool.Get().(*MyObj)
obj.Reset()           // always reset before use
defer pool.Put(obj)
// WARNING: pool items may be GC'd – don't store state you must keep
```

Escape analysis flags

```
go build -gcflags="-m" .    // shows: "moved to heap: x"
go build -gcflags="-m=2" .  // verbose with reason
```

Memory alignment — field ordering matters

```
// Put largest fields first to minimize padding
type Optimal struct {
    a int64 // 8
    b int64 // 8
    c int32 // 4
    d int16 // 2
    e int8  // 1
    // 1 byte padding to align to 8
}
```

False sharing fix

```
type Counter struct {
    v    int64
    _    [56]byte // pad to cache line boundary (64 bytes total)
}
```

Preemption

```
// Go 1.14+: asynchronous preemption (signal-based)
// Goroutines can be preempted even in tight CPU loops
// Prior to 1.14: only at function call sites
```

GC phases

```
Mark (concurrent) → Mark termination (STW) → Sweep (concurrent)
STW = stop-the-world pause (microseconds in modern Go)
```

Unit 1 — Scheduler: GMP, Preemption, and Syscall Blocking

Concept

Go 1.14+ uses asynchronous preemption — signals interrupt even tight CPU loops, so other goroutines get CPU time without explicit yield points. Syscalls use a handoff protocol: the M parks, the P moves to another M, the syscall completes, the M tries to acquire a P. Network I/O uses the netpoller (epoll/kqueue) — a goroutine parks, the OS notifies the netpoller when the socket is ready, the goroutine is rescheduled.

Code

```
package main

import (
    "fmt"
    "runtime"
    "sync/atomic"
    "time"
)

func main() {
    runtime.GOMAXPROCS(1) // single P: demonstrates preemption

    var counter int64
    done := make(chan struct{})

    // Tight loop — no explicit yield. In Go 1.14+ this goroutine
    // is preempted by signals, allowing other goroutines to run.
    go func() {
        for {
            select {
            case <-done:
                return
            default:
                atomic.AddInt64(&counter, 1)
            }
        }
    }()

    // This goroutine can run even though the above loop never yields.
    // Without async preemption (Go <1.14 with GOMAXPROCS=1) this would starve.
    go func() {
        time.Sleep(100 * time.Millisecond)
        close(done)
    }()

    time.Sleep(200 * time.Millisecond)
    fmt.Printf("counter=%d (both goroutines ran despite tight loop)\n", atomic.LoadInt64(&counter))
}
```

Exercise

Build: Run the program above with `GOMAXPROCS=1`. Add `runtime.Gosched()` inside the tight loop and measure if it makes any difference to scheduling fairness. **Input:** Two goroutines on `GOMAXPROCS=1` — one tight loop, one that sleeps 100ms then signals done **Output:** Confirm the second goroutine's signal was received (the loop stopped). Compare counter values with and without `runtime.Gosched()`. **Acceptance:** The program terminates in ~200ms with `GOMAXPROCS=1`, proving the tight loop goroutine was preempted. Document whether `Gosched()` changes the counter value significantly.

Interview

- What is asynchronous preemption and which Go version introduced it?
- When a goroutine makes a blocking syscall, what happens to its P?
- What is the netpoller and why does it allow goroutines to park without blocking an OS thread?

Source: `18-go-runtime-deep-dive-staff-level/02-escape-analysis-and-allocation-tradeoffs-deep.md`

Unit 2 — Escape Analysis and Allocation Tradeoffs (Deep)

Concept

Interface boxing always allocates when the stored value is a non-pointer type. Closures that capture variables cause those variables to escape to heap. Returning a pointer from a function causes the pointed-to value to escape. In hot paths, replace interface parameters with generics (Go 1.18+) or concrete types to eliminate boxing allocations.

Code

```
package main

// Pattern 1: interface boxing – allocates per call for non-pointer types.
// -gcflags="-m": "42 escapes to heap"
func withInterface(v interface{}) int {
    return v.(int) + 1
}

// Pattern 2: generic function – no boxing, no allocation for value types.
// -gcflags="-m": no escape for the value
func withGeneric[T int | int64 | float64](v T) T {
    return v + 1
}

// Pattern 3: concrete type – fastest, zero allocation.
func withConcrete(v int) int {
    return v + 1
}

// Pattern 4: closure captures x – x escapes to heap.
// -gcflags="-m": "moved to heap: x"
func makeAdder(x int) func(int) int {
    return func(n int) int { return n + x }
}

// Run: go build -gcflags="-m" ./...
// See which patterns cause "escapes to heap" and which do not.
```

Exercise

Build: In a hot path (1M iterations), benchmark three implementations of `add(v int) int`: (a) takes `interface{}`, (b) generic `[T int]`, (c) concrete `int`. Use `-benchmem -count=3`. **Input:** Three benchmark functions each calling their implementation 1M times in `b.N` iterations **Output:** ns/op and allocs/op for all three. Show the allocation difference between (a) and (b)/(c). **Acceptance:** (a) interface version has allocs/op > 0; (b) generic and (c) concrete have allocs/op = 0. You can explain why generics avoid boxing while interfaces do not.

Interview

- Why does storing an `int` in `interface{}` allocate but storing a `*int` does not always?
- What is the cost of interface dispatch compared to a direct function call?
- When would you choose a generic function over an interface in a performance-critical path?

Source: `18-go-runtime-deep-dive-staff-level/03-sync-pool-object-reuse-and-lifecycle-hazards.md`

Unit 3 — sync.Pool: Object Reuse and Lifecycle Hazards

Concept

`sync.Pool` reduces GC pressure by reusing allocated objects. Pool items may be collected any GC cycle — never use Pool for state that must persist across GC. Always `Reset` the object before use (it may contain data from a previous user). Never hold a reference to a pooled object after returning it with `Put`.

Code

```
package main

import (
    "bytes"
    "encoding/json"
    "sync"
)

var bufPool = sync.Pool{
    New: func() interface{} {
        return new(bytes.Buffer)
    },
}

// encode reuses a buffer from the pool instead of allocating per call.
func encode(v interface{}) ([]byte, error) {
    buf := bufPool.Get().(*bytes.Buffer)
    buf.Reset() // must reset — may contain data from previous use
    defer bufPool.Put(buf)

    if err := json.NewEncoder(buf).Encode(v); err != nil {
        return nil, err
    }
    // Copy before returning — buf goes back to pool, caller must not hold it.
    result := make([]byte, buf.Len())
    copy(result, buf.Bytes())
    return result, nil
}

// Hazard: do NOT do this — holding reference after Put.
func hazard(v interface{}) []byte {
    buf := bufPool.Get().(*bytes.Buffer)
    buf.Reset()
    json.NewEncoder(buf).Encode(v)
    bufPool.Put(buf)
    return buf.Bytes() // BUG: buf is back in pool — another goroutine may use it
}
```

Exercise

Build: Benchmark JSON encoding with and without `sync.Pool` for `bytes.Buffer`. Use a struct with 10 fields. Run `-benchmem -count=3`. **Input:** Benchmark both `json.Marshal` (allocates per call) and `encode` using the pool above **Output:** allocs/op for each approach. Show that pool version has fewer allocations. **Acceptance:** Pool version shows fewer allocs/op than `json.Marshal`. Use `-count=3` to verify the improvement is consistent across runs, not just a single lucky sample.

Interview

- When does `sync.Pool` drop its objects?

- Why must you call `Reset()` on an object retrieved from a pool?
- A pool stores database connections. Is this correct use of `sync.Pool`? Why or why not?

Source: [18-go-runtime-deep-dive-staff-level/04-memory-alignment-and-false-sharing.md](#)

Unit 4 — Memory Alignment and False Sharing

Concept

CPUs read and write in cache lines (64 bytes). If two goroutines write different fields that share a cache line, they contend for the same line — this is false sharing, and it serializes what should be parallel work. Fix by padding structs so each hot field occupies its own cache line. Field ordering also affects struct size: put the largest fields first to minimize padding between them.

Code

```
package main

import (
    "fmt"
    "sync"
    "testing"
    "unsafe"
)

// Unpadded: both counters share a cache line — false sharing under parallel writes.
type UnpaddedCounters struct {
    a int64
    b int64
}

// Padded: each counter has its own cache line.
type PaddedCounter struct {
    value    int64
    _        [56]byte // pad to 64 bytes (cache line size)
}

type PaddedCounters struct {
    a PaddedCounter
    b PaddedCounter
}

func benchFalseSharing(b *testing.B) {
    var c UnpaddedCounters
    var wg sync.WaitGroup
    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        wg.Add(2)
        go func() { defer wg.Done(); for j := 0; j < 1000; j++ { c.a++ } }()
        go func() { defer wg.Done(); for j := 0; j < 1000; j++ { c.b++ } }()
        wg.Wait()
    }
}

func main() {
    fmt.Printf("UnpaddedCounters size: %d bytes\n", unsafe.Sizeof(UnpaddedCounters{}))
    fmt.Printf("PaddedCounters size:    %d bytes\n", unsafe.Sizeof(PaddedCounters{}))
    // Run: go test -bench=. -benchmem -count=3
}
```

Exercise

Build: Benchmark a counter array where N goroutines each increment their own counter: (a) `[]int64` (unpadded, likely shares cache lines), (b) `[]PaddedCounter` (each counter on its own cache line). Use N = number of CPUs. **Input:** N goroutines each doing 1M increments on their assigned counter. Two implementations: unpadded array and padded array. **Output:** ns/op for both implementations with `-count=3`. On a multi-core machine, padded version should be faster. **Acceptance:** Padded version is measurably faster on multi-core (2× or more speedup is common). The unpadded version's performance degrades as CPU count increases; padded version scales better.

Interview

- What is a cache line and how large is it on modern x86?
 - Describe false sharing with a concrete example using two goroutines.
 - Why does padding fix false sharing but increase memory usage? When is the tradeoff worth it?
-

Source: `18-go-runtime-deep-dive-staff-level/05-capstone-pprof-microarchitecture-investigation.md`

Unit 5 — Capstone: pprof and Microarchitecture Investigation

Concept

Staff-level performance work combines runtime knowledge with profiling evidence. You need to know what the profile is telling you — “allocations in `sync.Pool.Get`” means your pool objects are being GC'd too fast, not that Pool is broken. Distinguish between allocation pressure, false sharing, and escape analysis issues by reading the profile before reading the code.


```

package main

import (
    "encoding/json"
    "fmt"
    "sync"
    "sync/atomic"
)

// Item has fields in suboptimal order – causes internal padding waste.
type Item struct {
    Active bool    // 1 byte + 7 bytes padding before Price
    Price  float64 // 8 bytes
    ID     int32   // 4 bytes + 4 bytes padding before Name pointer
    Name   string // 16 bytes (pointer + len)
}

// SharedState has two hot fields on the same cache line – false sharing.
type SharedState struct {
    readCount int64 // written by reader goroutines
    writeCount int64 // written by writer goroutines – shares cache line with readCount
}

// processItems has three issues:
// 1. allocates a new encoder per call (escape to heap each time)
// 2. Item struct layout wastes padding bytes
// 3. SharedState.readCount and writeCount share a cache line
func processItems(state *SharedState, items []Item) ([]byte, error) {
    atomic.AddInt64(&state.readCount, 1)

    // Issue 1: new bytes.Buffer per call – should use sync.Pool
    var buf []byte
    for _, item := range items {
        b, err := json.Marshal(item) // allocates per item
        if err != nil {
            return nil, err
        }
        buf = append(buf, b...)
    }
    atomic.AddInt64(&state.writeCount, 1)
    return buf, nil
}

func main() {
    state := &SharedState{}
    items := make([]Item, 100)
    for i := range items {
        items[i] = Item{ID: int32(i), Name: fmt.Sprintf("item-%d", i), Price: float64(i) * 1.5,
Active: true}
    }

    var wg sync.WaitGroup
    for i := 0; i < 8; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for j := 0; j < 10_000; j++ {
                processItems(state, items)
            }
        }()
    }
    wg.Wait()
    fmt.Printf("reads=%d writes=%d\n", state.readCount, state.writeCount)
}

```

Exercise

Build: Profile `processItems` under load. Identify: (1) what escapes to heap unnecessarily, (2) whether `SharedState` has false sharing, (3) the top allocation hot spot. Fix each one. Target: reduce `allocs/op` by 50% and improve throughput. **Input:** 8 goroutines each calling `processItems` 10K times. CPU and heap profile collected via pprof. **Output:** For each of the 3 issues: (a) which profile frame pointed to it, (b) the fix applied, (c) before/after benchmark numbers (ns/op, allocs/op). **Acceptance:** After all fixes: allocs/op reduced by $\geq 50\%$ vs original. Each fix is backed by a before/after benchmark. `SharedState` padded so reads and writes are on separate cache lines. You can explain what `-gcflags="-m"` shows before and after the escape fix.

Interview

- What does pprof output `sync.Pool.Get` in the allocation profile tell you?
 - Describe the steps you take when a service has high p99 latency but normal CPU usage.
 - A colleague claims padding structs is premature optimization. When is it and when is it not?
-

Phase — 19-production-go-observability-and-containers

Directory: 19-production-go-observability-and-containers/

Source: 19-production-go-observability-and-containers/00-quick-reference.md

Quick Reference — Production & Containers

Multi-stage Dockerfile (Go)

```
FROM golang:1.23-alpine AS builder RUN CGO_ENABLED=0 go build -ldflags="-w -s" -o /bin/app ./cmd/app
FROM scratch COPY --from=builder /bin/app /app ENTRYPOINT ["/app"]
```

Build flags

CGO_ENABLED=0 # static binary (required for scratch/distroless) -ldflags="-w -s" # strip debug info and symbol table (smaller binary) GOOS=linux GOARCH=amd64 # cross-compile

Health endpoints

GET /health/live → 200 if process alive (no external checks) GET /health/ready → 200 if dependencies up (db ping, cache ping)

Graceful shutdown

```
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM) <-quit ctx, _ := context.WithTimeout(context.Background(), 30*time.Second)
srv.Shutdown(ctx) // stops accepting, drains in-flight
```

k8s probe config

livenessProbe: httpGet: /health/live, failureThreshold: 3 readinessProbe: httpGet: /health/ready, failureThreshold: 1

Source: 19-production-go-observability-and-containers/01-scope-prod-service-24-by-7-ownership.md

Unit 1 — Scope: Production Service, 24/7 Ownership

Concept

A service running in production needs more than working code. It needs a Docker image that builds deterministically, health probes so Kubernetes knows when to route traffic, graceful shutdown so in-flight requests complete before the process exits, structured logs so operators can search and correlate events, and distributed tracing so you can follow a request across services. This module takes your existing HTTP API and adds each of these layers. By the end, the service is deployable to Kubernetes with the observability stack a real team expects.

Code

```
// Production service checklist – each item maps to a unit in this module.
//
// [ ] Docker image
//   - Multi-stage build: builder (golang:1.23) → runtime (scratch/distroless)
//   - CGO_ENABLED=0 for static binary
//   - Non-root user in final image
//
// [ ] Health probes
//   GET /health/live → 200 always (process is up)
//   GET /health/ready → 200 if deps reachable, 503 if not
//
// [ ] Graceful shutdown
//   - Catch SIGTERM
//   - Stop accepting new connections
//   - Drain in-flight requests (30s budget)
//   - Flush logs, close DB pool
//
// [ ] Structured logging
//   - slog with JSON handler in production
//   - Every request: method, path, status, latency, trace_id
//   - Never log PII
//
// [ ] OTel tracing
//   - initTracer with OTLP exporter
//   - Middleware creates one span per request
//   - Propagate trace context downstream (HTTP headers)
//
// [ ] Kubernetes deployment
//   - resources.requests and resources.limits set
//   - livenessProbe and readinessProbe configured
//   - terminationGracePeriodSeconds >= shutdown budget

package main

import "fmt"

func main() {
    fmt.Println("See units 02-05 for each item above.")
}
```

Exercise

Build: Read through the checklist above and map each item to one of units 02-05 in this module. **Input:** Your existing HTTP API service from a previous module. **Output:** A written list: for each checklist item, which file you will change and what you expect to add. **Acceptance:** You can explain to someone why each item matters in a production incident — what breaks if it is missing.

Interview

- What is the difference between a liveness probe and a readiness probe? What goes wrong if you conflate them?
 - Why does graceful shutdown matter in a rolling deployment?
 - What does “structured logging” give you that plain text logs do not?
-

Unit 2 — Docker: Images and Layers

Concept

A Docker image is a stack of read-only layers. Each `RUN`, `COPY`, and `ADD` instruction creates a new layer on top of the previous one. Docker caches layers — if a layer's instruction and inputs have not changed since the last build, Docker reuses the cached layer and skips re-executing it. This means layer order determines build speed. Put slow, rarely-changing instructions first (installing dependencies) and fast, frequently-changing instructions last (copying your source code). A smaller base image reduces the attack surface and download time.

Code

```
# BAD ORDER — copies code first, so every code change
# invalidates the go mod download layer (slow network step).

FROM golang:1.23
WORKDIR /app
COPY . .                # layer 1: everything — changes every commit
RUN go mod download      # layer 2: re-runs on every code change
RUN go build -o server . # layer 3: rebuild

# ---

# GOOD ORDER — dependency layer is cached until go.mod changes.

FROM golang:1.23
WORKDIR /app
COPY go.mod go.sum ./    # layer 1: only changes when deps change
RUN go mod download      # layer 2: cached until go.mod changes
COPY . .                # layer 3: source code — changes often
RUN go build -o server . # layer 4: rebuild only when code changes

# Base image size comparison (approximate):
#   ubuntu:22.04      ~77 MB
#   debian:bookworm-slim ~74 MB
#   golang:1.23-alpine ~250 MB (includes Go toolchain)
#   alpine:3.19       ~7 MB
#   scratch           0 MB (empty — just your binary)
#   gcr.io/distroless/static ~2 MB (libc, certs, no shell)
```

Exercise

Build: Package your API service as a Docker image using the good-order Dockerfile above. **Input:** Your API service source code with `go.mod` and `go.sum`. **Output:** A built Docker image. Run `docker history <image>` to see the layer breakdown.

Acceptance: Make a trivial code change (add a comment), rebuild, and confirm only the last two layers are rebuilt (`COPY` and `build`). Then swap the `COPY` and `go mod download` order and rebuild — confirm all layers invalidate.

Interview

- Why does Docker layer order affect build time in CI?
 - What is the practical difference between `alpine` and `scratch` as a base image?
 - If you add a secret to a `RUN` layer and then delete it in the next layer, is the secret safe?
-

Unit 3 — Docker Multi-Stage: Minimal Runtime Image

Concept

A multi-stage build uses two FROM statements in one Dockerfile. Stage 1 (builder) has the full Go toolchain and compiles the binary. Stage 2 (runtime) copies only the compiled binary into a minimal base image. The final image contains no Go compiler, no source code, no build tools, and no package manager — just the binary. This reduces image size from ~1 GB to ~10 MB and shrinks the attack surface dramatically. Set `CGO_ENABLED=0` to produce a statically linked binary that runs on `scratch` with no C library.

Code

```
# Stage 1: builder — full Go toolchain, produces static binary
FROM golang:1.23-alpine AS builder

WORKDIR /app

COPY go.mod go.sum ./
RUN go mod download

COPY . .

# CGO_ENABLED=0 → static binary (no libc dependency)
# -ldflags "-s -w" → strip debug symbols (smaller binary)
RUN CGO_ENABLED=0 GOOS=linux go build \
    -ldflags="-s -w" \
    -o server \
    ./cmd/server

# Stage 2: runtime — no compiler, no source, no shell
FROM scratch

# Copy TLS root certificates so HTTPS calls work
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/

# Copy the binary from the builder stage
COPY --from=builder /app/server /server

# Non-root: scratch has no users, so set numeric UID
USER 65534:65534

EXPOSE 8080

ENTRYPOINT ["/server"]

# Image size comparison after this change:
#   Single-stage golang:1.23      ~1.1 GB
#   Multi-stage with alpine base  ~20  MB
#   Multi-stage with scratch base ~8   MB
```

Exercise

Build: Convert your API service Dockerfile to a multi-stage build using `scratch` as the final base. **Input:** Your API service source code. **Output:** A built image. Run `docker images` to see the size. **Acceptance:** The scratch image must be under 20 MB. Run `docker run --rm <image>` and verify the server starts. Confirm `docker exec` into the container fails (no shell in scratch).

Interview

- Why does `CGO_ENABLED=0` matter when using `scratch` as a base?

- What is the security advantage of having no shell in the runtime image?
 - What do you lose by using `scratch` instead of `distroless`? When would you choose `distroless`?
-

Source: `19-production-go-observability-and-containers/04-health-liveness-readiness-endpoints-and-semantics.md`

Unit 4 — Health: Liveness and Readiness Endpoints

Concept

Kubernetes uses two separate health probes with different failure behaviors. Liveness: is the process alive and not deadlocked? A failure triggers a pod restart. Readiness: can the process serve traffic right now? A failure removes the pod from the load balancer without restarting it. These are different signals — do not mix them. Never put a database ping in your liveness probe. A database outage should take your pod out of rotation, not restart it in a loop. Put dependency checks only in readiness.

Code

```
package main

import (
    "context"
    "database/sql"
    "net/http"
    "time"
)

// LiveHandler – liveness probe.
// Only checks that the process is running and not deadlocked.
// Always returns 200. Kubernetes will restart the pod if this fails.
func LiveHandler(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
}

// ReadyHandler – readiness probe.
// Checks that dependencies are reachable.
// Returns 503 if any dependency is unavailable.
// Kubernetes removes the pod from the load balancer if this fails.
func ReadyHandler(db *sql.DB) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
        defer cancel()

        if err := db.PingContext(ctx); err != nil {
            http.Error(w, "db unavailable", http.StatusServiceUnavailable)
            return
        }

        // Add more dependency checks here (Redis, downstream services).
        // Each check must use a short timeout – never block indefinitely.

        w.WriteHeader(http.StatusOK)
    }
}

func main() {
    db, _ := sql.Open("postgres", "postgres://localhost/mydb?sslmode=disable")

    mux := http.NewServeMux()
    mux.HandleFunc("/health/live", LiveHandler)
    mux.HandleFunc("/health/ready", ReadyHandler(db))

    http.ListenAndServe(":8080", mux)
}
```

Exercise

Build: Add `/health/live` and `/health/ready` endpoints to your API service. **Input:** Your service running with a PostgreSQL connection. **Output:** Two endpoints that respond correctly based on dependency state. **Acceptance:** (1) Start your service with the DB stopped — `/health/ready` returns 503, `/health/live` returns 200. (2) Start the DB — `/health/ready` returns 200 within 5 seconds. (3) The readiness check must complete within 2s even when the DB is down (timeout enforced).

Interview

- A database goes down in production. Should your pods restart or just stop receiving traffic? Which probe controls each behavior?
 - What is the risk of a readiness probe with no timeout?
 - At what point during startup should the readiness probe start returning 200?
-

Source: 19-production-go-observability-and-containers/05-graceful-shutdown-signals-and-draining.md

Unit 5 — Graceful Shutdown: Signals and Draining

Concept

When Kubernetes rolls out a new version, it sends `SIGTERM` to the old pod. Your process must stop accepting new connections, finish all in-flight requests, flush logs, and close the database pool — then exit cleanly. If it takes too long, Kubernetes sends `SIGKILL` after `terminationGracePeriodSeconds` (default 30s), which kills the process immediately regardless of in-flight work. `SIGKILL` cannot be caught. Your shutdown budget must be shorter than `terminationGracePeriodSeconds` so you always exit before the hard kill.

Code

```
package main

import (
    "context"
    "log/slog"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"
)

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        // Simulate slow request – in-flight work that must complete.
        time.Sleep(3 * time.Second)
        w.WriteHeader(http.StatusOK)
    })

    server := &http.Server{
        Addr:    ":8080",
        Handler: mux,
    }

    // Start server in background.
    go func() {
        if err := server.ListenAndServe(); err != http.ErrServerClosed {
            slog.Error("server error", "err", err)
            os.Exit(1)
        }
    }()

    slog.Info("server started", "addr", ":8080")

    // Block until SIGTERM or SIGINT arrives.
    quit := make(chan os.Signal, 1)
    signal.Notify(quit, syscall.SIGTERM, syscall.SIGINT)
    <-quit

    slog.Info("shutdown signal received – draining connections")

    // Give in-flight requests 30s to complete.
    // After 30s, Shutdown returns and we exit.
    ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
    defer cancel()

    if err := server.Shutdown(ctx); err != nil {
        slog.Error("shutdown error", "err", err)
        os.Exit(1)
    }

    // Close DB pool, flush log buffers, etc.
    slog.Info("shutdown complete")
}
```

Exercise

Build: Add graceful shutdown to your API service using the pattern above. **Input:** Your running service. **Output:** On `SIGTERM`, the service completes all in-flight requests before exiting. **Acceptance:** Start your service. Send 10 concurrent requests that each take 3 seconds (use `hey -c 10 -n 10 -t 10`). While they are running, send `kill -SIGTERM <pid>`. Verify: (1) no new

requests are accepted after the signal, (2) all 10 in-flight requests return 200, (3) the process exits after the last request completes — not before.

Interview

- What is the difference between `SIGTERM` and `SIGKILL`?
 - If `terminationGracePeriodSeconds` is 30s and your shutdown budget is also 30s, what can go wrong?
 - How does `http.Server.Shutdown` differ from `http.Server.Close`?
-

Source: 19-production-go-observability-and-containers/06-structured-logging-slog-and-correlation-fields.md

Unit 6 — Structured logging in production (`slog` baseline)

Replace ad-hoc prints with **keyed fields** and stable event names.

Minimum fields to internalise:

```
request_id
trace_id (or trace/span IDs once OTEL hooked)
service name / version
error classification fields (code, retryable bool) optional but valuable
```

Practice

Inject **request_id** at edge middleware; propagate into logs for a synthetic `Payment` API path.

Simulate a **500** and verify you can trace the request through logs without guessing.

Interview prompts

PII in logs; cardinality of dynamic keys; sampling vs full fidelity.

Source: 19-production-go-observability-and-containers/07-prometheus-metrics-red-and-service-sli-starter.md

Unit 7 — Metrics with Prometheus conventions (RED basics)

Introduce counters/histograms summarising service behaviour—classical **RED** starter:

```
Rate (request rate)
Errors (error rate)
Duration (latency distribution)
```

Instrument `prod-service/` minimally:

```
http_requests_total{route,method,status}
http_request_duration_seconds_bucket (histogram)
optional business counters (payments_attempted_total, etc.)
```

Lab

Explain why **histogram buckets** must be chosen consciously (not infinite labels).

Interview prompts

High-cardinality label disasters; metric naming consistency; SLO thinking at high level.

Follow-on

Dashboard variables, alerting literacy, and false-positive hygiene for these same series are covered in **Unit 14**.

Source: `19-production-go-observability-and-containers/08-opentelemetry-tracing-spans-and-propagation.md`

Unit 8 — OpenTelemetry tracing across multi-hop flows

Trace a request across:

```
api → payment → inventory → queue publish (or gRPC hop)
```

Minimum concepts:

```
trace id propagation
span parent/child relationships
baggage cautions (don't smuggle secrets)
```

Practice

Add OTEL SDK wiring (HTTP server + outbound client spans). Verify a trace stitches IDs you already log (Unit 6).

Interview prompts

Sampling head vs tail; trace volume cost; debugging with traces vs logs vs metrics—when each wins.

Source: `19-production-go-observability-and-containers/09-kubernetes-backend-mindset-pods-deployments-services.md`

Unit 9 — Kubernetes backend mindset (not admin exam)

You are not aiming to be the cluster SRE on day one—you need **enough vocabulary to reason about how your Go binary lives**.

Core nouns:

```
Pod
Deployment (rollout/replicas)
Service (cluster networking abstraction)
ConfigMap / Secret (config & sensitive material injection—still “secrets in etcd” caveats)
```

Practice (paper acceptable)

Sketch: `api pod(s) → Service → dependency database` (managed or not) showing where readiness gates traffic.

Interview prompts

Why Deployments create new ReplicaSets; what happens during rollouts to in-flight connections (high level).

Source: 19-production-go-observability-and-containers/10-configuration-and-secret-injection-ownership.md

Unit 10 — Configuration & secret ownership (no hard-coded production creds)

Anti-pattern: `DB_PASSWORD=admin` in source.

Preferred patterns (choose what matches your org, but understand trade-offs):

```
environment variables injected at runtime
mounted files / secret stores
12-factor style separation of config from code
```

Practice

Create `config/` package reading env with validation (fail fast on boot if misconfigured).

Interview prompts

Secret rotation; least privilege for reading secrets; avoiding logging config dumps.

Source: 19-production-go-observability-and-containers/11-incident-triage-metrics-traces-logs-triangle.md

Unit 11 — Incident triage: tie metrics, traces, and logs into one story

Synthetic incident:

```
CPU rising OR error rate rising OR latency rising (pick combo)
```

Your job is to narrate an investigation path:

```
Confirm user-visible symptom (dashboards / alerts)
Form hypotheses (dependency? GC? lock contention? retry storm? DB saturation?)
Validate with metrics first, then traces, then targeted logs
Apply fix with explicit trade-off note
```

Deliverable

One-page **post-incident style note** (even fictional) with timeline + evidence links you would have collected.

Source: 19-production-go-observability-and-containers/12-production-multi-surface-failure-lab.md

Unit 12 — Production failure lab: break dependencies and prove observability pays off

Deliberately degrade:

```
database latency / errors
queue publish failures
worker stalls
aggressive client timeouts causing retry amplification
```

You must **detect** each class via your metrics/traces/logs story from Unit 11, not guess.

Rules

Each failure gets:

```
symptom → signal(s) used → root cause narrative → mitigation → prevention idea
```

No fix without evidence screenshots or metric panel descriptions—even textual “what I’d look at in Grafana/Jaeger” counts for learning artefacts now.

Source: 19-production-go-observability-and-containers/13-capstone-payment-platform-production-shape.md

Unit 13 — Capstone: `payment-platform/` as a production-shaped mini system

Compose (even partially stubbed honesty permitted if documented boundaries) `payment-platform/` with coherent services skeleton:

```
api
payment
inventory
worker (+ queue bridging earlier distributed/event areas lightly)
```

Must embed production primitives

Docker (multi-stage by default unless justified)

Metrics (Prometheus style)

Tracing (OpenTelemetry end-to-end at least api→one hop)

Structured logging correlation

`/health` + `/ready` split policy

Graceful shutdown

Synthetic chaos again (timeouts/failures) with evidence-driven fixes echoing Units 11–12

Interview checklist consolidation:

```
Docker multi-stage
graceful shutdown
metrics
tracing (spans propagation)
structured logging correlation ids
health vs readiness distinctions
OpenTelemetry vocabulary vs hand-wavy “we have traces” fluff debunk ethically
```

Source:

19-production-go-observability-and-containers/14-grafana-dashboards-variables-and-alerting-from-prometheus.md

Unit 14 — Grafana: turn Prometheus RED metrics into panels, variables, and honest alerts

Builds on **Unit 7 (Prometheus RED)**—same series should appear in Grafana as reusable operational views rather than anonymous scrape targets.

Learning outcomes

- Create a **small dashboard** with variables (`job` , `route` , or `instance`) so one board scales without copy-paste explosion.
- Choose **panel types** matching signal semantics (rates from counters, histograms as heatmaps/latency views where appropriate)—avoid lying “graphs of cumulative counters labelled as latency”.
- Encode **recording rules awareness** vs **heavy PromQL inside Grafana** trade-offs—when to simplify queries upstream.
- Author **two or three alert rules** (Prometheus alerting or Grafana-managed—pick one toolchain and justify) distinguishing **latency**, **error budget burn**, **absence of traffic** pitfalls.

Lab

Reuse metrics from `prod-service/` instrumentation from Unit7 (or equivalently enumerated series). Produce:

one dashboard JSON export OR screenshot-linked narrative
alert definitions with explicit threshold rationale (not tribal “greater than 5%” cargo cult)
written note describing one false-positive you expect and how you'd tune observability—not silence permanently

Interview prompts

- **Cardinality dashboards:** how exploding label sets break cost and cognition.
- **Alert fatigue:** paging vs ticketing vs dashboards-only—explicit policy you'd propose for a hypothetical on-call rotation.

Acceptance criteria

One-page appendix describing **exactly how** you'd verify an alert firing was **correct** versus noise (which queries, what control chart or baseline idea—high level suffices).

Phase — 20-kubernetes-backend-advanced-placement-and-ingress

Directory: `20-kubernetes-backend-advanced-placement-and-ingress/`

Source: `20-kubernetes-backend-advanced-placement-and-ingress/00-quick-reference.md`

Quick Reference — Kubernetes for Go Backends

Resource sizing

requests: scheduler placement (always set) limits: safety ceiling (OOM if memory exceeded, throttled if CPU exceeded)

Typical Go service starting point: requests: cpu: 50m, memory: 64Mi limits: cpu: 500m, memory: 256Mi

HPA triggers

CPU utilization (most common) Custom metrics via Prometheus adapter (RPS, queue depth)

Rolling update (zero downtime)

maxSurge: 1, maxUnavailable: 0 + readinessProbe must pass before old pods are removed

Key kubectl commands

kubectl rollout status deployment/name kubectl rollout undo deployment/name kubectl rollout history deployment/name kubectl top pods # requires metrics-server kubectl describe pod # events, resource usage kubectl exec -it -- /bin/sh # debug (if shell exists)

Pod lifecycle hooks

terminationGracePeriodSeconds: 30 # time for graceful shutdown preStop hook: sleep briefly to let LB drain connections before SIGTERM

Source: `20-kubernetes-backend-advanced-placement-and-ingress/01-hpa-and-autoscaling-metrics-basics.md`

Unit 1 — HPA and Autoscaling: Metrics Basics

Concept

A HorizontalPodAutoscaler watches a metric — typically CPU utilization — and adjusts the number of replicas in a Deployment to keep the metric near a target. When average CPU across all pods exceeds 70%, HPA adds replicas. When load drops, HPA removes replicas, but gradually to avoid flapping. HPA requires `metrics-server` to be installed in the cluster. Scale-down has a stabilization window (default 5 minutes) so a brief traffic drop does not immediately shrink your fleet.

Code

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-server
spec:
  replicas: 2          # HPA overrides this once active
  selector:
    matchLabels:
      app: api-server
  template:
    metadata:
      labels:
        app: api-server
    spec:
      containers:
      - name: api-server
        image: myregistry/api-server:latest
        resources:
          requests:
            cpu: "250m"    # HPA uses this as the baseline for utilization %
          limits:
            cpu: "500m"
---
# hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-server-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-server
  minReplicas: 2        # never scale below 2 (availability floor)
  maxReplicas: 10       # never scale above 10 (cost ceiling)
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70  # scale up when avg CPU > 70% of requests
```

Exercise

Build: Deploy your API service with the HPA configuration above to a local Kubernetes cluster (minikube or kind). **Input:** A running Deployment with CPU requests set and `metrics-server` installed. **Output:** HPA that scales replicas based on CPU load. **Acceptance:** Run `hey -z 60s -c 50 http://localhost:8080/` to generate load. Run `kubectl get hpa -w` in another terminal. Observe `REPLICAS` count increase beyond 2. Stop the load. Wait 5 minutes and observe scale-down back to 2.

Interview

- Why must `resources.requests.cpu` be set for HPA to work?
 - What is the risk of setting `minReplicas: 1` in production?
 - Why does scale-down have a stabilization window but scale-up does not?
-

Unit 2 — Resource Requests, Limits, and Pod Eviction

Concept

`requests` is what Kubernetes uses to schedule a pod onto a node — it is the guaranteed allocation. `limits` is the hard ceiling the container cannot exceed. For CPU, exceeding the limit causes throttling. For memory, exceeding the limit causes an OOM kill (the container is killed and restarted). Set `requests` to what your service needs at normal load. Set `limits` at 2-3x requests to absorb spikes without evicting the pod. If a node runs out of memory, Kubernetes evicts pods with no requests set (BestEffort) first, then pods where usage exceeds requests (Burstable), then pods where usage is within requests (Guaranteed).

Code

```
# Good resource configuration for an HTTP API service.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-server
spec:
  template:
    spec:
      containers:
      - name: api-server
        image: myregistry/api-server:latest
        resources:
          requests:
            cpu: "250m"      # 0.25 CPU cores — scheduler guarantee
            memory: "128Mi"  # 128 MB — scheduler guarantee
          limits:
            cpu: "500m"      # 0.5 CPU cores — throttle at this ceiling
            memory: "256Mi"  # 256 MB — OOM kill above this

# QoS classes (set automatically by Kubernetes):
#
#   Guaranteed:  requests == limits for all containers
#               → safest, last to be evicted
#
#   Burstable:   requests < limits (our config above)
#               → evicted if node pressure AND usage > requests
#
#   BestEffort:  no requests or limits set
#               → first to be evicted, avoid in production
```

Exercise

Build: Deploy your API service with memory limit set to `64Mi`. **Input:** An endpoint that allocates 128 MB of memory (e.g., `make([]byte, 128*1024*1024)`). **Output:** OOM kill event visible in pod description. **Acceptance:** Hit the endpoint. Run `kubectl describe pod <pod-name>` and find `OOMKilled` in the last state. Fix: either raise the limit to `256Mi` or reduce the allocation. Verify the pod no longer restarts after the fix.

Interview

- What is the difference between CPU throttling and memory OOM kill?
 - A pod is scheduled onto a node but the node runs out of memory. Which pods are evicted first?
 - Why is it dangerous to set no resource limits in production?
-

Unit 3 — Rolling Deployments and Rollbacks

Concept

A rolling update replaces old pods with new ones gradually. `maxUnavailable: 0` means Kubernetes waits for the new pod to pass its readiness probe before terminating an old pod — zero downtime. `maxSurge: 1` allows one extra pod to exist during the transition so replacement can proceed without waiting for an old pod to die first. If the new version is broken, `kubectl rollout undo` rolls back to the previous ReplicaSet immediately. Kubernetes keeps the previous ReplicaSet around for exactly this purpose.

Code

```
# deployment.yaml — rolling update strategy
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-server
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0 # never kill an old pod before new pod is ready
      maxSurge: 1       # allow 1 extra pod during the transition (4 total)
  template:
    spec:
      containers:
      - name: api-server
        image: myregistry/api-server:v2
        readinessProbe:
          httpGet:
            path: /health/ready
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 5
          failureThreshold: 3
```

```
# Deploy v2
kubectl set image deployment/api-server api-server=myregistry/api-server:v2

# Watch rollout progress
kubectl rollout status deployment/api-server

# v2 is broken — roll back to v1 immediately
kubectl rollout undo deployment/api-server

# Verify rollback
kubectl rollout status deployment/api-server
kubectl get pods -l app=api-server
```

Exercise

Build: Deploy v1 of your API service. Then deploy v2 with a deliberate bug (handler always returns 500). **Input:** Two image tags: `api-server:v1` (working) and `api-server:v2` (broken). **Output:** Successful rollback to v1 using `kubectl rollout undo`. **Acceptance:** (1) After v2 deploy, hit the endpoint and confirm 500s. (2) Run `kubectl rollout undo`. (3) Hit the endpoint again and confirm 200s. (4) Time the rollback — it should complete in under 60 seconds with 3 replicas.

Interview

- With `maxUnavailable: 0` and `maxSurge: 1`, what is the maximum number of pods running at any point during a rollout of a 3-replica deployment?
 - What does Kubernetes use to decide whether a new pod is ready to receive traffic during a rolling update?
 - How many previous ReplicaSets does Kubernetes keep by default? What controls this?
-

Source: `20-kubernetes-backend-advanced-placement-and-ingress/04-ingress-edge-routing-overview.md`

Unit 4 — Ingress: Edge Routing Overview

Concept

An Ingress resource defines routing rules for external HTTP and HTTPS traffic into the cluster. An Ingress controller (nginx, Traefik, or a cloud provider's implementation) reads these rules and configures its proxy accordingly. TLS is terminated at the Ingress — the traffic between the Ingress controller and your services travels as plain HTTP inside the cluster. Path-based routing lets you direct `/api/v1` to one service and `/static` to another, all behind a single external IP address.

Code

```
# ingress.yaml - TLS termination + path-based routing
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - api.example.com
      secretName: api-tls-secret # kubectl create secret tls api-tls-secret ...
  rules:
  - host: api.example.com
    http:
      paths:
      - path: /api/v1
        pathType: Prefix
        backend:
          service:
            name: api-service # ClusterIP Service for the API
            port:
              number: 8080
      - path: /health
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 8080
  ---
# The backing Service (ClusterIP - internal only)
apiVersion: v1
kind: Service
metadata:
  name: api-service
spec:
  selector:
    app: api-server
  ports:
  - port: 8080
    targetPort: 8080
```

Exercise

Build: Configure Ingress for your API service in a local cluster with the nginx Ingress controller. **Input:** Your deployed API service with a ClusterIP Service. **Output:** Ingress routing `/api/v1` to your service, accessible via `curl` through the Ingress IP.

Acceptance: (1) `curl http://<ingress-ip>/api/v1/products` returns 200. (2) Generate a self-signed cert and configure TLS — `curl -k https://api.example.com/api/v1/products` returns 200. (3) A path not listed in the rules (e.g., `/unknown`) returns 404 from the Ingress.

Interview

- Where does TLS termination happen in this architecture? What are the security implications?
 - What is the difference between a Service of type `ClusterIP`, `NodePort`, and `LoadBalancer`?
 - Why do most clusters run only one Ingress controller even if they have many Ingress resources?
-

Unit 5 — Service Mesh: Conceptual Tradeoffs

Concept

A service mesh injects a sidecar proxy (Envoy) into every pod. All inter-service traffic flows through these proxies, which gives you mutual TLS between services, fine-grained traffic control (retries, circuit breaking, canary weights), and consistent observability (metrics, traces) without changing application code. The cost is real: each pod runs an extra container consuming ~50-100 MB RAM and adding 1-5 ms of latency per hop. The operational complexity of Istio or Linkerd is significant. Add a mesh when you have a concrete, unsolved problem — not because it is fashionable.

Code

Feature comparison: with mesh vs without

FEATURE	WITHOUT MESH	WITH MESH (Istio/Linkerd)
mTLS between services	Manual cert rotation	Automatic, transparent
Retries / timeouts	Code in each service	VirtualService YAML
Circuit breaking	Library (hystrix/etc.)	DestinationRule YAML
Traffic splitting	Deploy new version	Weight: 90/10 in YAML
Per-service metrics	Instrument each service	Automatic from Envoy
Distributed tracing	Propagate headers in code	Partial auto + headers
Latency overhead	0 ms	1-5 ms per hop
RAM overhead per pod	0 MB	50-100 MB (sidecar)
Operational complexity	Low	High (CRDs, upgrades)
Debug difficulty	Standard Go tools	Must understand Envoy

When to add a mesh:

- You have 20+ services and mTLS is a compliance requirement
- You need canary deployments without code changes
- You need uniform circuit breaking across a large fleet

When NOT to add a mesh:

- You have < 10 services
- Your team is not yet operating Kubernetes confidently
- You can solve the problem (retries, mTLS) at the application layer

Exercise

Build: Audit your e-commerce service against the feature table above. **Input:** The list of services in your system (API, auth, inventory, payments). **Output:** A written decision: would you add a service mesh? For which specific feature? **Acceptance:** For each mesh feature, answer: (1) Do I need this? (2) Can I implement it at the application layer instead? (3) What is the cost of the application-layer solution vs the mesh solution? Your answer should conclude with a concrete threshold: “we would add a mesh when X.”

Interview

- What problem does mutual TLS (mTLS) solve that regular TLS does not?
- A team says “we need Istio for observability.” What would you suggest instead?
- Service mesh adds latency. Under what traffic pattern does 5 ms per hop become significant?

Source:

20-kubernetes-backend-advanced-placement-and-ingress/06-capstone-api-deployment-service-ingress-whiteboard.md

Unit 6 — Capstone sketch: `api → Deployment → Service → Ingress`

Deliver a single cohesive diagram + short narrative describing:

how users reach your API
how rolling updates interact with readiness probes
where timeouts are configured (ingress vs service vs app client)
how you would surface golden signals (latency/errors/saturation) for the API's SLO story (even if panels are imaginary)

Treat this as interview whiteboard rehearsal, not a Helm chart competition.

Phase — 21-production-hardening-integration-and-rollout-discipline

Directory: 21-production-hardening-integration-and-rollout-discipline/

Source: 21-production-hardening-integration-and-rollout-discipline/00-quick-reference.md

Quick Reference — Production Hardening

OTel tracer setup

```
tracer := otel.Tracer("service-name") ctx, span := tracer.Start(ctx, "operation-name") defer span.End()
span.SetAttributes(attribute.String("key", "value")) span.RecordError(err) span.SetStatus(codes.Error, err.Error())
```

Structured logging (slog, Go 1.21+)

```
slog.Info("event", "key", value, "key2", value2) slog.Error("failed", "error", err, "order_id", id) // JSON output in prod:
slog.NewJSONHandler(os.Stdout, nil)
```

Config pattern (env-based, 12-factor)

```
type Config struct { DatabaseURL string env:"DATABASE_URL,required" Port int env:"PORT" envDefault:"8080" } // Parse:
github.com/caarlos0/env/v11
```

govulncheck

```
go install golang.org/x/vuln/cmd/govulncheck@latest govulncheck ./... # check for known vulns in deps
```

Probe contracts

livenessProbe: only fails if process is stuck (not if DB is down) readinessProbe: fails if any critical dependency is unavailable
startupProbe: replaces liveness during slow startup

Source: 21-production-hardening-integration-and-rollout-discipline/01-probes-grace-periods-and-obs-contracts-pressure-test.md

Unit 1 — Probes, Grace Periods, and Observability Contracts

Concept

Kubernetes makes a contract with your service: readiness probe passes before any traffic arrives, liveness probe passes or the pod is restarted, and `terminationGracePeriodSeconds` gives you a window to drain before the hard kill. If your grace period is shorter than your longest in-flight request, Kubernetes will kill mid-flight requests during every rolling deploy. The shutdown budget in your Go code must be strictly less than `terminationGracePeriodSeconds` — set the Go shutdown timeout to 25s and `terminationGracePeriodSeconds` to 35s.

Code

```
# deployment.yaml – full probe and grace period configuration
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-server
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  template:
    spec:
      # Grace period must exceed your Go shutdown timeout.
      # Go code: context.WithTimeout(ctx, 25*time.Second)
      # k8s kills: after 35s
      terminationGracePeriodSeconds: 35
      containers:
      - name: api-server
        image: myregistry/api-server:latest
        resources:
          requests:
            cpu: "250m"
            memory: "128Mi"
          limits:
            cpu: "500m"
            memory: "256Mi"

      # Liveness: restart pod if deadlocked.
      # Use a very light check – just HTTP 200.
      livenessProbe:
        httpGet:
          path: /health/live
          port: 8080
        initialDelaySeconds: 10    # wait for startup
        periodSeconds: 10
        failureThreshold: 3       # 3 failures = restart

      # Readiness: stop sending traffic if deps are down.
      # DB ping here is correct – not in liveness.
      readinessProbe:
        httpGet:
          path: /health/ready
          port: 8080
        initialDelaySeconds: 5
        periodSeconds: 5
        failureThreshold: 2       # 2 failures = remove from LB
        successThreshold: 1       # 1 success = back in rotation
```

Exercise

Build: Set `terminationGracePeriodSeconds: 5` deliberately too low, then observe the failure. **Input:** Your API service with requests that take 10 seconds to complete. **Output:** In-flight requests killed mid-flight during a rolling deploy. **Acceptance:** (1) Deploy with grace period 5s. Start 10 slow requests. Trigger a rolling update. Observe connection resets or 502s. (2) Fix: set grace period to 35s and Go shutdown timeout to 25s. Repeat the test. All 10 requests complete with 200.

Interview

- Your longest request takes 20 seconds. What should `terminationGracePeriodSeconds` be, and why not exactly 20?
- A readiness probe starts failing in production. What does Kubernetes do? What does it NOT do?

- What is `initialDelaySeconds` for, and what happens if it is set too low?
-

Source: `21-production-hardening-integration-and-rollout-discipline/02-otel-metrics-tracing-correlation-policy.md`

Unit 2 — OTel: Metrics, Tracing, and Correlation

Concept

OpenTelemetry provides vendor-neutral tracing, metrics, and logs with a single instrumentation API. A span is a named, timed unit of work. A trace is a tree of spans across services connected by a trace ID. Each service propagates the trace context via HTTP headers (`traceparent`), so you can follow a single request from the load balancer through the API handler into the database query. The OTLP exporter sends telemetry to a collector — Jaeger for traces, Prometheus for metrics.


```

package main

import (
    "context"
    "net/http"

    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/exporters/otlp/otlptrace/otlptracehttp"
    "go.opentelemetry.io/otel/propagation"
    "go.opentelemetry.io/otel/sdk/resource"
    sdktrace "go.opentelemetry.io/otel/sdk/trace"
    semconv "go.opentelemetry.io/otel/semconv/v1.21.0"
    "go.opentelemetry.io/otel/trace"
)

// initTracer configures the global tracer. Call at startup.
// Returns a shutdown function – call it on graceful shutdown.
func initTracer(ctx context.Context) (func(context.Context) error, error) {
    exporter, err := otlptracehttp.New(ctx,
        otlptracehttp.WithEndpoint("localhost:4318"),
        otlptracehttp.WithInsecure(),
    )
    if err != nil {
        return nil, err
    }

    tp := sdktrace.NewTracerProvider(
        sdktrace.WithBatcher(exporter),
        sdktrace.WithResource(resource.NewWithAttributes(
            semconv.SchemaURL,
            semconv.ServiceName("api-server"),
        )),
    )

    otel.SetTracerProvider(tp)
    otel.SetTextMapPropagator(propagation.TraceContext{})

    return tp.Shutdown, nil
}

var tracer = otel.Tracer("api-server")

// TracingMiddleware creates a span for every HTTP request.
// Extracts upstream trace context from headers if present.
func TracingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        ctx := otel.GetTextMapPropagator().Extract(r.Context(),
            propagation.HeaderCarrier(r.Header))

        ctx, span := tracer.Start(ctx, r.Method+" "+r.URL.Path,
            trace.WithSpanKind(trace.SpanKindServer))
        defer span.End()

        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

// In your business logic – create a child span for DB calls.
func getProduct(ctx context.Context, id string) error {
    _, span := tracer.Start(ctx, "db.getProduct")
    defer span.End()
    // ... DB query
    return nil
}

```

Exercise

Build: Add OTEL tracing to your API service. Run Jaeger locally with Docker. **Input:** Your service with `initTracer`, `TracingMiddleware`, and at least one child span in a DB call. **Output:** Traces visible in Jaeger UI at `http://localhost:16686`. **Acceptance:** Make 5 requests to your API. In Jaeger, find the traces for your service. Each trace must show: (1) an HTTP server span, (2) a DB query child span, (3) the trace ID is the same across both spans. Verify that requests to different endpoints produce separate traces.

Interview

- What is the difference between a trace and a span?
 - What HTTP header does the W3C Trace Context standard use to propagate trace IDs?
 - Why should you use the OTLP exporter rather than exporting directly to Jaeger?
-

Source: `21-production-hardening-integration-and-rollout-discipline/03-config-logging-and-feature-flag-discipline.md`

Unit 3 — Config, Logging, and Feature Flag Discipline

Concept

12-factor app config: all configuration comes from environment variables, never hardcoded. Validate all required env vars at startup and fail fast with a clear error if any are missing — a misconfigured service that starts silently is harder to debug than one that refuses to start. `slog` (Go 1.21+) is the standard structured logger: use JSON output in production so logs are machine-readable, text in development so they are human-readable. Never log passwords, tokens, credit card numbers, or any PII — structured logging makes it easy to accidentally include fields that contain sensitive data.


```

package main

import (
    "fmt"
    "log/slog"
    "net/http"
    "os"
    "time"
)

// Config holds all service configuration from environment variables.
// Fail fast: missing required vars cause os.Exit at startup.
type Config struct {
    Port          string
    DatabaseURL    string
    RedisURL       string
    LogLevel       string
    Environment    string // "production" | "development"
}

func loadConfig() Config {
    required := map[string]string{
        "PORT":          os.Getenv("PORT"),
        "DATABASE_URL": os.Getenv("DATABASE_URL"),
    }
    for key, val := range required {
        if val == "" {
            fmt.Fprintf(os.Stderr, "missing required env var: %s\n", key)
            os.Exit(1)
        }
    }
    env := os.Getenv("ENVIRONMENT")
    if env == "" {
        env = "development"
    }
    return Config{
        Port:          required["PORT"],
        DatabaseURL:    required["DATABASE_URL"],
        RedisURL:       os.Getenv("REDIS_URL"), // optional
        Environment:    env,
    }
}

// initLogger sets up slog: JSON in production, text in development.
func initLogger(env string) {
    var handler slog.Handler
    if env == "production" {
        handler = slog.NewJSONHandler(os.Stdout, nil)
    } else {
        handler = slog.NewTextHandler(os.Stdout, nil)
    }
    slog.SetDefault(slog.New(handler))
}

// LoggingMiddleware logs each request with method, path, status, and latency.
func LoggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        rw := &responseWriter{ResponseWriter: w, status: 200}
        next.ServeHTTP(rw, r)
        slog.Info("request",
            "method", r.Method,
            "path", r.URL.Path,
            "status", rw.status,
            "latency_ms", time.Since(start).Milliseconds(),
            // Do NOT add: r.Header.Get("Authorization"), user email, etc.
        )
    })
}

```

```

    )
    })
}

type responseWriter struct {
    http.ResponseWriter
    status int
}

func (rw *responseWriter) WriteHeader(status int) {
    rw.status = status
    rw.ResponseWriter.WriteHeader(status)
}

```

Exercise

Build: Move all hardcoded values in your service to environment variables. Add `LoggingMiddleware`. **Input:** Your service with at least `PORT` and `DATABASE_URL` as required env vars. **Output:** Service that fails fast on missing config, with JSON request logs in production mode. **Acceptance:** (1) Start without `PORT` set — see a clear error and exit code 1. (2) Start with `ENVIRONMENT=production` — logs are JSON. (3) Start with `ENVIRONMENT=development` — logs are human-readable text. (4) Make 3 requests — each produces one log line with method, path, status, and latency.

Interview

- Why should a service fail at startup rather than at first request when a required config value is missing?
 - What is the risk of logging `r.Header` directly in your middleware?
 - What makes `slog` better than `log.Printf` for production services?
-

Source:

21-production-hardening-integration-and-rollout-discipline/04-capstone-http-redis-postgres-grpc-obs-rollout.md

Unit 4 — Capstone: HTTP + Redis + Postgres + gRPC + Observability + Rollout

Concept

A production-ready service is not one feature — it is the combination of all concerns working together without conflict. Graceful shutdown must flush the OTel exporter before closing. The readiness probe must check all dependencies the service actively uses. Resource limits must be set before HPA is configured. Structured logs must include the trace ID so you can correlate a log line with the trace in Jaeger. This capstone wires everything together.

Code

```
// Production readiness checklist – verify each item before calling a service production-ready.
//
// BINARY AND IMAGE
// [ ] Multi-stage Dockerfile, scratch or distroless final base
// [ ] CGO_ENABLED=0, static binary
// [ ] Image size < 20 MB
// [ ] Non-root user in container
//
// HEALTH
// [ ] GET /health/live returns 200 always
// [ ] GET /health/ready returns 503 when DB or cache unavailable
// [ ] Readiness check uses 2s timeout per dependency
//
// SHUTDOWN
// [ ] Catches SIGTERM via signal.Notify
// [ ] http.Server.Shutdown called with context
// [ ] Shutdown budget (25s) < terminationGracePeriodSeconds (35s)
// [ ] OTel provider flushed in shutdown sequence
// [ ] DB pool closed after server.Shutdown returns
//
// OBSERVABILITY
// [ ] slog with JSON handler in production
// [ ] Every request logged: method, path, status, latency_ms, trace_id
// [ ] OTel tracing: one span per request, child spans for DB and Redis calls
// [ ] trace_id injected into log fields from span context
//
// KUBERNETES
// [ ] resources.requests and resources.limits set on all containers
// [ ] livenessProbe configured (path, initialDelay, period, failureThreshold)
// [ ] readinessProbe configured (path, initialDelay, period, failureThreshold)
// [ ] terminationGracePeriodSeconds set
// [ ] HPA configured with minReplicas >= 2
// [ ] Ingress configured with TLS
//
// CONFIG
// [ ] All config from env vars
// [ ] Missing required vars cause fail-fast exit at startup
// [ ] No secrets in code or Docker image layers
```

Exercise

Build: Deploy your complete e-commerce API service to a local Kubernetes cluster (minikube or kind) with all items on the checklist above. **Input:** Your API service with PostgreSQL, Redis, and at least two endpoints. **Output:** A running service in Kubernetes with Jaeger for traces and structured logs. **Acceptance:** (1) Run `hey -z 30s -c 20 http://<ingress-ip>/api/v1/products`. (2) Open Jaeger UI — traces must appear for each request with DB and Redis spans. (3) Check pod logs — each log line must be JSON with a `trace_id` field. (4) Run `kubectl get hpa -w` — observe scale-up under load. (5) Trigger a rolling update — zero 5xx errors during the rollout.

Interview

- During shutdown, in what order should you close resources: HTTP server, OTel exporter, DB pool, log flush?
 - A trace shows a request took 800 ms but your logs show 200 ms latency. What would explain the discrepancy?
 - What breaks first if you deploy without setting `resources.requests`?
-

Source:

21-production-hardening-integration-and-rollout-discipline/05-supply-chain-govulncheck-sbom-and-image-scanning-basics.md

Unit 5 — Supply Chain: govulncheck, SBOM, and Image Scanning

Concept

Supply chain security means knowing what is in your binary and whether any of it has known vulnerabilities. `govulncheck` scans your Go module graph against the Go vulnerability database and reports only vulnerabilities reachable from your code — not just present in your dependencies. An SBOM (Software Bill of Materials) documents every package in your binary. `trivy` or `grype` scans your Docker image for CVEs in OS packages and language dependencies. These three tools answer: what am I shipping, and is any of it known-broken?

Code

```
# 1. Check Go dependencies for known vulnerabilities.
# Only reports vulnerabilities your code actually calls.
govulncheck ./...

# 2. Generate an SBOM in SPDX format from your Go binary.
# Documents every module in the binary.
go version -m ./server > sbom.txt           # simple: module list from binary
syft ./server -o spdx-json > sbom.json      # full SBOM with syft

# 3. Scan your Docker image for CVEs.
trivy image myregistry/api-server:latest

# 4. Scan with grype (alternative to trivy)
grype myregistry/api-server:latest

# Output from trivy looks like:
# CRITICAL: 0 HIGH: 2 MEDIUM: 5 LOW: 12
# For each HIGH/CRITICAL: package name, CVE ID, fixed version.

# 5. Makefile CI target — add to your existing CI step.
# Makefile
.PHONY: ci
ci: test lint govulncheck image-scan

govulncheck:
    govulncheck ./...

image-scan: docker-build
    trivy image --exit-code 1 --severity HIGH,CRITICAL myregistry/api-server:latest
    # --exit-code 1 makes trivy return non-zero on findings → CI fails
```

Exercise

Build: Run the three tools against your API service and its Docker image. **Input:** Your service's Go modules and a built Docker image. **Output:** Clean reports from `govulncheck`, and a `trivy` report showing zero HIGH or CRITICAL findings. **Acceptance:** (1) `govulncheck ./...` exits 0 with no vulnerabilities. If it finds any, upgrade the affected module. (2) `trivy image <your-image>` — fix any HIGH or CRITICAL CVEs by either upgrading the Alpine base image tag or the affected Go dependency. (3) Add `govulncheck ./...` as a step in your Makefile's `ci` target and confirm it runs in CI.

Interview

- What is the difference between a vulnerability being present in a dependency vs being reachable in your code?

- Why does `trivy` find CVEs even in a `scratch`-based image?
 - An SBOM is useful after an incident. Explain how.
-

Phase — 22-backend-security-authn-authz-and-abuse

Directory: 22-backend-security-authn-authz-and-abuse/

Source: 22-backend-security-authn-authz-and-abuse/00-quick-reference.md

Quick Reference — Backend Security

JWT (golang-jwt/jwt/v5)

```
// Sign token := jwt.NewWithClaims(jwt.SigningMethodHS256, jwt.MapClaims{ "sub": userID, "exp":
time.Now().Add(15*time.Minute).Unix(), }) signed, _ := token.SignedString(secret)

// Verify — always check signing method jwt.Parse(signed, func(t jwt.Token) (any, error) { if _, ok := t.Method.
(jwt.SigningMethodHMAC); !ok { return nil, fmt.Errorf("unexpected alg") } return secret, nil })
```

JWT vs session tradeoffs

JWT: stateless, horizontally scalable, hard to revoke Session: stateful (DB/Redis), easy to revoke, simpler security model Refresh tokens: short-lived access token + long-lived refresh token in httpOnly cookie

Rate limiting

```
rate.NewLimiter(rate.Every(time.Second), burstSize) limiter.Allow() // non-blocking limiter.Wait(ctx) // blocking
```

OWASP top risks (backend)

1. Injection (SQL, command) — use parameterized queries
 2. Broken auth — validate JWT alg, use short expiry + refresh
 3. Sensitive data exposure — never log tokens/PII
 4. IDOR — authorize every resource access by ownership
 5. Security misconfiguration — security headers, TLS, no debug in prod
-

Source: 22-backend-security-authn-authz-and-abuse/01-threat-model-and-owasp-backend-mindset.md

Unit 1 — Threat Model and OWASP Backend Mindset

Concept

Threat modeling is listing what an attacker could do at each point in your system and deciding how you prevent it. Start from the data flow: what enters your API, what touches the database, what leaves in responses. The OWASP API Security Top 10 names the most common backend failures: broken authentication, broken object-level authorization (IDOR — user A reads user B's data), injection, and excessive data exposure (returning full DB rows when the client needs two fields). These are not theoretical — they appear in real breach reports.

Code

Threat model for POST /orders

DATA FLOW:

Client → [HTTPS] → API handler → DB insert → DB

THREAT TABLE:

ENDPOINT	THREAT	MITIGATION
POST /orders	Replay attack (same order 2x)	Idempotency key in header
POST /orders	SQL injection in product_id	Parameterized queries only
POST /orders	User places order as other user	Check: order.UserID == auth.UserID
POST /orders	Negative quantity in payload	Validate: quantity > 0
POST /orders	Missing auth token	JWT middleware runs first
GET /orders/:id	IDOR: read another user's order	WHERE user_id = \$authUserID
GET /orders/:id	Enumeration: sequential IDs	Use UUIDs, not integer IDs
Any	Excessive data exposure	Return only fields the client needs

TRUST BOUNDARIES:

- Internet → API: untrusted. Validate everything.
- API → DB: trusted network, but use parameterized queries regardless.
- API → downstream services: authenticated with service tokens.

BLAST RADIUS:

- Compromised user token: attacker can read/modify that user's orders only
- Compromised service token: attacker can read all orders
- SQL injection: full DB read/write

Exercise

Build: Create a threat model table for POST /orders in your e-commerce API. **Input:** Your API's data flow: what the handler receives, what it queries, what it returns. **Output:** A table with at least 6 threat/mitigation pairs covering auth, authorization, input validation, and data exposure. **Acceptance:** For each threat: (1) you can write the specific code or config that implements the mitigation, (2) you can write a test that proves the mitigation works (e.g., a test that sends a tampered user ID and expects 403).

Interview

- What is the difference between authentication and authorization? Give a concrete example where auth passes but authz should fail.
- What is IDOR and why does role-based access control alone not prevent it?
- Why do you use UUIDs instead of integer IDs for order identifiers?

Source: 22-backend-security-authn-authz-and-abuse/02-jwt-vs-session-tradeoffs.md

Unit 2 — JWT vs Session Tradeoffs

Concept

JWT (JSON Web Token): the server signs a token at login; the client sends it on every request; the server verifies the signature without a DB lookup. Stateless and horizontally scalable, but hard to revoke before expiry — a stolen token is valid until it expires. Session: the server stores a session record in Redis or a DB; the client sends a session ID cookie; the server looks it up on every request. Stateful and requires shared storage, but you can revoke a session instantly by deleting the record. Use JWTs with short expiry (15 minutes) plus refresh tokens for stateless APIs. Use sessions when instant revocation is a hard requirement.


```

package main

import (
    "errors"
    "net/http"
    "strings"
    "time"

    "github.com/golang-jwt/jwt/v5"
)

var signingKey = []byte("use-a-real-secret-from-env")

type Claims struct {
    UserID string `json:"user_id"`
    Role   string `json:"role"`
    jwt.RegisteredClaims
}

// IssueToken creates a signed JWT with 15-minute expiry.
func IssueToken(userID, role string) (string, error) {
    claims := Claims{
        UserID: userID,
        Role:   role,
        RegisteredClaims: jwt.RegisteredClaims{
            ExpiresAt: jwt.NewNumericDate(time.Now().Add(15 * time.Minute)),
            IssuedAt:  jwt.NewNumericDate(time.Now()),
        },
    }
    // Always specify the algorithm explicitly – never accept "none".
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString(signingKey)
}

// VerifyToken parses and validates a JWT.
func VerifyToken(tokenStr string) (*Claims, error) {
    token, err := jwt.ParseWithClaims(tokenStr, &Claims{},
        func(t *jwt.Token) (interface{}, error) {
            // Critical: reject tokens with unexpected algorithm.
            if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
                return nil, errors.New("unexpected signing method")
            }
            return signingKey, nil
        })
    if err != nil || !token.Valid {
        return nil, errors.New("invalid token")
    }
    return token.Claims.(*Claims), nil
}

// JWTMiddleware extracts and validates the Bearer token.
func JWTMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        auth := r.Header.Get("Authorization")
        if !strings.HasPrefix(auth, "Bearer ") {
            http.Error(w, "unauthorized", http.StatusUnauthorized)
            return
        }
        claims, err := VerifyToken(strings.TrimPrefix(auth, "Bearer "))
        if err != nil {
            http.Error(w, "unauthorized", http.StatusUnauthorized)
            return
        }
        // Store claims in context for downstream handlers.
        _ = claims
        next.ServeHTTP(w, r)
    })
}

```

```
    })  
}
```

Exercise

Build: Implement JWT authentication middleware for your API service. **Input:** Four test cases run with `go test -v`. **Output:** Each test case produces the expected HTTP status code. **Acceptance:** (1) Valid token → handler called, returns 200. (2) Expired token → 401. (3) Token with tampered payload (change user ID, re-encode without re-signing) → 401. (4) Token signed with a different key → 401. All four cases must be covered by table-driven tests.

Interview

- A user's JWT is stolen. The token has 14 minutes left. What are your options?
 - What attack does checking `t.Method.(*jwt.SigningMethodHMAC)` prevent?
 - Why is a 15-minute access token expiry a security improvement over a 24-hour expiry?
-

Source: `22-backend-security-authn-authz-and-abuse/03-refresh-tokens-rotation-and-revocation.md`

Unit 3 — Refresh Tokens: Rotation and Revocation

Concept

A short-lived access token (15 minutes) means users would have to log in every 15 minutes. Refresh tokens solve this: a long-lived token (7 days) stored in an `httpOnly` cookie is exchanged for a new access token without re-entering credentials. Token rotation: on every refresh, invalidate the old refresh token and issue a new one. If an old refresh token is used after rotation, it means either a replay attack or the token was stolen — revoke all refresh tokens for that user (family compromise). Store refresh tokens in the database so they can be revoked.


```

package main

import (
    "context"
    "crypto/rand"
    "database/sql"
    "encoding/hex"
    "net/http"
    "time"
)

// RefreshToken represents a stored refresh token.
type RefreshToken struct {
    Token      string
    UserID     string
    ExpiresAt  time.Time
}

// generateToken creates a cryptographically random token string.
func generateToken() (string, error) {
    b := make([]byte, 32)
    if _, err := rand.Read(b); err != nil {
        return "", err
    }
    return hex.EncodeToString(b), nil
}

// IssueRefreshToken stores a new refresh token in the DB.
func IssueRefreshToken(ctx context.Context, db *sql.DB, userID string) (string, error) {
    token, err := generateToken()
    if err != nil {
        return "", err
    }
    _, err = db.ExecContext(ctx,
        `INSERT INTO refresh_tokens (token, user_id, expires_at) VALUES ($1, $2, $3)`,
        token, userID, time.Now().Add(7*24*time.Hour))
    return token, err
}

// RotateRefreshToken: validate old token, delete it, issue new one.
// Returns error if token is invalid, expired, or already rotated (replay).
func RotateRefreshToken(ctx context.Context, db *sql.DB, oldToken string) (string, string, error) {
    var rt RefreshToken
    err := db.QueryRowContext(ctx,
        `DELETE FROM refresh_tokens WHERE token = $1 AND expires_at > now()
        RETURNING token, user_id, expires_at`, oldToken).
        Scan(&rt.Token, &rt.UserID, &rt.ExpiresAt)
    if err == sql.ErrNoRows {
        // Token not found: either expired or already used (replay attack).
        // Revoke ALL tokens for this user – family compromise.
        db.ExecContext(ctx, `DELETE FROM refresh_tokens WHERE user_id =
            (SELECT user_id FROM refresh_tokens_audit WHERE token = $1)`, oldToken)
        return "", "", err
    }
    if err != nil {
        return "", "", err
    }
    // Issue new access token and new refresh token.
    accessToken, _ := IssueToken(rt.UserID, "user")
    newRefresh, err := IssueRefreshToken(ctx, db, rt.UserID)
    return accessToken, newRefresh, err
}

// RefreshHandler handles POST /auth/refresh.
func RefreshHandler(db *sql.DB) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {

```



```

    cookie, err := r.Cookie("refresh_token")
    if err != nil {
        http.Error(w, "unauthorized", http.StatusUnauthorized)
        return
    }
    access, refresh, err := RotateRefreshToken(r.Context(), db, cookie.Value)
    if err != nil {
        http.Error(w, "unauthorized", http.StatusUnauthorized)
        return
    }
    http.SetCookie(w, &http.Cookie{
        Name: "refresh_token", Value: refresh,
        HttpOnly: true, Secure: true, SameSite: http.SameSiteStrictMode,
        MaxAge: 7 * 24 * 3600,
    })
    w.Header().Set("Content-Type", "application/json")
    w.Write([]byte(`{"access_token":"` + access + `"}`))
}
}

```

Exercise

Build: Implement the token refresh endpoint with rotation and replay detection. **Input:** Three test cases with an in-memory or test DB. **Output:** Correct HTTP responses for each case. **Acceptance:** (1) Valid refresh token → returns new access token, old refresh token no longer valid. (2) Expired refresh token → 401. (3) Use the same refresh token twice (replay) → second request returns 401, and after the second attempt all refresh tokens for that user are revoked.

Interview

- Why must refresh tokens be stored in the database rather than as JWTs?
- What is a token family, and when do you revoke the whole family?
- Why set the refresh token cookie as `HttpOnly` and `Secure`?

Source: [22-backend-security-authn-authz-and-abuse/04-rbac-authorization-modelling.md](#)

Unit 4 — RBAC: Authorization Modelling

Concept

RBAC (role-based access control) assigns permissions to roles, and roles to users. Middleware checks the user's role before allowing access to a route. RBAC answers "can this role perform this action?" But RBAC alone does not prevent IDOR. IDOR (insecure direct object reference) is a separate check: "does this user own this specific resource?" A user role can call `GET /orders/:id` — but you must also verify that the order belongs to the requesting user, not just that the user has the "user" role.


```

package main

import (
    "context"
    "net/http"
)

type Role string

const (
    RoleAdmin Role = "admin"
    RoleUser  Role = "user"
)

type contextKey string

const claimsKey contextKey = "claims"

type Claims struct {
    UserID string
    Role   Role
}

// RequireRole middleware: reject requests where the user's role
// does not match one of the allowed roles.
func RequireRole(allowed ...Role) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            claims, ok := r.Context().Value(claimsKey).(*Claims)
            if !ok || claims == nil {
                http.Error(w, "unauthorized", http.StatusUnauthorized)
                return
            }
            for _, role := range allowed {
                if claims.Role == role {
                    next.ServeHTTP(w, r)
                    return
                }
            }
            http.Error(w, "forbidden", http.StatusForbidden)
        })
    }
}

// OwnershipCheck: verify the resource belongs to the requesting user.
// Call this INSIDE the handler after fetching the resource from DB.
func requireOwnership(claims *Claims, resourceOwnerID string) error {
    if claims.Role == RoleAdmin {
        return nil // admins can access any resource
    }
    if claims.UserID != resourceOwnerID {
        return &authzError{"not your resource"}
    }
    return nil
}

type authzError struct{ msg string }

func (e *authzError) Error() string { return e.msg }

// Route setup example:
//
// mux.Handle("GET /products", JWTMiddleware(RequireRole(RoleAdmin, RoleUser)(listProducts)))
// mux.Handle("POST /products", JWTMiddleware(RequireRole(RoleAdmin)(createProduct)))
// mux.Handle("DELETE /products", JWTMiddleware(RequireRole(RoleAdmin)(deleteProduct)))
// mux.Handle("GET /orders/{id}", JWTMiddleware(RequireRole(RoleAdmin, RoleUser)(getOrder)))

```

```
//
// Inside getOrder handler:
//  order := fetchOrderFromDB(id)
//  if err := requireOwnership(claims, order.UserID); err != nil {
//      http.Error(w, "forbidden", http.StatusForbidden)
//      return
//  }

func claimsFromContext(ctx context.Context) *Claims {
    c, _ := ctx.Value(claimsKey).(*Claims)
    return c
}
```

Exercise

Build: Add RBAC and ownership checks to your API service. **Input:** Two users in your test DB — one with role `admin`, one with role `user`. Each has separate orders. **Output:** Correct 200/403 responses based on role and ownership. **Acceptance:** (1) Admin can `DELETE /products/:id` → 200. (2) User cannot `DELETE /products/:id` → 403. (3) User can `GET /orders/:id` for their own order → 200. (4) User cannot `GET /orders/:id` for another user's order → 403. (5) Admin can `GET /orders/:id` for any order → 200.

Interview

- Explain the difference between RBAC and ABAC (attribute-based access control). When would you use ABAC?
- A user has role `admin`. Does that automatically prevent IDOR? Why or why not?
- How do you test that an ownership check is not accidentally bypassed?

Source: `22-backend-security-authn-authz-and-abuse/05-api-security-basics-and-hardening-hooks.md`

Unit 5 — API Security Basics and Hardening Hooks

Concept

Rate limiting prevents abuse: a single client cannot exhaust your server or brute-force credentials. `golang.org/x/time/rate` implements a token bucket — each IP gets a bucket refilled at a constant rate; requests that exceed it get 429. Security headers prevent browser-based attacks: `X-Content-Type-Options: nosniff` stops MIME-type sniffing, `X-Frame-Options: DENY` stops clickjacking, `Strict-Transport-Security` enforces HTTPS. Never return stack traces or internal error details to clients — log them server-side, return a generic message to the caller.


```

package main

import (
    "net"
    "net/http"
    "sync"

    "golang.org/x/time/rate"
)

// SecurityHeadersMiddleware adds defensive HTTP headers to every response.
func SecurityHeadersMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("X-Content-Type-Options", "nosniff")
        w.Header().Set("X-Frame-Options", "DENY")
        w.Header().Set("Strict-Transport-Security", "max-age=31536000; includeSubDomains")
        w.Header().Set("Content-Security-Policy", "default-src 'none'")
        w.Header().Set("X-Request-ID", r.Header.Get("X-Request-ID"))
        next.ServeHTTP(w, r)
    })
}

// IPRateLimiter enforces per-IP rate limits.
type IPRateLimiter struct {
    mu      sync.Mutex
    limiters map[string]*rate.Limiter
    r        rate.Limit // tokens per second
    b        int        // bucket size (burst)
}

func NewIPRateLimiter(r rate.Limit, b int) *IPRateLimiter {
    return &IPRateLimiter{
        limiters: make(map[string]*rate.Limiter),
        r:        r,
        b:        b,
    }
}

func (l *IPRateLimiter) limiter(ip string) *rate.Limiter {
    l.mu.Lock()
    defer l.mu.Unlock()
    if lim, ok := l.limiters[ip]; ok {
        return lim
    }
    lim := rate.NewLimiter(l.r, l.b)
    l.limiters[ip] = lim
    return lim
}

// RateLimitMiddleware: 10 requests/second per IP, burst of 20.
func RateLimitMiddleware(limiter *IPRateLimiter) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            ip, _, _ := net.SplitHostPort(r.RemoteAddr)
            if !limiter.limiter(ip).Allow() {
                http.Error(w, "rate limit exceeded", http.StatusTooManyRequests)
                return
            }
            next.ServeHTTP(w, r)
        })
    }
}

// ErrorResponse returns a safe error response – no stack traces, no internals.
func ErrorResponse(w http.ResponseWriter, code int, message string) {
    w.Header().Set("Content-Type", "application/json")
}

```

```
w.WriteHeader(code)
// Generic message only — log the real error server-side.
w.Write([]byte(`{"error":"` + message + `"}`))
}
```

Exercise

Build: Add per-IP rate limiting (10 req/s, burst 20) and security headers to your API service. **Input:** Your running service. **Output:** 429 responses when rate limit is exceeded. Security headers on every response. **Acceptance:** (1) Run `hey -c 1 -q 50 -z 2s http://localhost:8080/api/v1/products` — observe 429 responses in the output. (2) Run `curl -I http://localhost:8080/api/v1/products` — verify `X-Content-Type-Options`, `X-Frame-Options`, and `Strict-Transport-Security` are present. (3) Trigger a handler panic — verify the response body contains no stack trace, only a generic error message.

Interview

- What is the difference between a rate limit and a circuit breaker?
 - Why should `Strict-Transport-Security` only be set over HTTPS, not HTTP?
 - A handler panics in production. What happens to the goroutine serving the request, and how do you prevent it from crashing the entire server?
-

Source: `22-backend-security-authn-authz-and-abuse/06-secret-handling-config-and-least-privilege.md`

Unit 6 — Secret handling in Go services (no printf accidents)

Rules of thumb:

```
load secrets from env / mounted files / secret manager integration pattern for your org
never log secrets—even “temporarily”
rotate without redeploy thrash where possible (still often requires rolling restart practical honesty)
```

Practice: implement config struct with `redacted String()` for logs.

Source: `22-backend-security-authn-authz-and-abuse/07-rate-limiting-and-abuse-patterns.md`

Unit 7 — Rate limiting & abuse resistance (incl. idempotency abuse angles)

Design limits at edge (gateway) vs app (finer grained) trade spectrum.

Include **idempotency key abuse**: unbounded key cardinality, storage DoS, TTL strategy.

Deliverable: sketch policy for `/payments` endpoint bursts + duplicate retry storm friendliness.

Source: `22-backend-security-authn-authz-and-abuse/08-ssrf-and-untrusted-fetching.md`

Unit 8 — SSRF: fetching user-controlled URLs from the backend

Danger: background jobs hitting internal metadata endpoints because user supplied “URL to import.”

Mitigations sketch:

```
allow-lists / block private IP ranges / DNS rebinding awareness high level
separate network egress controls
never forward internal auth headers blindly
```

Interview: give a concrete Go `http.Client` footgun example and how you'd harden a fetcher.

Source: `22-backend-security-authn-authz-and-abuse/09-mutual-tls-internal-trust-model.md`

Unit 9 — mTLS conceptual (internal service-to-service trust)

Explain why east-west traffic sometimes uses **mutual TLS**:

```
service identity via certificates
encryption + authentication combined
often automated via PKI/mesh bridging Areal8 mentions lightly
```

No lab mandate to provision full PKI—conceptual narration + handshake story sufficient unless you choose infra depth willingly.

Source: `22-backend-security-authn-authz-and-abuse/10-capstone-auth-service-security-shaped.md`

Unit 10 — Capstone sketch: minimal `auth-service/` with pragmatic security posture

Compose a deliberately small `auth-service/` exhibiting:

```
issue access token (whatever format chosen—JWT common)
refresh rotation skeleton (even if persisted in sqlite for practice)
RBAC middleware hook on a protected `/admin`-ish route stub
basic rate-limit counter (in-memory acceptable if documented non-prod)
SSRF-safe URL fetch stub OR explicit refusal pattern documented
secrets never logged demonstration test idea (table-driven asserting redaction fields)
```

Document known gaps honestly (no “production-ready” posture pretence without hardening checklist).

Phase — 23-interview-reasoning-and-end-to-end-labs

Directory: 23-interview-reasoning-and-end-to-end-labs/

Source: 23-interview-reasoning-and-end-to-end-labs/00-quick-reference.md

Quick Reference — Interview Gotchas

Concurrency traps

// Loop var capture (pre-1.22): pass as arg or assign tc:=tc // WaitGroup.Add BEFORE go, never inside goroutine // Close channel from SENDER only, never receiver // Closing nil channel panics; sending to closed channel panics

Slice traps

b := a[1:3] // shares backing array — writes affect original copy(out, src) // safe copy — use when you need independence
data[:10] // 10MB backing array still held — GC can't free it

Context traps

// Always defer cancel() immediately after WithTimeout/WithCancel // Check ctx.Done() in long-running goroutines or they leak // Don't store ctx in structs — pass as first parameter

Interface nil trap

var p *MyStruct = nil var i MyInterface = p // i != nil (has type info)! if i == nil { ... } // NEVER true — check the concrete value instead

defer gotcha

for i := 0; i < 5; i++ { defer fmt.Println(i) // all defers run at function exit, LIFO } // prints: 4 3 2 1 0

Common interview questions

1. What happens if you range over nil map? → safe, zero iterations
 2. What happens if you read from nil channel? → blocks forever
 3. Difference between make([]T,0) and var s []T? → both nil-safe, make allocates
 4. When does append allocate? → when len == cap
-

Source: 23-interview-reasoning-and-end-to-end-labs/01-concurrency-interview-drill-and-qa-cards.md

Unit 1 — Concurrency Interview Drill and QA Cards

Concept

Concurrency interview questions test whether you understand goroutine lifecycle, race conditions, and channel semantics — not just syntax. The most common mistakes are: capturing a loop variable by reference (all goroutines see the last value), writing to a map from multiple goroutines without a lock, sending to a closed channel (panic), and acquiring two mutexes in opposite order in two goroutines (deadlock). Predict the behavior before running. Then verify with `-race`.

Code

```
package main

import (
    "fmt"
    "sync"
)

// Gotcha 1: loop variable capture.
// What does this print?
func gotcha1() {
    for i := 0; i < 3; i++ {
        go func() { fmt.Println(i) }() // BUG: all goroutines print 3
    }
    // Fix: pass i as argument
    for i := 0; i < 3; i++ {
        go func(n int) { fmt.Println(n) }(i) // prints 0, 1, 2 (any order)
    }
}

// Gotcha 2: concurrent map write – DATA RACE.
// Does this race?
func gotcha2() {
    m := map[string]int{}
    var wg sync.WaitGroup
    for i := 0; i < 10; i++ {
        wg.Add(1)
        go func(n int) {
            defer wg.Done()
            m["key"] = n // RACE: multiple goroutines write concurrently
        }(i)
    }
    wg.Wait()
    // Fix: use sync.Mutex or sync.Map
}

// Gotcha 3: send to closed channel – PANIC.
// What happens?
func gotcha3() {
    ch := make(chan int, 1)
    close(ch)
    ch <- 1 // panic: send on closed channel
    // Fix: only the sender closes; close after all sends are done.
}

// Gotcha 4: deadlock from lock ordering.
// mu1 then mu2 in one goroutine; mu2 then mu1 in another.
func gotcha4() {
    var mu1, mu2 sync.Mutex
    go func() { mu1.Lock(); mu2.Lock(); mu2.Unlock(); mu1.Unlock() }()
    go func() { mu2.Lock(); mu1.Lock(); mu1.Unlock(); mu2.Unlock() }() // DEADLOCK
    // Fix: always acquire locks in the same order everywhere.
}

func main() {}
```

Exercise

Build: For each of the four gotchas above, predict the output or behavior BEFORE running the code. **Input:** Each snippet run with `go run -race .` or `go test -race -count=100`. **Output:** Your written prediction for each, then the actual result.

Acceptance: (1) Gotcha 1: explain why the loop-variable version prints 3 three times, not 0, 1, 2. (2) Gotcha 2: `go test -race` must report a data race. Fix it and verify the race disappears. (3) Gotcha 3: reproduce the panic, then fix the code so it never panics. (4) Gotcha 4: reproduce the deadlock, then fix it by enforcing lock order. All four fixes must pass `-race`.

Interview

- In Go, is it safe to read a map from multiple goroutines if no goroutine writes?
 - What is the rule for who should close a channel?
 - Two goroutines each hold one lock and wait for the other. What is the name of this condition and what is the standard prevention technique?
-

Source: `23-interview-reasoning-and-end-to-end-labs/02-interface-design-and-refactor-scenarios.md`

Unit 2 — Interface Design and Refactor Scenarios

Concept

Good interfaces are small (1-3 methods), defined near the consumer not the implementation, and named for what the caller needs. An interface with 15 methods is a concrete type wearing a mask — it cannot be implemented by anything other than the original type, so it provides no abstraction benefit. The interface segregation principle: split large interfaces into small ones so that callers depend only on what they use. A service that only reads users should not be forced to depend on methods that write users.

Code

```
package main

// BEFORE: fat interface – hard to mock, forces all implementers to satisfy 8 methods.

type UserRepository interface {
    FindByID(id string) (*User, error)
    FindByEmail(email string) (*User, error)
    ListByRole(role string) ([]*User, error)
    Create(u *User) error
    Update(u *User) error
    Delete(id string) error
    UpdatePassword(id, hash string) error
    RecordLogin(id string) error
}

// AFTER: three focused interfaces, each 1-3 methods.
// Each service depends only on what it needs.

// UserReader: for services that only query users.
type UserReader interface {
    FindByID(id string) (*User, error)
    FindByEmail(email string) (*User, error)
}

// UserWriter: for services that create or modify users.
type UserWriter interface {
    Create(u *User) error
    Update(u *User) error
    Delete(id string) error
}

// UserAuthStore: for the auth service specifically.
type UserAuthStore interface {
    FindByEmail(email string) (*User, error)
    UpdatePassword(id, hash string) error
    RecordLogin(id string) error
}

// Services declare the interface they need – not the full UserRepository.

type OrderService struct {
    users UserReader // only needs to look up users
}

type AdminService struct {
    users interface { // compose: needs both read and write
        UserReader
        UserWriter
    }
}

type AuthService struct {
    users UserAuthStore // auth-specific operations only
}

type User struct {
    ID    string
    Email string
    Role  string
}
```

Exercise

Build: Take the `UserRepository` interface in your project and apply the split shown above. **Input:** Your existing codebase with one large `UserRepository` interface. **Output:** Three focused interfaces. All call sites updated to depend on the specific interface they need. **Acceptance:** (1) `OrderService` uses only `UserReader` — confirm it does not import `UserWriter` methods. (2) `AuthService` uses only `UserAuthStore`. (3) Each focused interface has a test mock that is under 30 lines (fat interfaces generate huge mocks). (4) `go build ./...` passes with no errors.

Interview

- Why should interfaces be defined by the consumer, not the implementer?
 - You have a `Logger` interface with `Debug`, `Info`, `Warn`, `Error`, `Fatal`, `With`, `WithContext`. How would you split it?
 - What is the difference between interface embedding (composition) and interface inheritance?
-

Source: `23-interview-reasoning-and-end-to-end-labs/03-context-cancellation-timeout-and-leak-fix.md`

Unit 3 — Context Cancellation, Timeout, and Leak Fix

Concept

A goroutine leak is a goroutine that blocks forever because no one reads from its channel or because it does not check `ctx.Done()`. Leaks accumulate silently — the goroutine count grows with every request and never falls. Use `runtime.NumGoroutine()` in tests to detect leaks: record the count before, run the operation, wait briefly, check the count after. The fix is always the same: every goroutine that blocks must select on `ctx.Done()` so it exits when the caller cancels.


```

package main

import (
    "context"
    "fmt"
    "runtime"
    "time"
)

// LEAKY: goroutine blocks forever if ctx is cancelled before work completes.
func leakyWorker(ctx context.Context, jobs <-chan int) {
    go func() {
        for job := range jobs { // blocks here if jobs is never closed
            fmt.Println("processing", job)
        }
    }()
}

// FIXED: goroutine exits when ctx is cancelled.
func fixedWorker(ctx context.Context, jobs <-chan int) {
    go func() {
        for {
            select {
            case job, ok := <-jobs:
                if !ok {
                    return // channel closed – exit cleanly
                }
                fmt.Println("processing", job)
            case <-ctx.Done():
                return // context cancelled – exit cleanly
            }
        }
    }()
}

// TestNoGoroutineLeak shows the test pattern.
// In a real test: use testing.T and a shorter wait.
func TestNoGoroutineLeak() {
    before := runtime.NumGoroutine()

    jobs := make(chan int, 10)
    ctx, cancel := context.WithCancel(context.Background())

    fixedWorker(ctx, jobs)
    jobs <- 1
    jobs <- 2

    // Cancel context – goroutine should exit.
    cancel()

    // Give goroutine time to exit.
    time.Sleep(10 * time.Millisecond)

    after := runtime.NumGoroutine()
    if after > before {
        fmt.Printf("LEAK: goroutine count went from %d to %d\n", before, after)
    } else {
        fmt.Println("OK: no goroutine leak")
    }
}

func main() {
    TestNoGoroutineLeak()
}

```

Exercise

Build: Write a test that verifies your worker pool has no goroutine leak. **Input:** Your worker pool implementation from a previous module. **Output:** A `TestWorkerPoolNoLeak` test that passes with `go test -race`. **Acceptance:** (1) Record goroutine count before starting the pool. (2) Start the pool, send 10 jobs, cancel the context. (3) `time.Sleep(100 * time.Millisecond)` to allow cleanup. (4) Assert goroutine count has returned to the baseline (± 2 for test infrastructure). (5) The test must also pass with `go test -race` — no races during shutdown.

Interview

- How do you detect a goroutine leak in a test?
 - A goroutine is stuck waiting on a channel that will never receive. What are your two options for fixing this?
 - Why does `context.WithTimeout` not guarantee the goroutine exits after the timeout?
-

Source: `23-interview-reasoning-and-end-to-end-labs/04-memory-model-slices-maps-and-retention.md`

Unit 4 — Memory Model: Slices, Maps, and Retention

Concept

A slice header is three fields: a pointer to the backing array, a length, and a capacity. When you take a subslice (`s[0:10]`), the new slice shares the same backing array as the original. If the original was a 100 MB buffer, the subslice keeps all 100 MB alive in memory even though you only use 10 bytes. The fix is `copy`: allocate a new slice with exactly the size you need and copy the data in — the original backing array can then be garbage collected. This is a common, silent memory leak in code that processes large buffers.


```

package main

import (
    "fmt"
    "runtime"
)

// LEAKY: subslice retains the entire backing array.
func firstTenLeaky(large []byte) []byte {
    return large[0:10] // shares backing array – all 100 MB retained
}

// FIXED: copy into a new, small slice – backing array can be GC'd.
func firstTenSafe(large []byte) []byte {
    result := make([]byte, 10)
    copy(result, large[0:10])
    return result
}

// Slice aliasing gotcha: modifying a subslice modifies the original.
func aliasDemo() {
    original := []int{1, 2, 3, 4, 5}
    sub := original[1:3] // sub = [2, 3], shares backing array
    sub[0] = 99
    fmt.Println(original) // [1 99 3 4 5] – original modified!

    // Fix: copy when you need an independent slice.
    safe := make([]int, 2)
    copy(safe, original[1:3])
    safe[0] = 42
    fmt.Println(original) // [1 99 3 4 5] – original unchanged
}

func memStats() uint64 {
    var ms runtime.MemStats
    runtime.GC()
    runtime.ReadMemStats(&ms)
    return ms.HeapAlloc
}

func main() {
    large := make([]byte, 100*1024*1024) // 100 MB

    before := memStats()

    leaky := firstTenLeaky(large)
    large = nil // clear original reference
    runtime.GC()
    afterLeaky := memStats()
    fmt.Printf("leaky:  heap after GC = %d MB (large array still alive)\n",
        afterLeaky/1024/1024)

    _ = leaky
    leaky = nil
    safe := firstTenSafe(make([]byte, 100*1024*1024))
    large = nil
    runtime.GC()
    afterSafe := memStats()
    fmt.Printf("safe:   heap after GC = %d MB (large array collected)\n",
        afterSafe/1024/1024)

    _ = safe
    _ = before
    aliasDemo()
}

```

Exercise

Build: Write a benchmark that measures heap allocation for the leaky vs safe approach. **Input:** A `large` buffer of 100 MB. **Output:** `go test -benchmem` output showing allocation difference. **Acceptance:** (1) `BenchmarkLeaky` retains ~100 MB after GC (verify with `runtime.ReadMemStats`). (2) `BenchmarkSafe` retains ~10 bytes after GC. (3) Modify a subslice and verify (with a test assertion) that the original is changed — then apply the `copy` fix and verify the original is not changed.

Interview

- You extract a small prefix from a large buffer and store it in a cache. After 1000 requests, your heap is 10 GB. What is likely happening?
 - What is the difference between `len` and `cap` of a slice?
 - When does `append` allocate a new backing array vs reuse the existing one?
-

Source: `23-interview-reasoning-and-end-to-end-labs/05-race-detector-and-profiling-combined-incident.md`

Unit 5 — Race Detector and Profiling: Combined Incident

Concept

Real incidents combine multiple issues. A slow endpoint under load triggers both a data race and a memory spike. Fix the race first — a race is a correctness bug and can produce data corruption or crashes. Then address the allocation — a performance problem. The race detector (`-race`) finds races at runtime by instrumenting memory accesses. The benchmark `-benchmem` flag shows allocations per operation. Fix the race with a mutex or channel. Fix the allocation by reusing objects or reducing copies.

Code

```
package main

import (
    "net/http"
    "sync"
)

// BUGGY handler: shared map with no lock – DATA RACE.
// Also allocates a new map on every request – UNNECESSARY ALLOCATION.

var requestCounts map[string]int // shared, unprotected

func countHandler(w http.ResponseWriter, r *http.Request) {
    requestCounts[r.URL.Path]++ // RACE: concurrent reads and writes
    w.WriteHeader(http.StatusOK)
}

// FIXED handler: mutex protects the map, same map reused across requests.

type Counter struct {
    mu      sync.Mutex
    counts map[string]int
}

func NewCounter() *Counter {
    return &Counter{counts: make(map[string]int)}
}

func (c *Counter) Handler(w http.ResponseWriter, r *http.Request) {
    c.mu.Lock()
    c.counts[r.URL.Path]++
    c.mu.Unlock()
    w.WriteHeader(http.StatusOK)
}

// To reproduce the race:
// go test -race -count=100 -run TestCountHandler
//
// The race detector will report:
// WARNING: DATA RACE
// Write at 0x... by goroutine N:
// Previous read at 0x... by goroutine M:
//
// To find the allocation hot spot:
// go test -bench=BenchmarkCountHandler -benchmem
// Look for allocs/op > 0 in the hot path.
//
// After fixing:
// go test -race -count=100 -run TestCountHandler → no race reported
// go test -bench=BenchmarkCountHandler -benchmem → allocs/op should drop

func main() {
    requestCounts = make(map[string]int)

    counter := NewCounter()

    mux := http.NewServeMux()
    mux.HandleFunc("/buggy", countHandler) // has race
    mux.HandleFunc("/fixed", counter.Handler) // race-free

    http.ListenAndServe(":8080", mux)
}
```

Exercise

Build: Reproduce the data race in `countHandler`, fix it, then find and fix the allocation hot spot. **Input:** The buggy `countHandler` above. **Output:** A fixed handler that passes `go test -race -count=100` and shows reduced `allocs/op` in benchmarks. **Acceptance:** (1) Write a test that sends 100 concurrent requests to `countHandler`. Run with `-race` — the race detector must fire. (2) Apply the `Counter` fix. Run with `-race` again — no race reported. (3) Write a benchmark for the fixed handler. Run with `-benchmem`. Identify if there are unnecessary allocations in the hot path and eliminate them. (4) Final state: `-race` clean, `allocs/op` is 0 or minimized in the benchmark.

Interview

- The race detector does not find all races. What kind of race does it miss?
- You see `allocs/op: 48` in a benchmark for a simple counter increment. What are the likely causes?
- A race causes a map to panic with “concurrent map read and map write” in production. The race detector was not enabled. How do you reproduce and confirm the fix before deploying?

Source: `23-interview-reasoning-and-end-to-end-labs/06-distributed-scenario-payment-inventory-failure-chain.md`

Unit 6 — Distributed systems scenario lab: payment succeeds, inventory fails

Narrate and document:

```
retry policy (when safe vs dangerous)
idempotency & dedupe story
queue handoff / saga compensation framing (Areas 12/14)
eventual consistency symptom vs bug distinction
```

Deliverable: architectural note + sequence outline (textual fine) + explicit **open questions** you’d ask stakeholders.

Source: `23-interview-reasoning-and-end-to-end-labs/07-production-incident-triage-script.md`

Unit 7 — Production incident triage scripting (latency + errors + resource pressure)

Incident recipe:

```
Start from user-visible symptom
Query metrics hypotheses (SATURATION? ERRORS?)
Drill traces for slow spans
Align logs via correlation identifiers
Iterate until single dominant factor emerges OR honest multi-factor story accepted
```

Deliverable

Template you can reuse: fields for timeline, hypotheses, proofs, rollout.

Practice with synthetic DB slowdown + retry amplification scenario.

Source: 23-interview-reasoning-and-end-to-end-labs/08-architecture-whiteboard-payment-platform.md

Unit 8 — Architecture whiteboard: payment platform services

Whiteboard (paper/digital) these services and their interactions:

```
API
payment
inventory
worker
notification
```

Annotate:

```
queue usage
retry/timeouts ownership
idempotency surfaces
observability hooks
data ownership boundaries (who owns which tables/events)
```

Voice-over ≤5 minutes without reading notes—record yourself if helpful.

Source: 23-interview-reasoning-and-end-to-end-labs/09-enterprise-scale-stress-questions-first-drill.md

Unit 9 — Enterprise-scale scenario drill (qualitative “100M orders” stress)

Given stylised scale signals:

```
worker slow, queue backlog growing, latency rising, memory rising
stack includes Go API + Redis + Postgres + queue
```

You must **ask questions first** (data shape, traffic pattern, hot keys, DB indexes, consumer lag, GC, retry storms) before proposing fixes.

Deliverable: structured Q&A + ranked hypotheses + measurement plan. No “rewrite the whole platform” without stating what evidence would justify it.

Source: 23-interview-reasoning-and-end-to-end-labs/10-final-staff-composite-payment-platform-lab.md

Unit 10 — Final staff-style lab: `payment-platform/` under composite failures

Inject simultaneously (or staged if overwhelmed—document staging honestly):

```
duplicate payment attempts
timeouts & ambiguous responses
queue backlog growth
goroutine leak
CPU pegged profile
latency SLO breach symptoms
worker slowdown
```

You must produce **one consolidated incident report** including:

```
problem statement
symptoms timeline
investigation evidence (race? pprof? metrics? traces? logs?)
root cause(s)
fix + trade-offs
prevention / guardrails
```

Explicitly cross-link reasoning to earlier areas (concurrency, distributed, prod, perf) in the narrative—this is integration of *thinking*, not isolated trivia.
